软件工程课程

内容总结

王鸽

大纲

- 软件工程概念
- 项目立项
- 需求分析
- 系统设计
- 系统实现
- 软件测试

大纲

- 软件工程概念
- 项目立项
- 需求分析
- 系统设计
- 系统实现
- 软件测试

软件工程概念

- **软件**是指在计算机系统中运行的程序、程序所处理的数据以及与程序开发、使用和维护相关的文档的集合。
- **软件危机**是指在软件开发和维护过程中，由于开发方法落后、管理不完善以及软件系统复杂度不断提高而产生的一系列困难和问题，主要表现在多个方面：
  - 软件开发成本增加
  - 软件开发周期过长
  - 软件质量难以保证
  - 软件维护工作困难
  - 软件开发效率低下

软件工程概念

- **软件生命周期**涵盖项目计划、需求分析、系统设计、编码、测试、维护等核心活动：

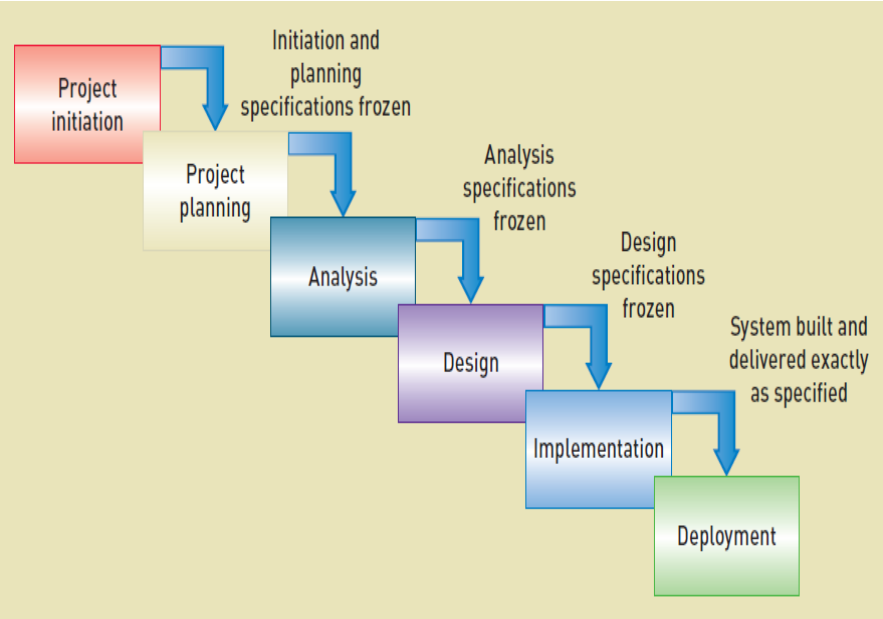
1. **项目立项**：该阶段核心目标是回答“系统是否具备可行性”及“若可行，项目计划如何”。
2. **需求分析**：需求分析阶段的核心任务是准确界定“系统必须做什么”。
3. **系统设计**：系统设计分为概要设计与详细设计两部分，其中，**概要设计**聚焦于回答“系统如何设计”的问题。**详细设计**属于模块内部的结构级设计，是对概要设计的精细化拆解。
4. **编码**：编码阶段的核心任务是将设计阶段的过程描述转化为计算机可执行的程序代码。
5. **测试**：**测试**是保障软件质量的关键手段，通过设计测试用例对软件各组成部分进行系统性验证。
6. **维护**：维软件维护的动因主要包括三类：修复运行中发现的潜在缺陷、适应软件运行环境的变化、响应用户业务变更带来的功能扩展与增强需求

软件开发生命周期模型

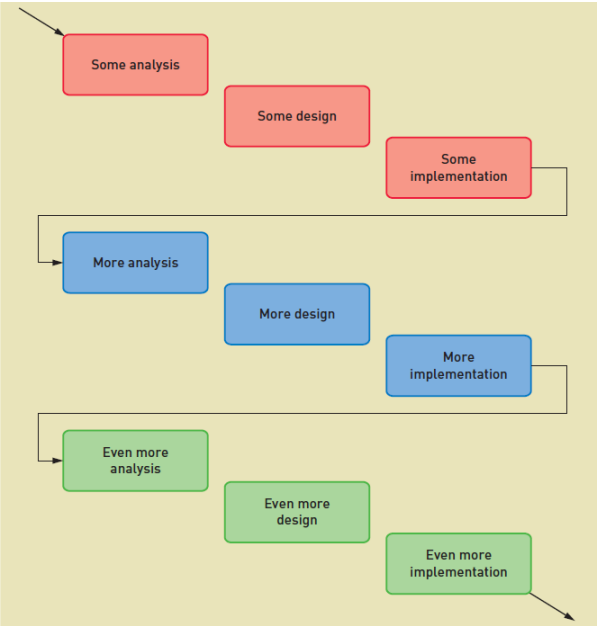
分类:

- 预测型模型:
  - 瀑布模型（Waterfall Model）
- 适应性模型
  - 迭代模型（Iterative Model）
  - 螺旋模型（Spiral Model）

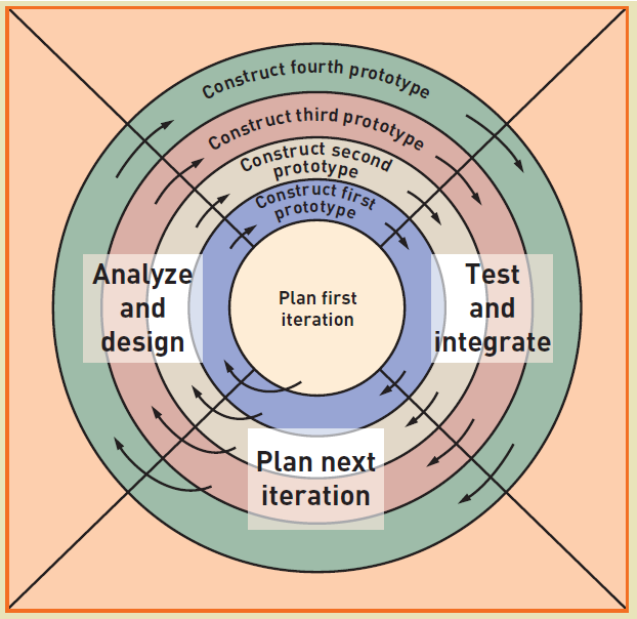
软件开发生命周期模型



瀑布模型：一蹴而就



迭代模型



螺旋模型

软件开发生命周期模型

- **瀑布模型**:
- **特点：**瀑布模型严格遵循软件生存周期各阶段的目标、任务规范与质量要求开展开发活动，该模型尤为强调开发前期需求定义的完备性与准确性，通过强化各阶段活动的严谨管控，力求规避开发后期修正所带来的高昂成本。
- **优点：**传统瀑布模型为软件产业发展提供了基础性框架，部分缓解了早期软件危机
- **缺点：**本质上属于线性顺序模型，适应性差
- 适用：简单功能与性能明确、完整且无重大需求变更的软件开发场景

软件开发生命周期模型

- **迭代模型**:
- **特点：**增量模型又称为渐增模型，也称为有计划的产品改进模型，它从一组给定的需求开始，通过构造一系列可执行中间版本来实施开发活动。
- 增量模型和瀑布模型之间存在本质区别：瀑布模型属于整体开发模型，规定在开始下一阶段工作之前，必须完成前一阶段的所有细节；而增量模型属于非整体开发模型，它推迟某些阶段或所有阶段中的细节实现，从而较早地产生可工作的软件。
- **优点：**快速交付、适应变化、降低风险
- **缺点：**可能不适用于某些类型的软件、需要更多的沟通和协作、系统可能不稳定
- **适用：**复杂的软件系统

软件开发生命周期模型

- **螺旋模型**:
- **特点：**螺旋模型将开发过程拆解为多个螺旋周期，每个周期的阶段划分与瀑布模型高度契合
- **优点：全过程风险管理、阶段质量保障、项目价值验证**
- **缺点：**高度依赖风险评估经验、迭代次数难以控制
- **适用**：复杂、高风险、需求不明确项目的系统性开发框架

软件开发方法

- **结构化方法：**1968年，Edsger Dijkstra 提出“结构化编程”思想，强调程序应避免使用无条件跳转语句（如GOTO），转而采用**顺序、选择、循环三种基本控制结构**
- **面向对象方法：**面向对象开发方法是一种以“对象”为中心、**模拟人类认知世界方式**的软件开发方法论。其核心理念是将现实世界中的实体及其相互关系直接映射为软件系统中的对象及对象间的关系，从而缩小问题空间与解决方案空间之间的鸿沟。

软件开发方法

- **结构化方法：**1968年，Edsger Dijkstra 提出“结构化编程”思想，强调程序应避免使用无条件跳转语句（如GOTO），转而采用顺序、选择、循环三种基本控制结构

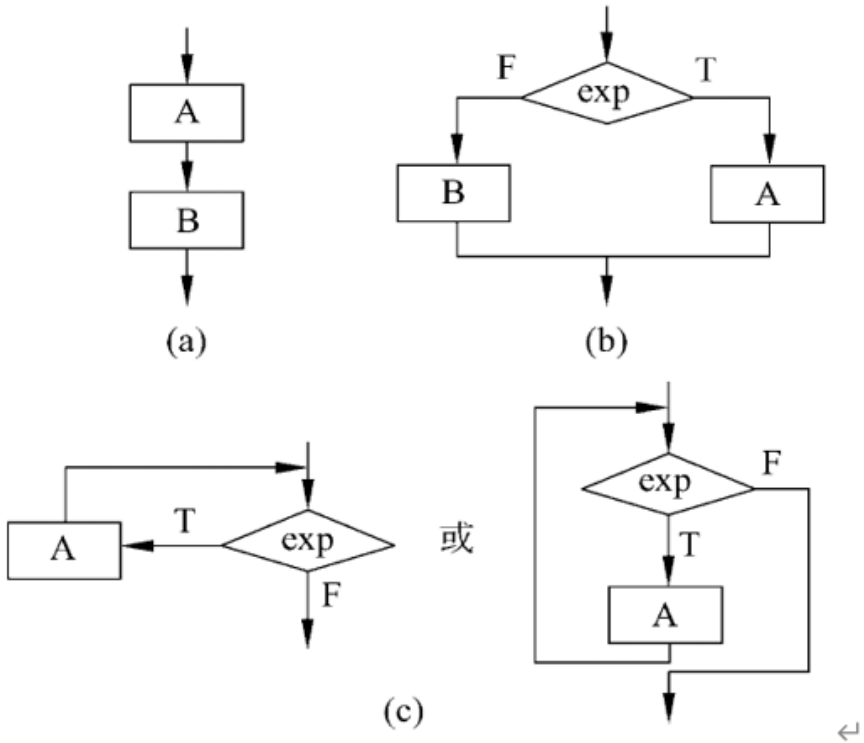


图 1-7 结构化方法的三种结构：（a）顺序；（b）选择；（c）循环

软件开发方法

- **面向对象开发方法**是一种以“对象”为中心、模拟人类认知世界方式的软件开发方法论。其核心理念是将现实世界中的实体及其相互关系直接映射为软件系统中的对象及对象间的关系，从而缩小问题空间与解决方案空间之间的鸿沟。
- 对象：对象是面向对象方法中最基本的运行实体，它是一个封装了数据（属性）和操作（方法）的独立单元，能够对外提供特定的服务。
- 封装：封装是指将对象的属性与操作绑定在一起，并隐藏对象内部实现细节的机制。
- 继承是类之间的一种层次关系，它允许一个子类共享父类的属性和操作。
- 多态是指在继承关系中，同一操作在不同子类中可以实现不同的行为。

例题

- 以下哪一项不是软件危机的表现形式( )
- A. 开发的软件不满足用户需要　　B. 开发的软件可维护性差
- C. 开发的软件价格便宜　　D. 开发的软件可靠性差

答案：C

【选择】一个软件项目是否进行开发的结论是在（　）文档中作出的。
A．软件计划
B．可行性报告
C．测试报告
D．需求分析说明书

- 答案：B

【选择】编码阶段是位于哪个阶段之后( )
A. 详细设计
B. 可行性研究
C. 总体设计
D. 需求分析

- 答案： A

【选择】模块内部的算法设计在结构化方法的（　）阶段进行。
A．系统分析
B．概要设计
C．详细设计
D．可行性分析

- 答案：C

【选择】需求分析阶段的任务是确定（ ）
A．软件开发方法
B．软件开发工具
C．软件开发费
D．软件系统的功能

- 答案： D

例题

- 瀑布模型本质上是一种(　)。
- A．线性顺序模型　　B．顺序迭代模型
- C．线性迭代模型　　D．及早见产品模型

答案：A

例题

- 瀑布模型表达了一种系统的、顺序的软件开发方法。以下关于瀑布模型的叙述中，正确的是(　)。
- A．瀑布模型能够非常快速地开发大规模软件项目
- B．只有很大的开发团队才可使用瀑布模型
- C．瀑布模型已不再适合于现今的软件开发环境
- D．瀑布模型适用于软件需求确定、开发过程能够采用线性方式完成的项目

答案：D

例题

- 瀑布模型的关键不足在于( )
- A. 过于简单　　B. 各个阶段需要进行评审
- C. 过于灵活　　D. 不能适应需求的动态变更

答案：D

【选择】（ ）是把对象的属性和操作结合在一起，构成一个独立的对象，其内部信息对外界是隐蔽的，外界只能通过有限的接口与对象发生联系。
A 封装
B 继承
C 多态性
D 消息

- 答案：A

【选择】程序的三种基本控制结构是:

A．过程、子程序和分程序

B．顺序、选择和重复

C．递归、堆栈和队列

D．调用、返回和转移

- 答案：B

【选择】程序的三种基本控制结构的共同特点是:
A．不能嵌套使用
B．只能用来写简单程序
C．已经用硬件实现
D．只有一个入口和一个出口

- 答案：D

【简答】请阐述结构化方法（Structured architectural）和面向对象方法（Object-oriented architectural）的不同点

- 答案:
- 结构化方法注重控制流程和数据流程的设计，主要通过顺序、选择和循环等结构来完成程序的开发;
- 面向对象方法将数据和方法封装在类的内部,通过类的实例化来创建对象。

【简答】请阐述瀑布模型、螺旋模型和敏捷开发的特点和适用的系统

- 答案:
- 瀑布模型：每个步骤依次进行，适合需求明确、简单的系统
- 螺旋模型：每个步骤依次进行，完成一轮迭代后进入下一轮迭代，适合需求尚不完备、需要与用户多次沟通、功能较为复杂的系统
- 敏捷开发：迭代进行，但步骤间没有明确的先后关系，适合用户需求变动频繁的系统

大纲

- 软件工程概念
- 项目立项
- 需求分析
- 系统设计
- 系统实现
- 软件测试

可行性分析

- **可行性研究的任务：**可行性研究的根本任务是在软件项目正式启动之前，从多个维度系统评估项目的可行性与投资价值，为管理层的项目决策提供科学依据。
- **其核心目标是回答三个关键问题：**
  - 技术可行性：该项目在现有技术条件下是否能够实现？
  - 经济可行性：该项目在经济上是否值得投资？
  - 社会可行性：该项目在法律法规和社会伦理层面是否存在障碍？

项目计划

- **项目计划的任务：**一个完整的软件项目计划通常包括项目范围、项目资源、进度安排、成本估算、培训计划以及项目里程碑等内容。这些内容共同构成了软件项目实施的总体规划。

项目计划

- **软件规模估算：**
- 1）代码行估算
- 将系统按功能模块进行分解，直到每个模块的规模可以较为准确地估算。
- 2）功能点估算
- 将系统功能划分为五类基本组件——外部输入、外部输出、外部查询、内部逻辑文件、外部接口文件。根据各类组件的数量及其复杂度权重，计算出未调整功能点数。

项目计划

- **软件工作量估算：**
- COCOMO模型：COCOMO模型将项目类型分为三类：
- （1）组织型：相对较小、较简单的软件项目。
- （2）嵌入型：要求在紧密联系的硬件、软件和操作的限制条件下运行，通常与某种复杂的硬件设备紧密结合在一起。
- （3）半独立型：介于上述两种软件之间，规模和复杂度都属于中等或更高，最大可达30万行。

项目计划

- **软件工作量估算：**
- **基本COCOMO：**基本COCOMO是一个静态单变量模型，它用一个已估算出来的原代码行数为自变量的经验函数计算软件开发工作量，计算公式为：

$$E = a \times (KLOC)^b,$$

其中，E是工作量（人月），KLOC为估计提交的代码行数，a和b为根据项目类型确定的常数，取值可参考表 2 1。根据工作量，可以估算开发时间，计算公式如下：

$$D = c \times (E)^d$$

其中D为所需的开发时间（月），c和d为根据项目类型确定的常数

表2-1 基本COCOMO模型参数举例

方 式	a	b	c	d
组织型	2.4	1.05	2.5	0.38
半独立型	3.0	1.12	2.5	0.35
嵌入型	3.6	1.2	2.5	0.32

项目计划

- **软件工作量估算：**
- 生产率及所需人员数的公式：

$$\text{生产率} = \frac{KLOC}{E},$$

$$\text{人员数} = \frac{E}{D},$$

例题

- 可行性研究的根本任务是？
- A. 确定项目预算
- B. 评估项目是否值得做、是否能够做
- C. 制定项目进度计划
- D. 确定项目团队

答案：B

例题

- 社会可行性分析主要评估？
- A. 项目是否盈利
- B. 技术是否成熟
- C. 是否符合法律法规和社会伦理
- D. 进度是否可控

答案：C

例题

- 以下哪项不属于技术可行性分析的内容？
- A. 技术成熟度评估
- B. 资源可用性分析
- C. 市场需求预测
- D. 技术风险识别

答案：C

例题

- 以下哪项不属于技术可行性分析的内容？
- A. 技术成熟度评估
- B. 资源可用性分析
- C. 市场需求预测
- D. 技术风险识别

答案：C

例题

- 某软件公司计划开发一个“企业进销存管理系统”，项目类型为半独立型。经需求分析，估算该系统的代码规模为25,000行（即25KSLOC）。采用COCOMO 基本模型进行工作量估算。

- 已知COCOMO 81基本模型的工作量计算公式为：工作量 = a × (代码行数)^b（单位：人月）

表2-1 基本COCOMO模型参数举例

方 式	a	b
组织型	2.4	1.05
半独立型	3.0	1.12
嵌入型	3.6	1.2

- （1）计算该项目的工作量（单位：人月），结果保留一位小数

- （2）如果该项目需要投入8名开发人员，估算开发时间（单位：月），结果保留一位小数

- （3）假设该公司历史项目的平均生产率为500行/人月，不考虑其他因素，按此生产率估算工作量应为多少？与COCOMO模型估算结果相比有何差异？请分析差异产生的原因。

例题

- 答案：（1）工作量估算
- （注意单位换算：25,000行 = 25 KSLOC，使用25代入公式，不要代入25000）
- 代入表格中半独立型公式a=3.0，b=1.12

$$E = a \times (KLOC)^b,$$

- 计算可得E=110.4人月
- （2）开发时间估算：工作量=110.4人月，团队人数N=8，因此开发时间T=110.4/8=13.8月

例题

- （3）计算生产率：规模=25000行，生产率=500行/人月，生产率=25000/500=50人月
- COCOMO模型估算为110.4人月，生产率估算为50人月，差距产生的原因可能为：

1）项目类型因素：进销存管理系统属于半独立型，系统集成复杂度较高，需要更多沟通和协调成本，而生产率估算忽略了项目类型的影响

2）工作量组成部分：COCOMO模型估算的是项目总工作量，包括需求分析、设计、编码、测试、文档编写等全部活动；生产率估算通常仅考虑编码阶段

大纲

- 软件工程概念
- 项目立项
- 需求分析
- 系统设计
- 系统实现
- 软件测试

需求分析

- **需求分析概念：**需求分析是指在软件开发初期，通过调查、沟通、归纳、建模和确认等手段，对用户需求、业务需求和系统需求进行识别、理解、整理、细化与表达的过程。
- **需求分析分类（两种划分方式）：**
  - 功能性需求和非功能性需求：功能性需求是指系统必须提供的功能、服务和行为，非功能性需求是指系统在实现功能的同时必须满足的质量要求和约束条件
  - 用户需求和系统需求：用户需求是从用户或业务使用者角度提出的需求，用来说明用户希望系统提供哪些支持、完成哪些任务以及达到什么目标。系统需求是在用户需求基础上进一步细化和规范化形成的需求描述，用来更准确地说明系统应具备的功能、性能、接口和约束条件。

需求分析

- **利益相关者：**利益相关者是指所有对软件系统有利益诉求、能够影响系统开发过程或受系统运行结果影响的个人、群体或组织。
- **两种分类（两种划分方式）：**
  - 操作利益相关者与执行利益相关者：操作利益相关者是指直接使用软件系统或与系统进行交互的个人或群体。执行利益相关者是指不直接使用系统，但其决策、管理或监督活动对系统有重要影响的个人或群体。
  - 内部利益相关者与外部利益相关者：内部利益相关者是指项目组织内部的人员或群体，他们属于开发团队所在的组织，或属于客户组织内部。外部利益相关者是指项目组织外部的个人或群体，他们与项目没有直接的雇佣关系，但其利益可能受到项目的影响，或能够影响项目的实施。

需求分析

- **利益相关者举例：**
- 在一个医院信息管理系统中，护士是典型的操作利益相关者。
- 护理部主任是典型的执行利益相关者。
- 医院的IT部门人员是内部利益相关者。
- 卫生健康委员会（卫健委）是重要的外部利益相关者。

需求分析

- **利益相关者举例：**
- 在医院信息管理系统中，临床护士属于内部操作利益相关者。
- 护理部主任、医院IT部门主管、院长均属于内部执行利益相关者。
- 患者（或其家属）通过移动端应用查看检验报告、预约挂号、在线咨询，属于外部操作利益相关者。
- 卫生健康委员会是典型的外部执行利益相关者

需求分析

- **利益相关者举例：**
- 在医院信息管理系统中，临床护士属于内部操作利益相关者。
- 护理部主任、医院IT部门主管、院长均属于内部执行利益相关者。
- 患者（或其家属）通过移动端应用查看检验报告、预约挂号、在线咨询，属于外部操作利益相关者。
- 卫生健康委员会是典型的外部执行利益相关者

需求分析

利益相关者举例：

例子：上市公司会计系统的利益相关者

内部-操作相关者：出纳，会计，内部审计师
内部-执行相关者：高级经理，董事会
Both：业务经理

外部-操作相关者：合作伙伴组织；客户
外部-执行相关者：投资者，监管机构
Both：外部审计师



需求分析

- **需求建模方法：**
  - 结构化建模方法：
    - 数据流图、数据字典、结构化语言、判定表、判定树
  - 面向对象需求建模方法：
    - 用例图、活动图、系统顺序图、实体-联系图、域模型类图

需求分析

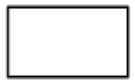
结构化建模方法:

- 数据流图
- 数据字典
- 结构化语言
- 判定表
- 判定树

需求分析

- 数据流图

表 3-1　　数据流图中的基本图形符号

符号	说明
	圆或椭圆。表示加工，也称数据处理，它对数据流进行某些操作或变换。应在圆圈内写上加工名字（一般为动词短语），在分层数据流图中，加工还应编号
	矩形。表示数据流的源点和终点，是软件系统外部环境中的实体（包括人员、组织或其他软件系统），统称外部实体。应在方框内写上相应的名称
	双杠。表示文件或称数据存储，暂时保存数据，它可以是数据库文件或任何形式的数据组织。指向存储的数据可理解为写入，从存储引出的数据可理解为读出。要用名词或名词短语进行命名
	箭头。表示数据流，由一组成分固定的数据项组成。数据流必须有流向，箭头表示数据流动的方向，即在加工之间、加工与源点/终点之间、加工与数据存储之间流动。除与数据存储之间的数据流不用命名外，其他数据流均需用名词或名词短语命名

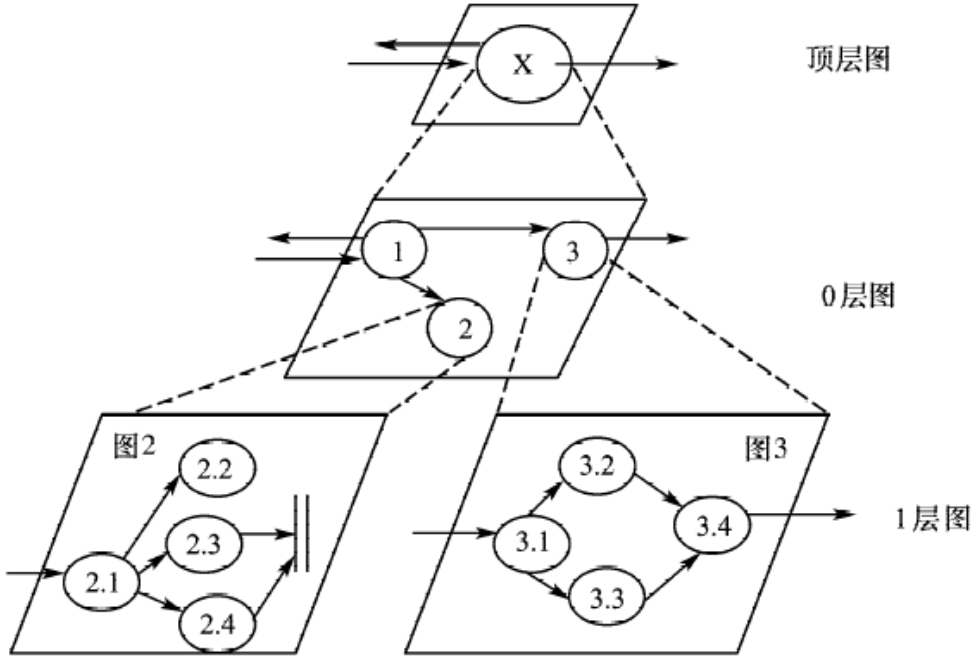
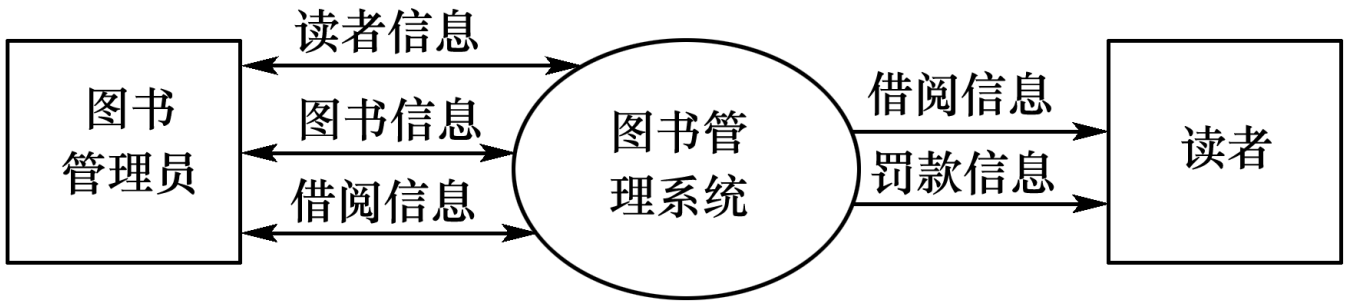


图 3-2　系统分层数据流图

需求分析

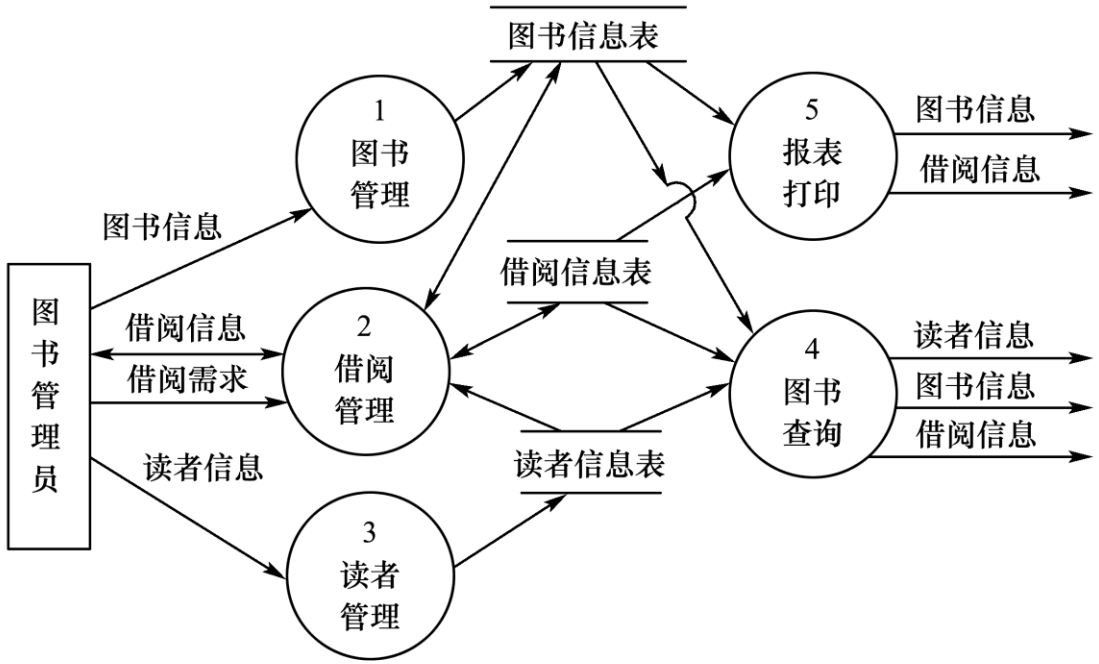
- 数据流图
- **在图书馆负责人和计算机系的技术人员通过了“图书管理系统”《项目开发计划》后，项目组随即进入了项目开发阶段，计算机系教师与图书馆相关业务人员紧密合作，经过15天的工作，形成了“图书管理系统”《软件需求说明书》。主要数据流图如下：**



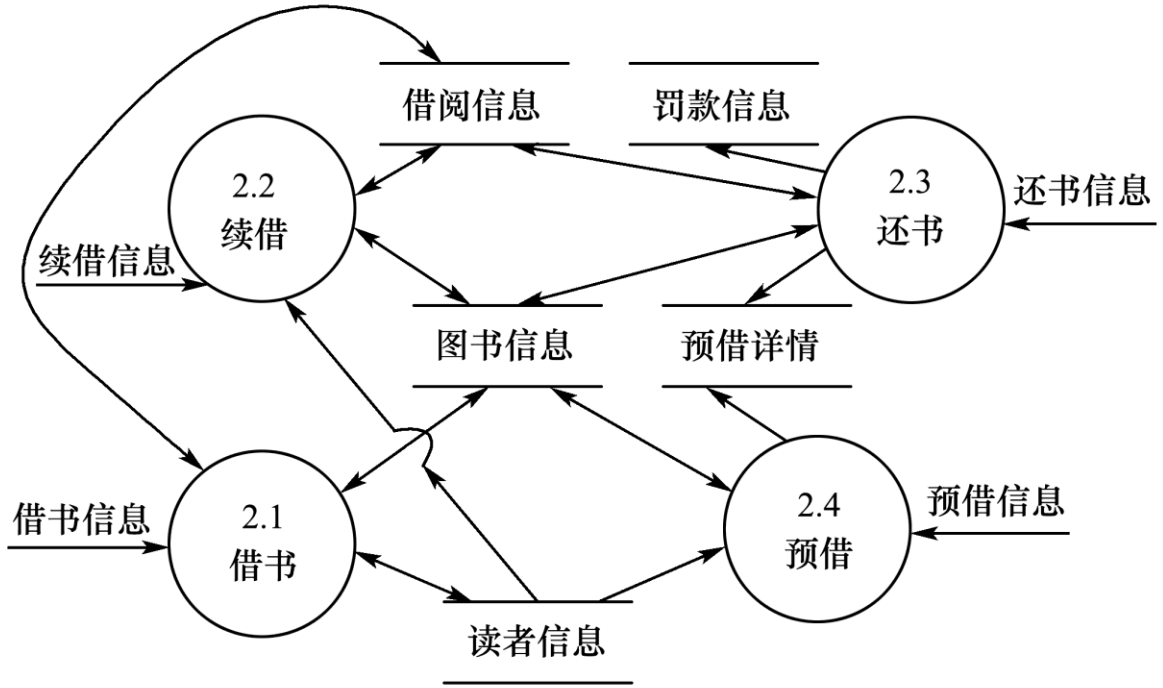
(a)顶层数据流图

需求分析

- 数据流图



(b)0层数据流图



(c)借阅管理加工的一层数据流图

需求分析

- 数据流图
  - 注意事项
  - 1）数据流图反映系统“做什么”，不反映“如何做”，因此箭头上的数据流名称只能是名词或名词短语，整个图中不反映加工的执行顺序。
  - 2）如果一张数据流图中的某个加工分解成另一张数据流图时，则上层图为父图，直接下层图为子图。子图应编号，子图上的所有加工也应编号，子图的编号就是父图中相应加工的编号
  - 3）子图的输入、输出数据流和父图相应加工的输入、输出数据流必须一致

需求分析

- 数据字典
- 一般说来，数据词典的每个条目中应包括以下信息。
- （1）名字：数据流、数据项、数据存储或外部实体的名称。
- （2）别名或编号：第（1）项中对象的其他名字。
- （3）分类：数据流、数据项、加工、数据存储、外部实体等。
- （4）内容描述：描述内容或数据结构等。
- （5）何处使用：哪些加工使用该条目。

需求分析

- 数据字典

表 3-2　　数据词典中常用符号

符号	含义	举例及说明
=	定义为	
+	与，连接	x=a+b 表示 x 由 a 和 b 组成
[…|…]	选择	x=[a|b|c]表示 x 由 a 或 b 或 c 组成，如：性别=[男|女]，表示性别是“男”或“女”之一
{…}	重复	x={a}表示 x 由 0 个或多个 a 组成，如：姓名=1{汉字}4，表示姓名为 1 至 4 个汉字； 又如：1{字母}，表示至少 1 个字母；{字母}，表示 0 至任意个字母
(…)	可选	x=(a)表示 a 可在 x 中出现，也可不出现
..	连接符	x=1..9 表示 x 可取 1 到 9 任意一个值
…	注释	表示两个 * 之间的内容是对条目的注释

需求分析

- 数据字典

(1)“入库通知单”数据流的定义：

数据流名：入库通知单
数据流来源：“审查申请”加工
数据流去向：“图书入库”加工
组成：采购者＋采购者部门＋{图书详情＋数量}

(2)“入库通知单”数据流中的“图书详情”数据结构的定义：

名称：图书详情
组成：书名＋作者＋出版社＋单价＋购书日期

需求分析

- 数据字典

(3)“入库通知单”数据流中的“数量”数据项的定义：

数据项名：数量
别名：无
取值范围：正整数

(4)数据存储“图书库”的定义：

名称：图书库
组成：{图书编号＋图书详情＋目前库存量}
组织方式：按图书编号从小到大排列

需求分析

- 数据字典

(5)外部实体“教师”的定义：

名称：教师
简述：向教师图书室提供图书的教师
从外部实体输入：报销申请 向外部实体输出：入库证明

(6)“审查申请”加工的定义：

名称：审查申请
输入：报销申请、专业教师登记表、专项资金
输出：拒绝信息或入库通知单
加工逻辑：IF 报销人员是专业教师且专项资金未用完 THEN 可以报销 ELSE 拒绝报销

需求分析

- 结构化语言

表 3-3　　结构化语言

	书写格式	含义
选择结构	IF〈条件〉 THEN〈策略〉	如果〈条件〉 则〈策略〉
	IF〈条件〉 THEN〈策略 1〉 ELSE〈策略 2〉 按下列条件选择策略 CASE〈条件 1〉〈策略 1〉 CASE〈条件 2〉〈策略 2〉 … CASE〈条件 N〉〈策略 N〉	如果〈条件〉 则〈策略 1〉 否则〈策略 2〉 按下列条件选择策略 情况〈条件 1〉〈策略 1〉 情况〈条件 2〉〈策略 2〉 … 情况〈条件 N〉〈策略 N〉
循环结构	WHILE〈条件〉DO 〈策略〉 END DO	每当〈条件〉成立则 〈策略〉 返回
	REPEAT 〈策略〉 UNTIL〈条件〉	重复以下操作 〈策略〉 直到〈条件〉

需求分析

- 结构化语言

例如，在图书借阅的例子中，可以用下面的选择结构表示：

```
IF 读者类型 = “学生” THEN
    借阅上限 = 5
ELSE IF 读者类型 = “教师” THEN
    借阅上限 = 10
ELSE
    借阅上限 = 3
ENDIF
```

需求分析

- 结构化语言

在图书馆预约例子中，可以用下面的重复结构表示：

DO WHILE 还有未处理的预约请求

　发送预约到馆通知

　更新预约状态为“已通知”

ENDDO

需求分析

- 判定表

条件	条件取值的组合
动作	相应条件组合下应执行的动作

图 3-7　判定表的结构

需求分析

- 判定表

某数据流图中有一个“确定保险类别”的加工，指的是申请汽车驾驶保险时，要根据申请者的情况确定不同的保险类别。加工逻辑为：如果申请者的年龄在21岁以下，要额外收费；如果申请者是21岁以上并是26岁以下的女性，适用于A类保险；如果申请者是26岁以下的已婚男性，或者是26岁以上的男性，适用于B类保险；如果申请者是21岁以下的女性或是26岁以下的单身男性，适用于C类保险。除此之外的其他申请者都适用于A类保险。

需求分析

- 判定表

（1）提取问题中的条件。条件是年龄、性别及婚姻。

（2）标出条件的取值。如表3-3所示，用符号代替条件的取值。

条件名	取 值	符 号	取值数m
年龄	年龄≤21 21 < 年龄≤26 年龄 > 26	C Y L	$m_1 = 3$
性别	男 女	M F	$m_2 = 2$
婚姻	未婚 已婚	S E	$m_3 = 2$

需求分析

- 判定表

（3）计算所有条件的组合数N：

$$N = \prod_{i=1}^{3} m_i = 3 \times 2 \times 2 = 12$$

（4）提取可能采取的动作或措施。这些措施要适用于A类保险、B类保险、C类保险和额外收费。

需求分析

- 判定表

（5）制作判定表

判 定 表

	1	2	3	4	5	6	7	8	9	10	11	12
年龄	C	C	C	C	Y	Y	Y	Y	L	L	L	L
性别	F	F	M	M	F	F	M	M	F	F	M	M
婚姻	S	E	S	E	S	E	S	E	S	E	S	E
A类保险					√	√			√	√		
B类保险				√				√			√	√
C类保险	√	√	√				√					
额外收费	√	√	√	√								

需求分析

- 判定树

要素	说明
根节点	决策的起点，第一个需要判断的条件
内部节点	中间的条件判断点
分支	从节点出发的边，代表条件的取值
叶子结点	决策的重点，表示执行的操作或结论

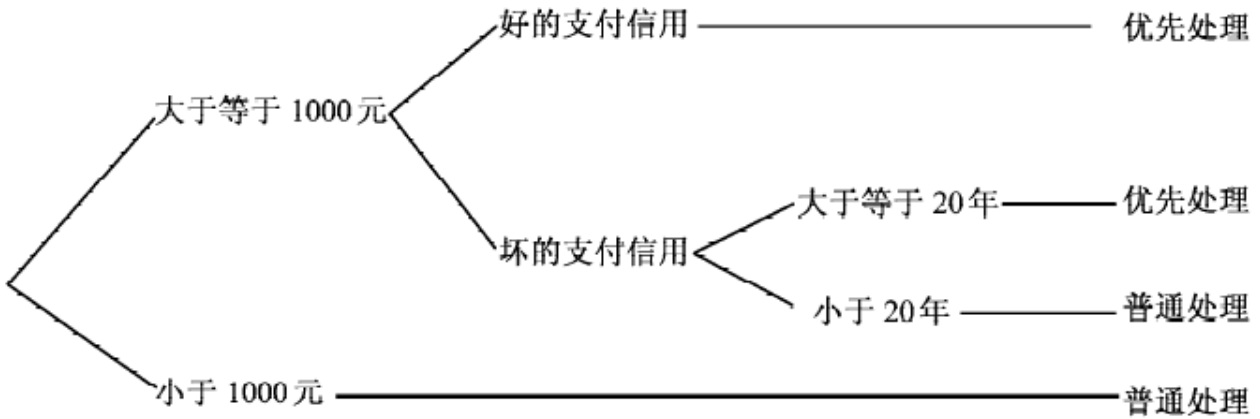
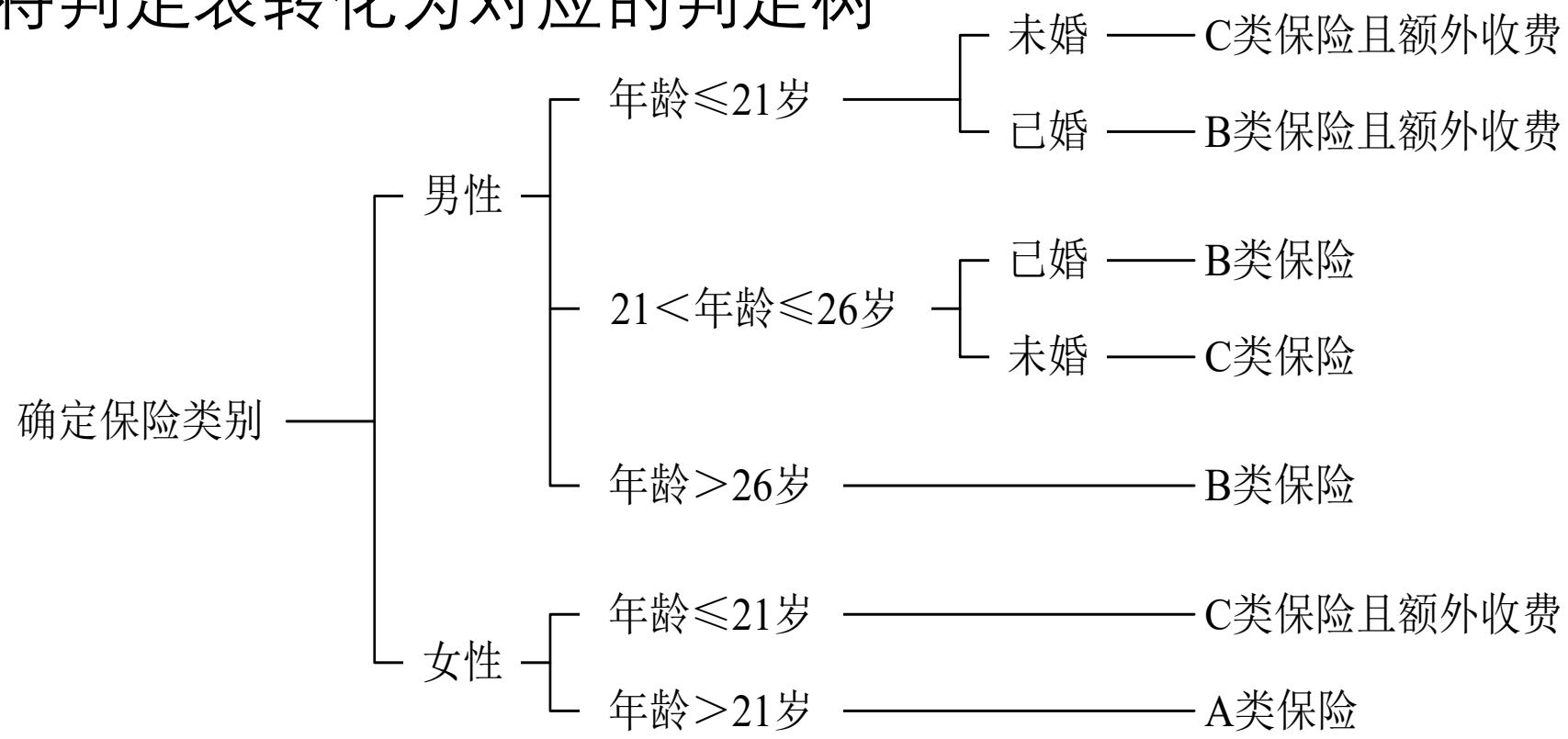


图 3-8　图书优惠政策判定树

需求分析

- 判定树
- 可以将判定表转化为对应的判定树



需求分析

- 面向对象需求建模方法:
  - 用例
  - 用例图、用例描述
  - 活动图
  - 系统顺序图
  - 实体-联系图
  - 域模型类图

需求分析

- **用例：**
- 用例（Use Case）是面向对象需求分析中的核心概念，用于描述系统在外部参与者（Actor）的交互下所执行的完整功能单元。
- 确定用例的方式:
- 1）FURPS+：功能性需求+非功能性需求
- 2）事件分解：外部事件、时间事件、状态事件
- 3）CRUD方法：增删改查 Creat Report Update Delete

需求分析

- **需求识别的方式：**
- 1）FURPS+：功能性需求+非功能性需求
- 2）事件分解：外部事件、时间事件、状态事件
- 3）CRUD方法：增删改查 Creat Report Update Delete

需求识别方法：FURPS+

- Functional requirements（功能需求）
- Usability requirements（可用性需求）
- Reliability requirements（可靠性需求）
- Performance requirements（性能需求）
- Security requirements（安全需求）
- + even more categories…

Requirement categories	FURPS + categories	Example requirements
Functional	Functions	Business rules and processes
Nonfunctional 可用性 可靠性 性能 安全 设计限制 开发 接口 物理层 支持	Usability Reliability Performance Security + Design constraints Implementation Interface Physical Support	User interface, ease of use Failure rate, recovery methods Response time, throughput Access controls, encryption Hardware and support software Development tools, protocols Data interchange formats Size, weight, power consumption Installation and updates

需求分析

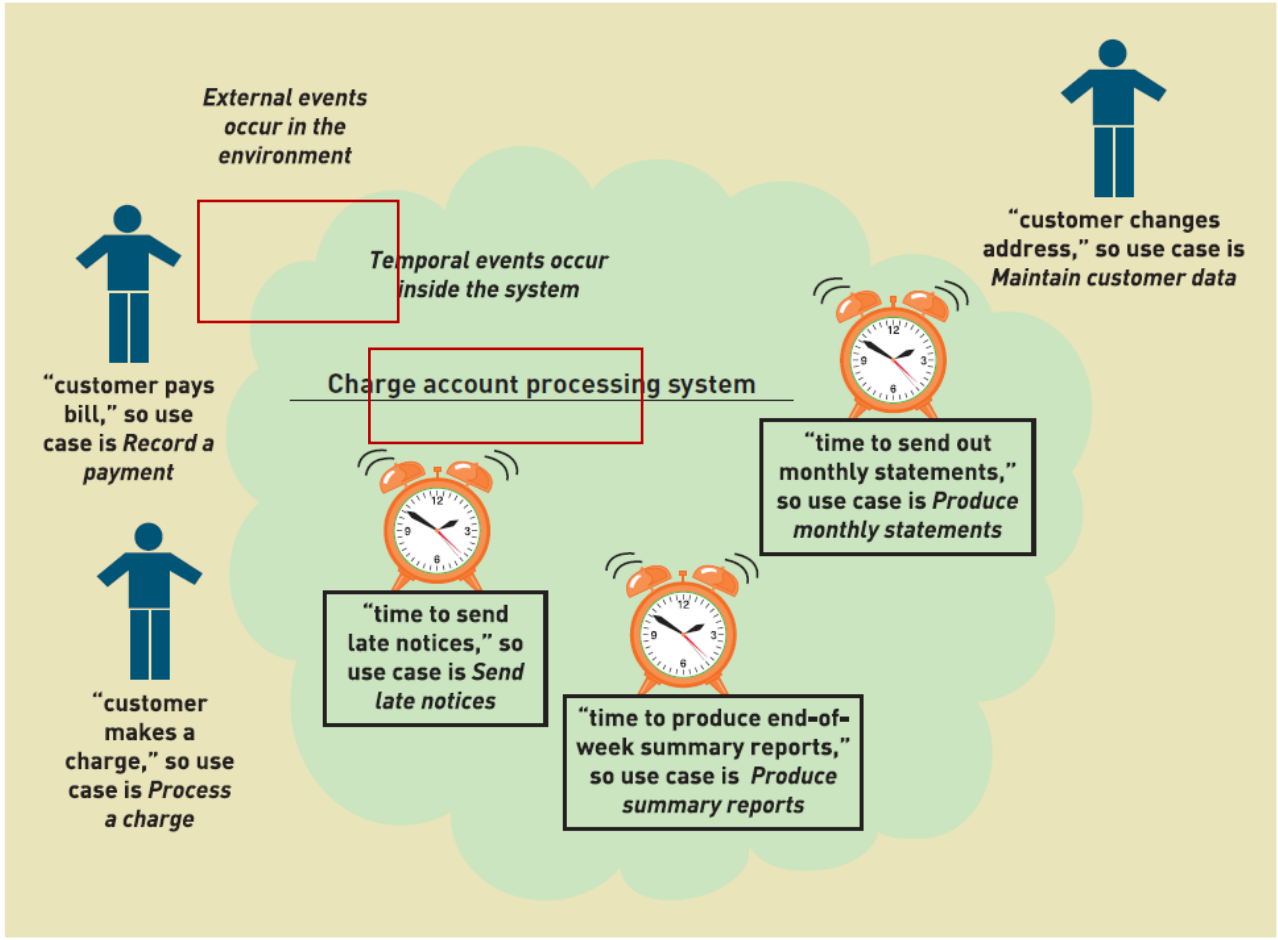
- 2）事件分解技术

- 外部事件:外部事件发生在系统外部的事件，通常由外部代理或参与者发起

- 时间事件:时间事件由于到达某个时间点而发生的事件

- 状态事件:状态事件(内部事件):当系统内部发生某些事情时发生的事件（库存物品到达重新订购点）

需求分析


External events occur in the environment
Temporal events occur inside the system
Charge account processing system
"customer pays bill," so use case is Record a payment
"customer makes a charge," so use case is Process a charge
"customer changes address," so use case is Maintain customer data
"time to send out monthly statements," so use case is Produce monthly statements
"time to send late notices," so use case is Send late notices
"time to produce end-of-week summary reports," so use case is Produce summary reports

需求分析

- 3）CRUD： Create（增）, Read/Report（查）, Update（改）, and Delete (删)
- 用于验证、改进或交叉检查用例的技术

Data entity/domain class	CRUD	Verified use case
Customer	Create	Create customer account
	Read/report	Look up customer Produce customer usage report
	Update	Process account adjustment Update customer account
	Delete	Update customer account (to archive)

需求分析

- **用例：**
- 确定用例的规范:
- 1）命名：用例命名需遵循动宾结构，以 “动词 + 名词” 形式准确描述功能行为，比如 “提交订单”“查询图书”，避免使用 “登录”“管理”
- 2）用例粒度要合适

需求分析-其他知识点

- 1）命名方式：动词+名词（do sth）

User	User goal and resulting use case
Potential customer	Search for item Fill shopping cart View product rating and comments
Marketing manager	Add/update product information Add/update promotion Produce sales history report
Shipping personnel	Ship items Track shipment Create item return

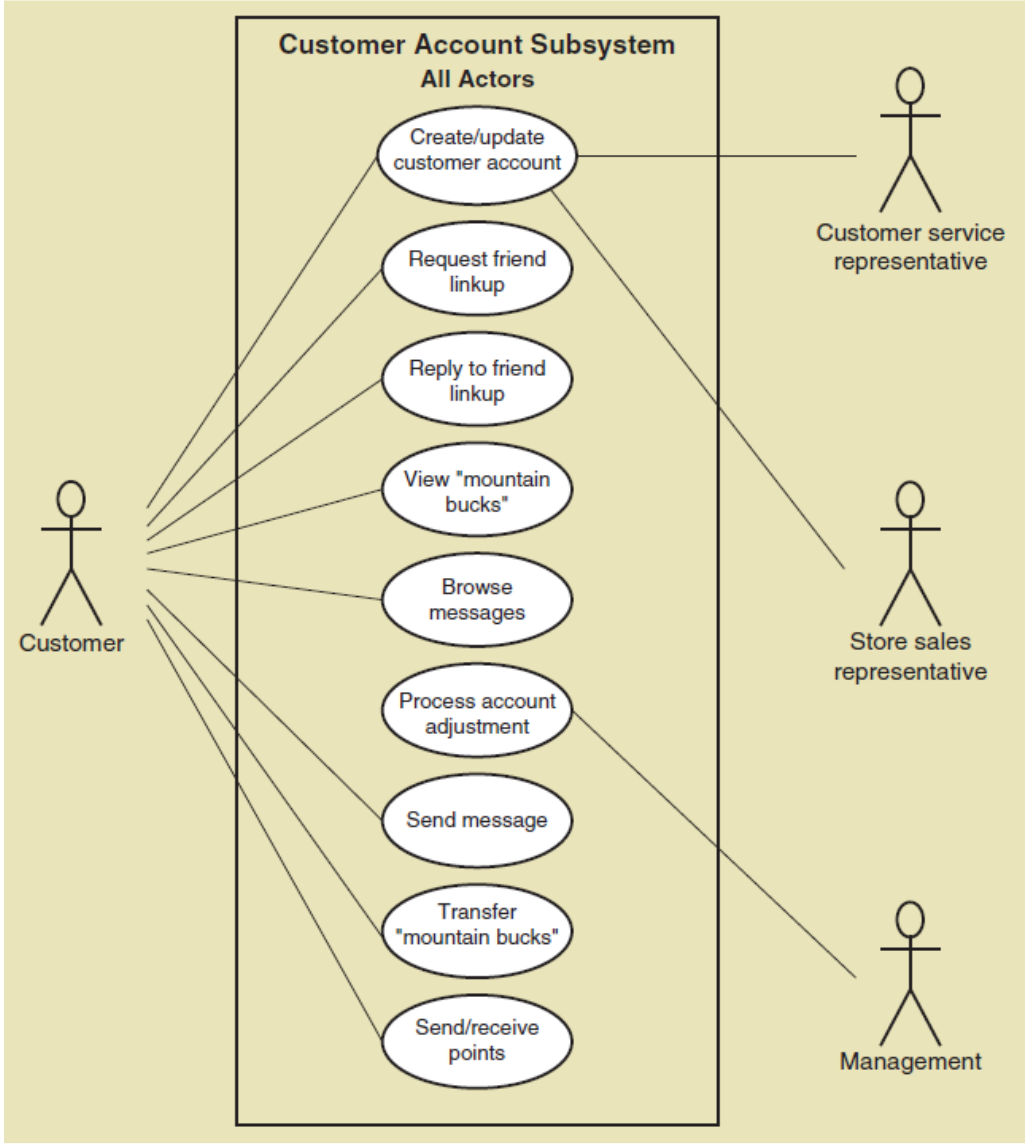
需求分析-其他知识点

- 2）用例需要有合适的粒度:
  - A: 在表格中输入一个顾客姓名
  - B: 增加新顾客信息
  - C: 增加新顾客、删除顾客、跟踪预期付款的顾客、联系以前的顾客

- B最为合适

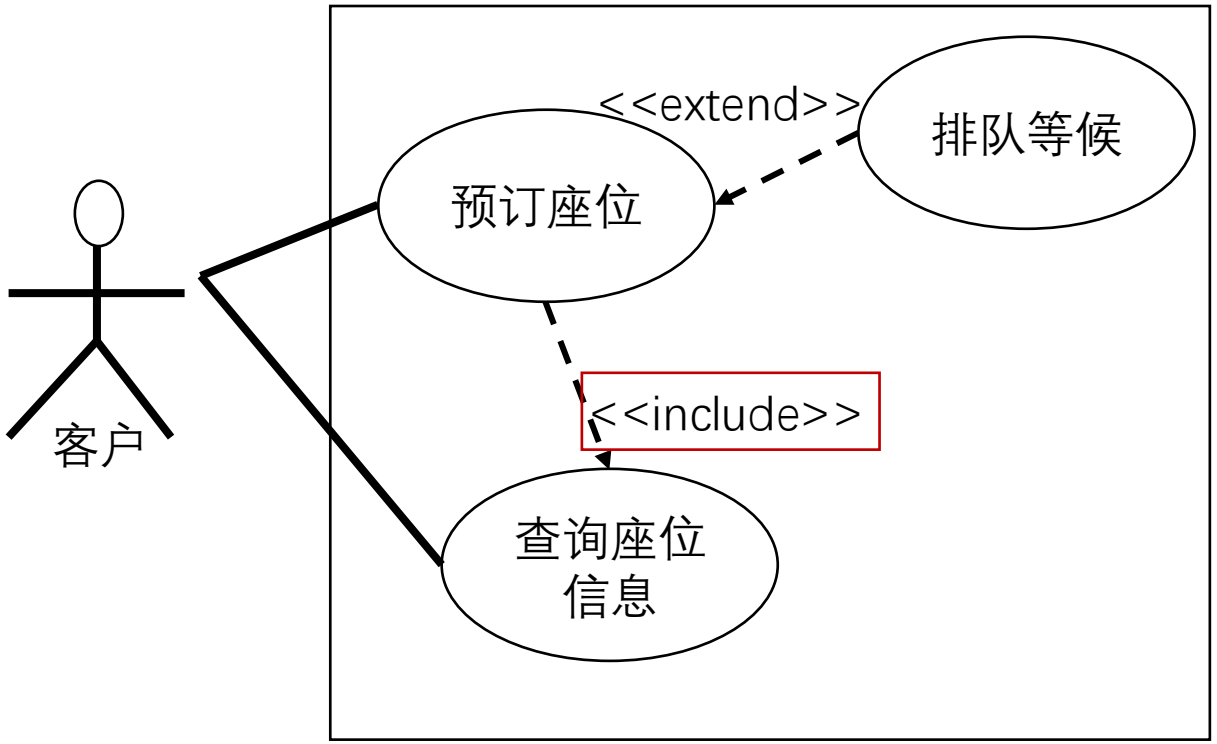
需求分析

- 用例图（Use Case Diagram）
  - 方框：系统边界
  - 椭圆框：用例
  - 小人：用户



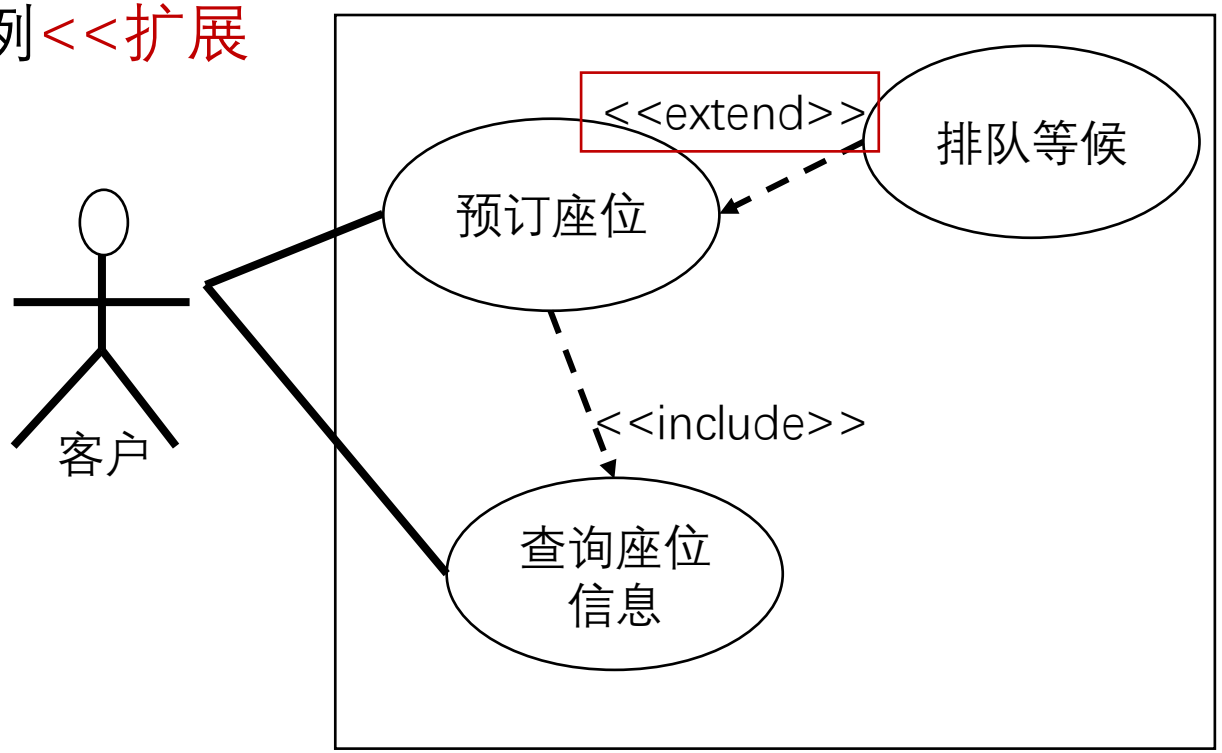
Use Case Diagrams
The <<Extend>> relationship

- 当执行A用例时，必须执行B用例才完整
- 这种情况就是A用例<<包含>>了A用例

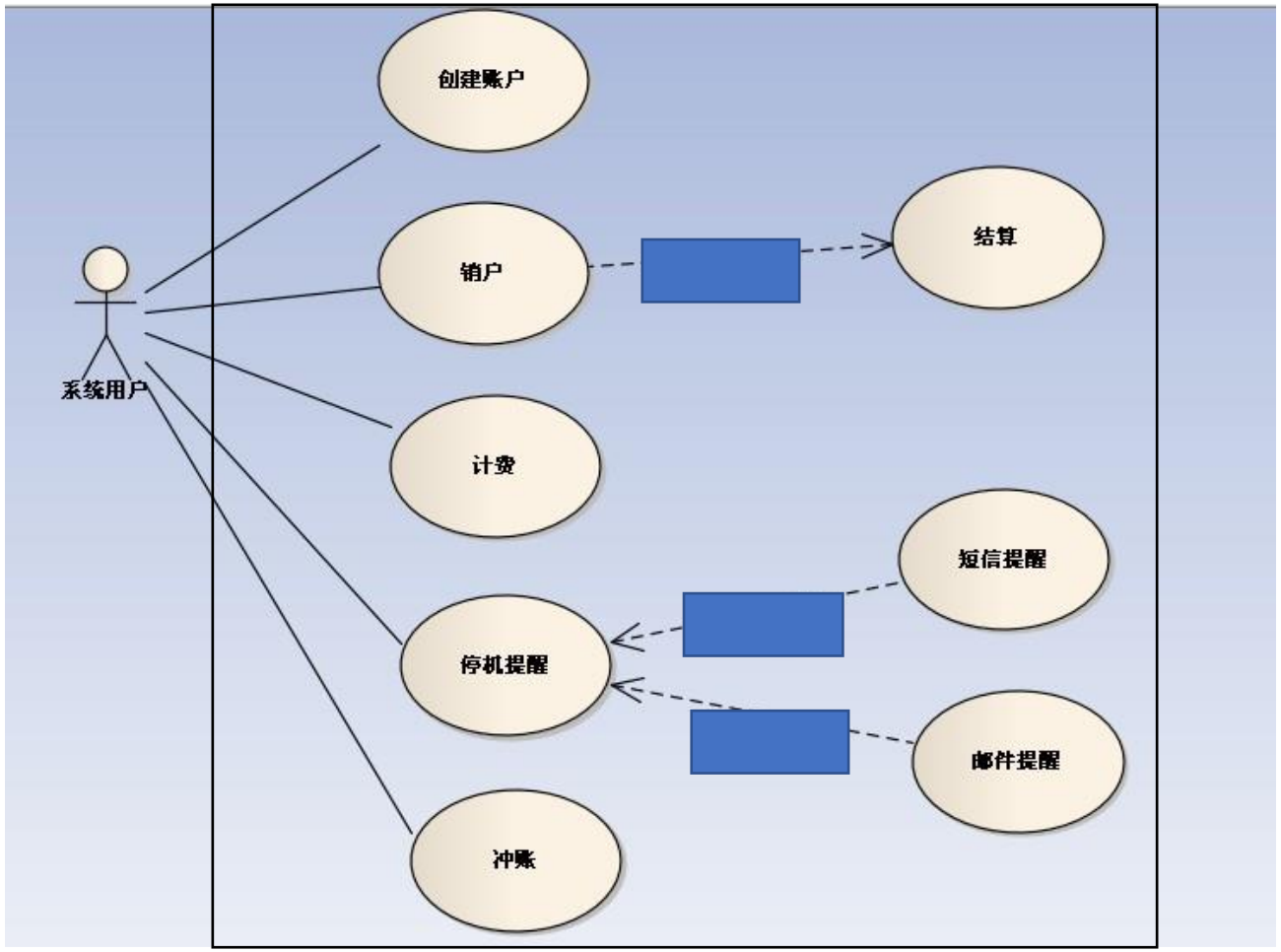


Use Case Diagrams
The <<Extend>> relationship

- 当执行A用例时，有些情况下，会执行B用例，有些情况下不会执行B用例
- 这种情况就是B用例<<扩展>>了A用例



An Use Case Diagram Example



需求分析

- 用例描述
- 用例图提供了系统功能的高层视图，但无法表达功能的详细内容。因此，每个用例需要配以详细的用例描述。用例描述通常采用结构化文本形式

要素	说明
用例名称	用例的唯一标识，通常采用“动词+名词”的格式
参与者	执行该用例的参与者
前置条件	用例执行前必须满足的条件
基本流程	用例正常执行时的主路径，描述参与者与系统之间的交互步骤
备选流程	异常情况或分支路径的处理
后置条件	用例执行后系统的状态
业务规则	用例涉及的业务约束和规则

需求分析

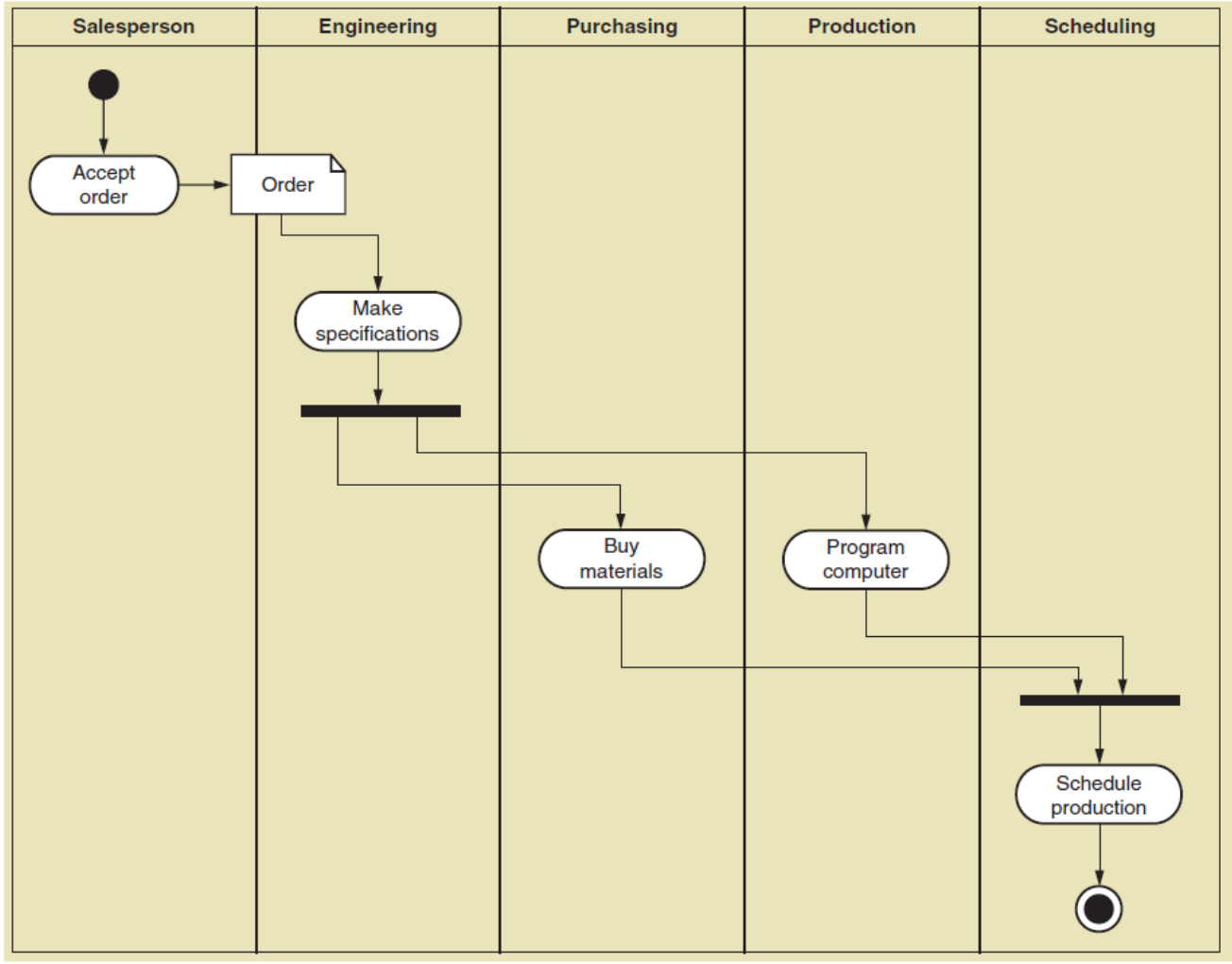
- 用例描述

表 3-8 用例描述说明

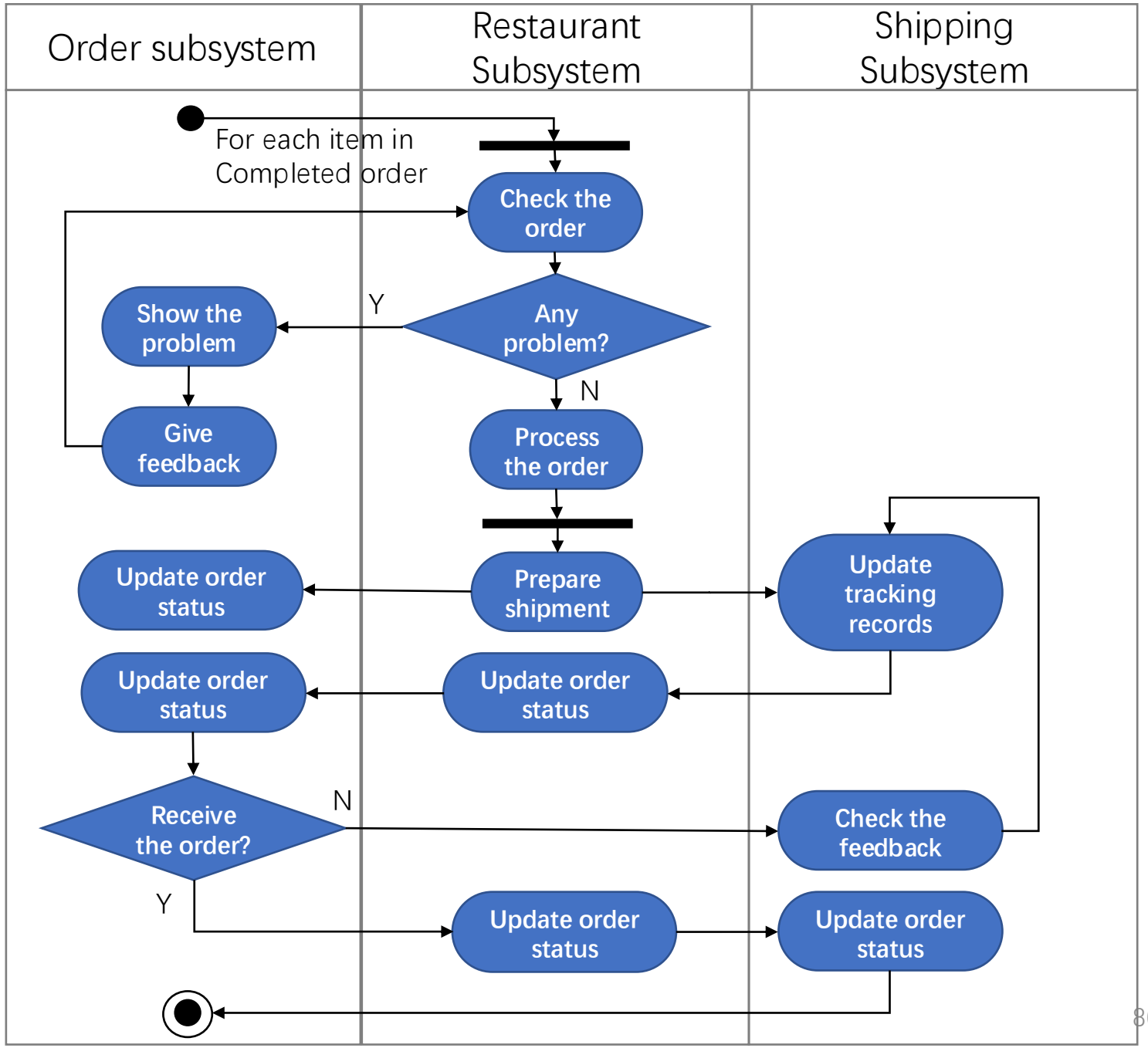
要素	说明
用例名称	借阅图书
参与者	读者
前置条件	读者已登录系统 读者借阅数量未达上限 读者无逾期未还图书
基本流程	1.读者提交借阅申请，包含图书 ISBN 和借阅证号 2.系统验证读者资格（借阅数量、逾期情况） 3.系统验证图书状态（是否在库） 4.系统创建借阅记录，记录借出日期和应还日期 5.系统更新图书状态为“已借出” 6.系统生成借阅凭证并返回给读者
备选流程	2a.读者借阅数量已达上限：系统提示“已达借阅上限”，用例结束 2b.读者存在逾期未还图书：系统提示“请先归还逾期图书”，用例结束 3a.图书状态为“已借出”：系统提示“图书已借出，是否预约”，用例结束 3b.图书状态为“损坏”或“遗失”：系统提示“图书不可借”，用例结束
后置条件	系统成功记录借阅信息 图书状态更新为“已借出” 读者已借数量增加 1

需求分析

- 活动图（Activity Diagram）
  - 泳道：不同对象
  - 黑色圆点：开始
  - 黑色圈+圆点：结束
  - 圆角矩形：执行语句
  - 黑色加粗横线：并行处理

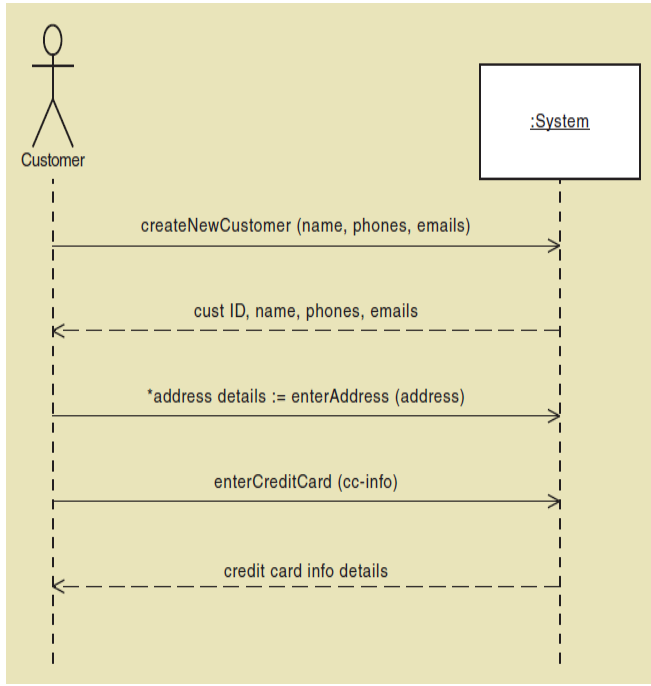


- 活动图:
- 用活动图描述一个外卖配送案例，包括订单子系统、餐厅子系统和配送子系统

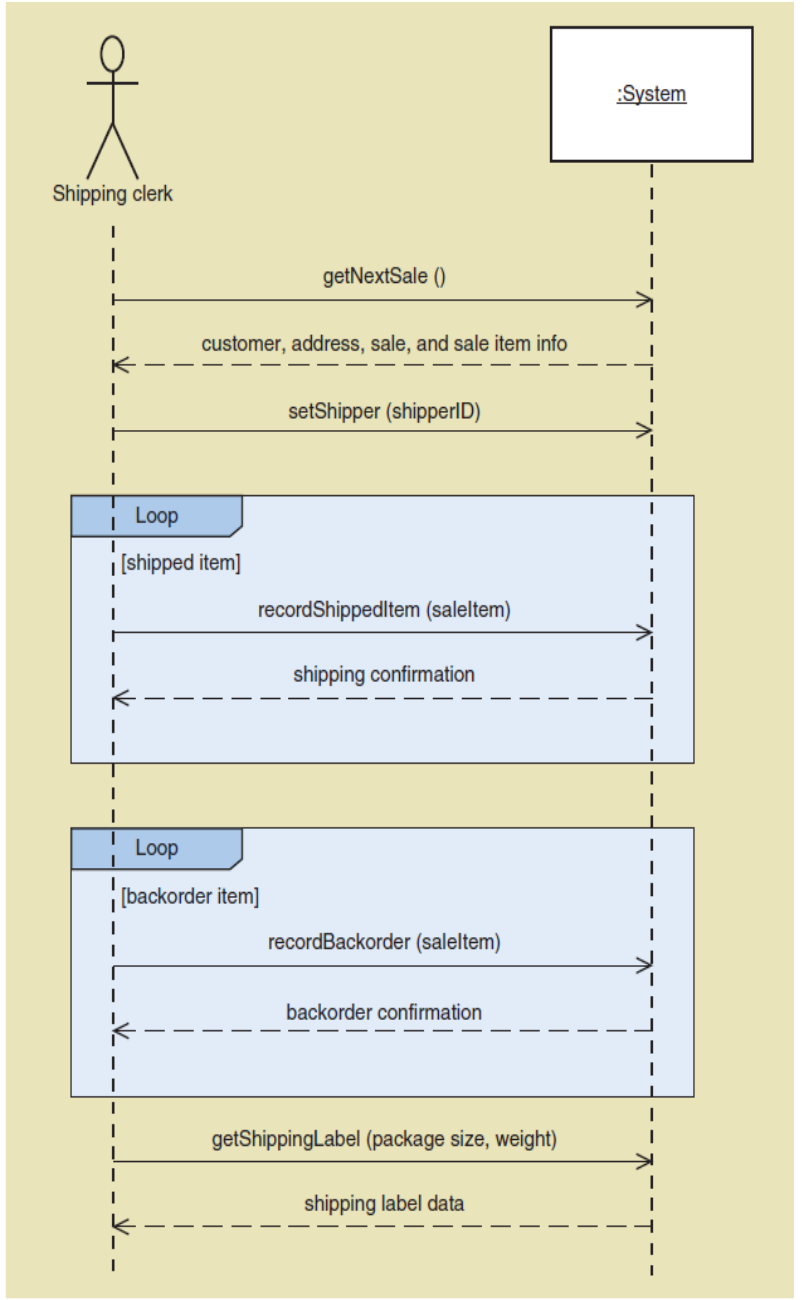


需求分析

- 系统顺序图（System Sequence Diagram）
  - 只有用户和:System（与设计部分的顺序图区分）
  - 双向箭头版：
    - 一个方向为Method（参数）
      一个方向为返回值
  - 单箭头版：



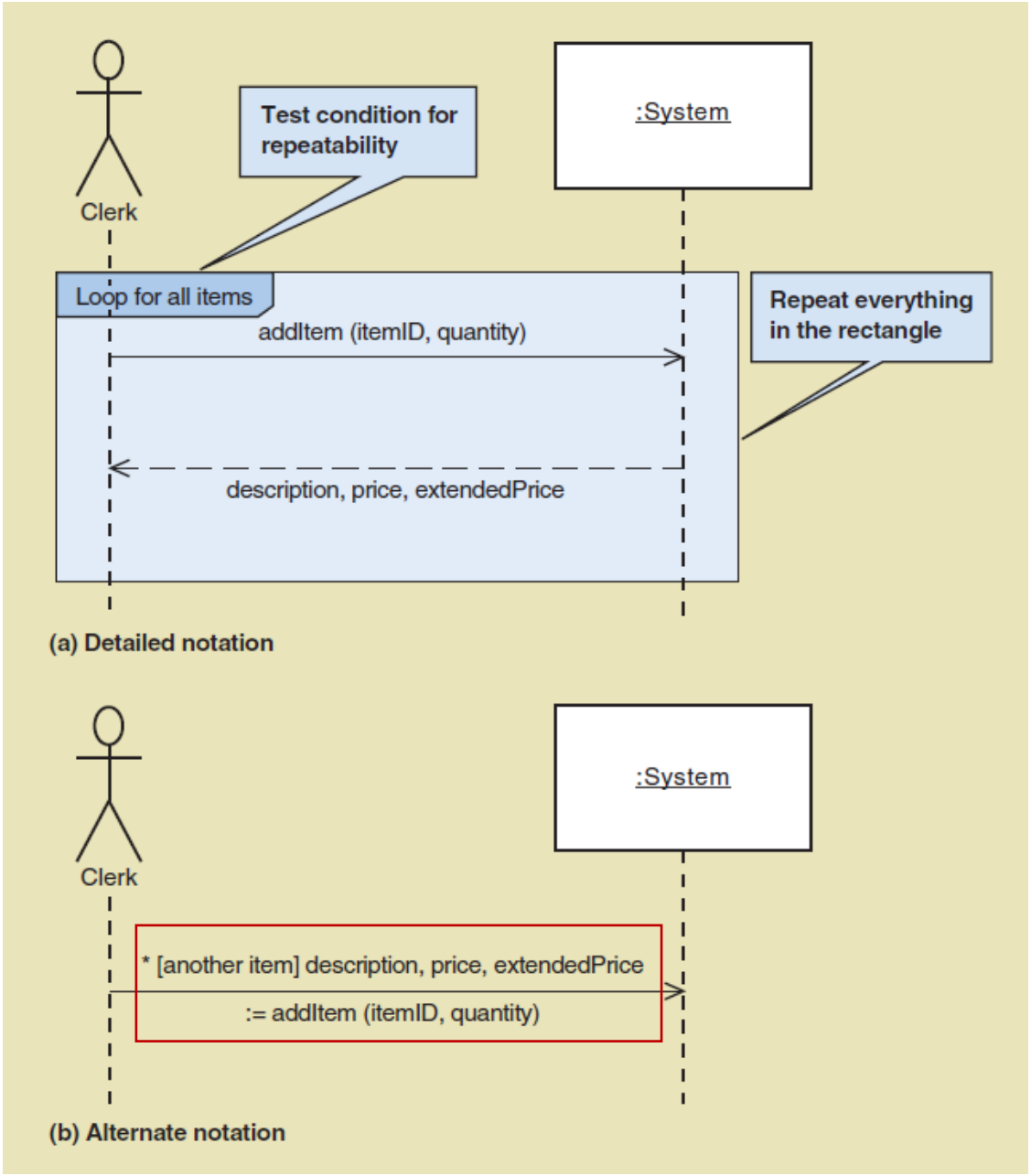
语句版



双向箭头版

* *[true/false condition] return-value := message-name (parameter-list)*

SSD Message Examples with Loop Frame （带环路的系统顺序图）

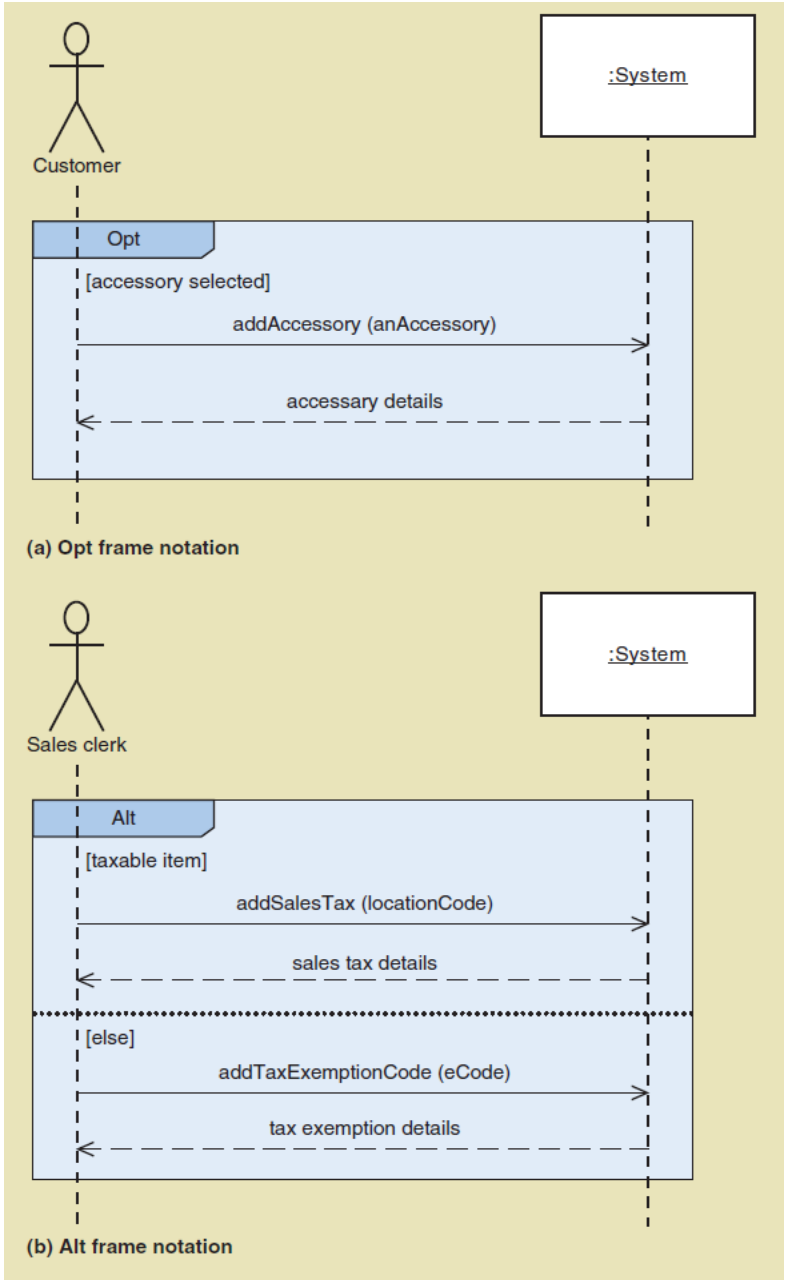


© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

SSD Message Examples

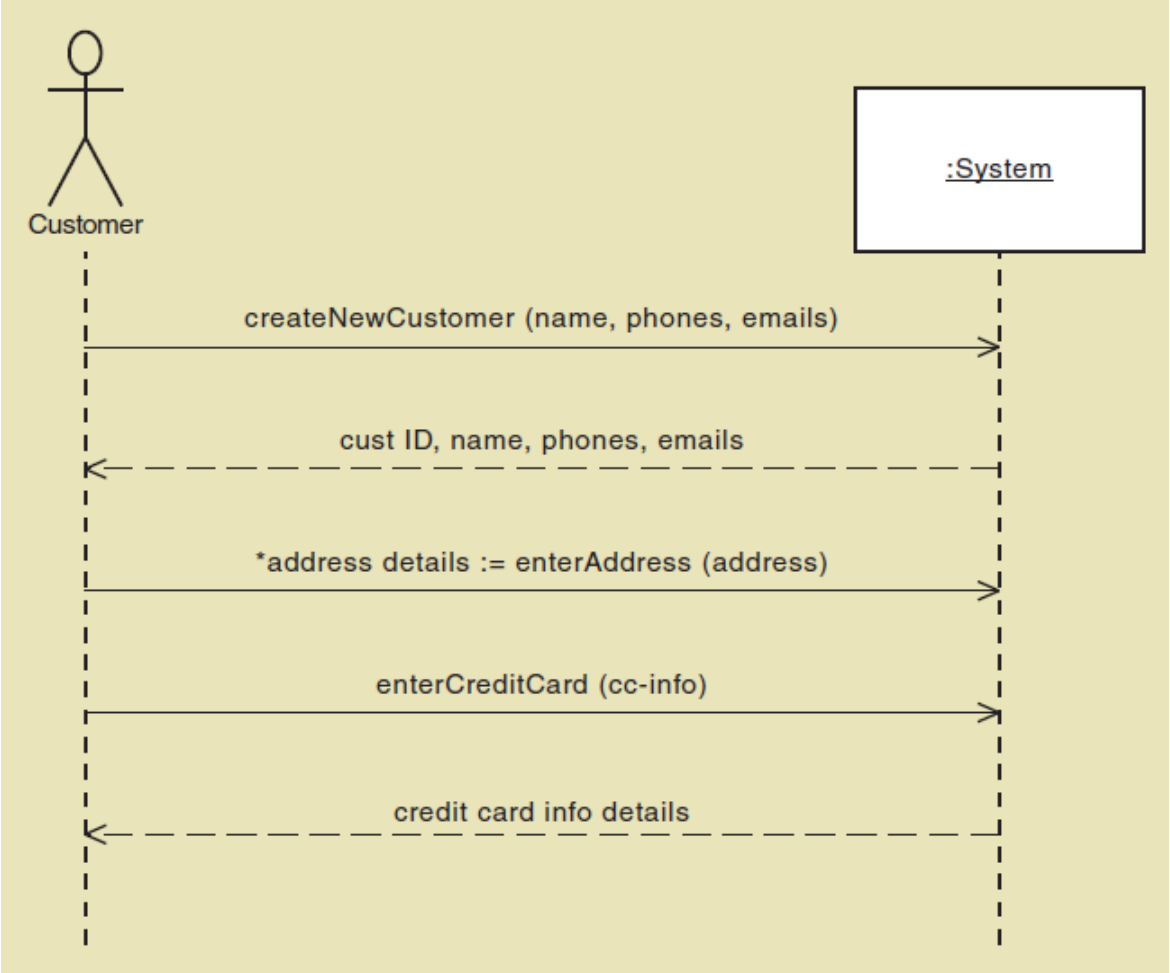
Opt Frame (optional)
可选框架

Alt Frame (if-else)
判断框架



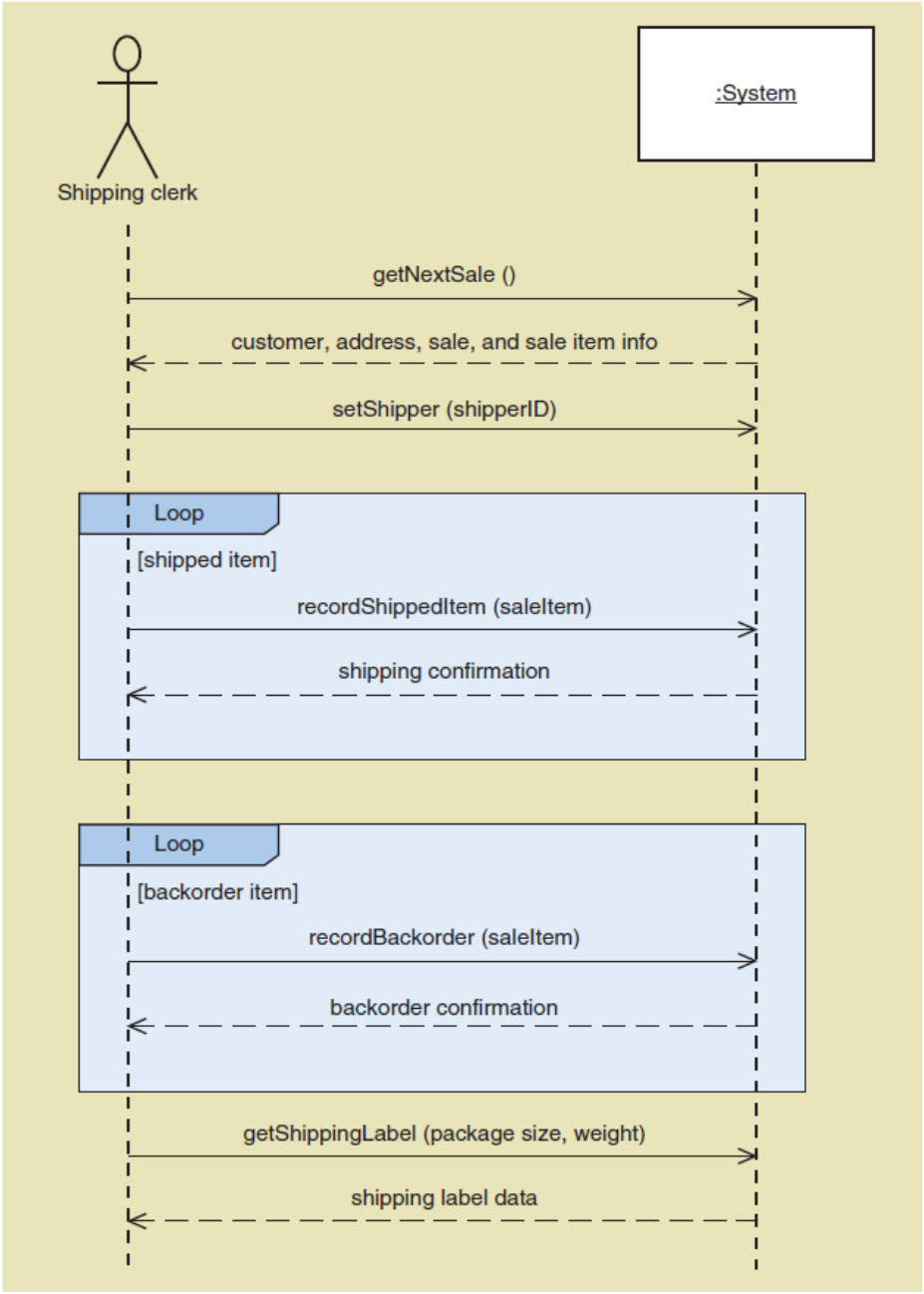
© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

SSD for *Create customer account* Use case（创建客户账户用例的顺序图）



© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

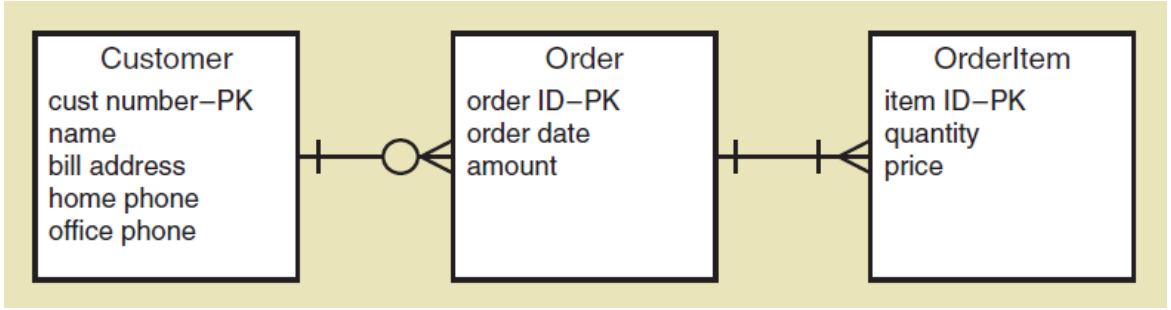
SSD for *Ship items* Use Case（创建运输商品用例的顺序图）

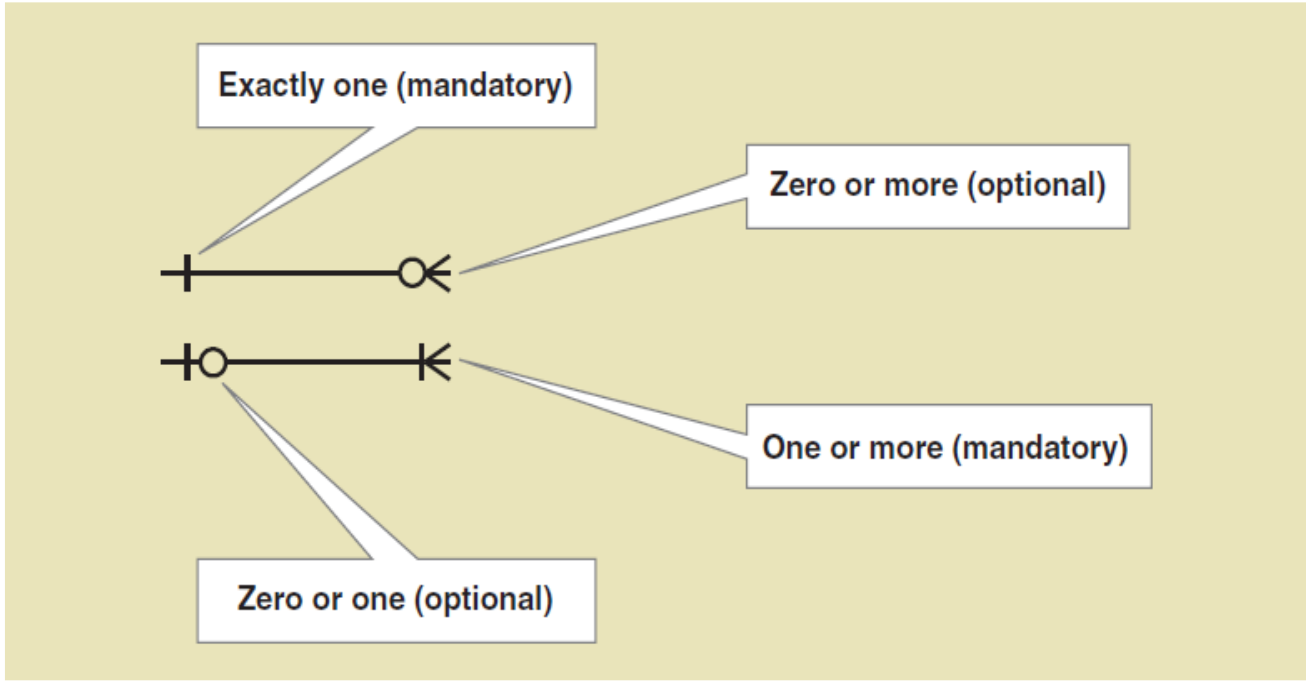


© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

需求分析

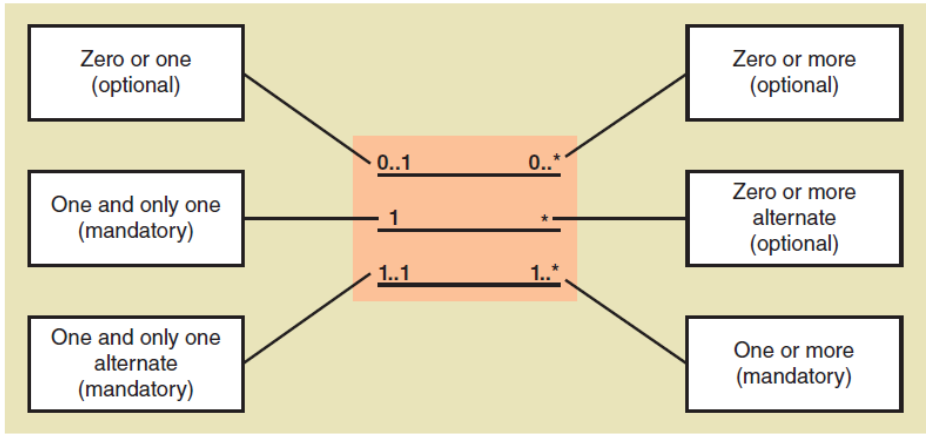
- 实体关系图（Enity-Relationship Diagram）
  - 长方形：类
    - 第一行：类名
    - 第二行：属性
  - 类间符号：类之间的数量关系
    - 鱼尾纹符号，见右图

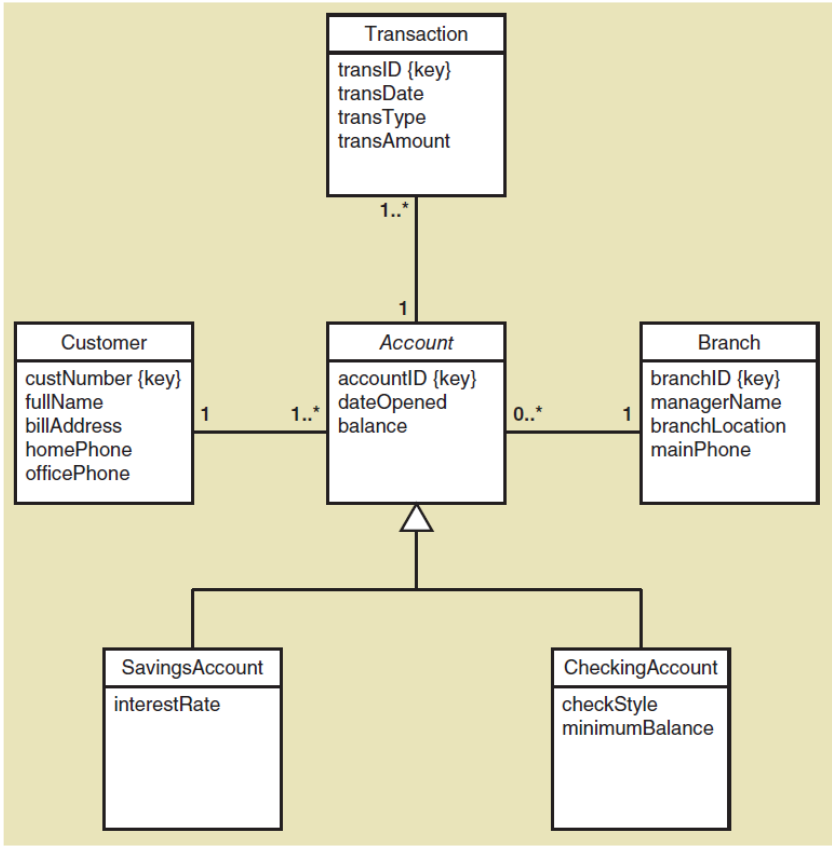




需求分析

- 域模型类图（Domain Model Class Diagram）
  - 长方形：类
    - 第一行：类名
    - 第二行：属性
    - 没有第三行
  - 类间数字：类之间的数量关系
    - 例子：1个Customer有1个或多个Account（1..*）

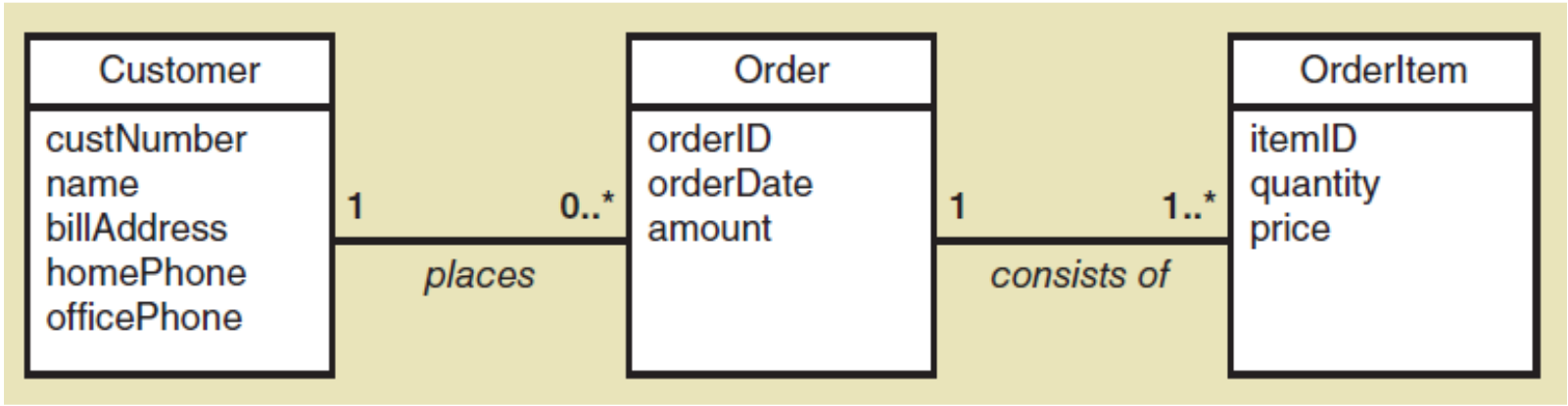




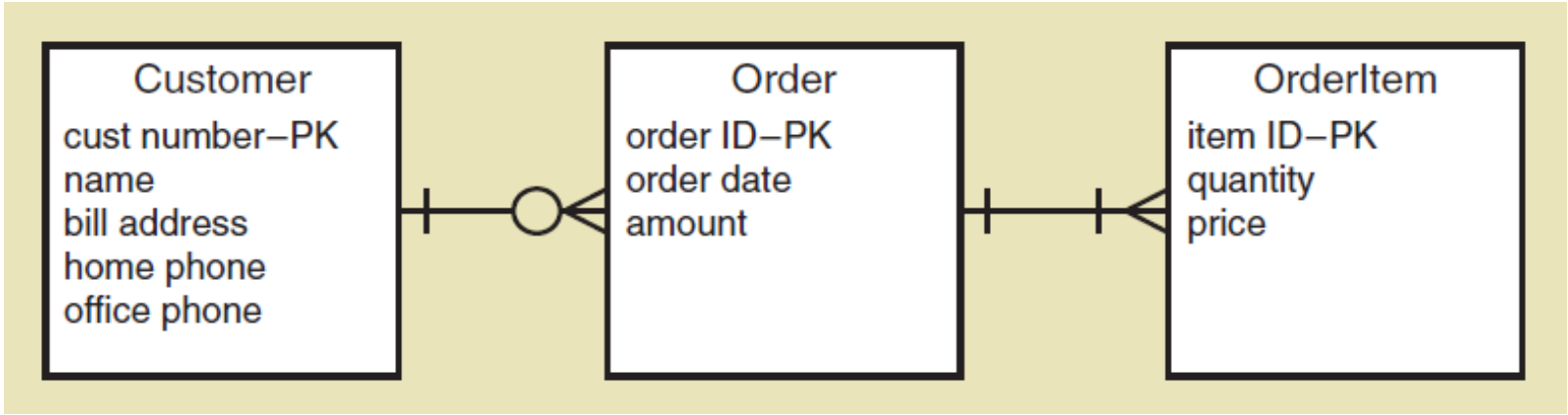
需求分析

- 实体关系图转换成域模型类图

上：域模型类图;



下：实体关系图

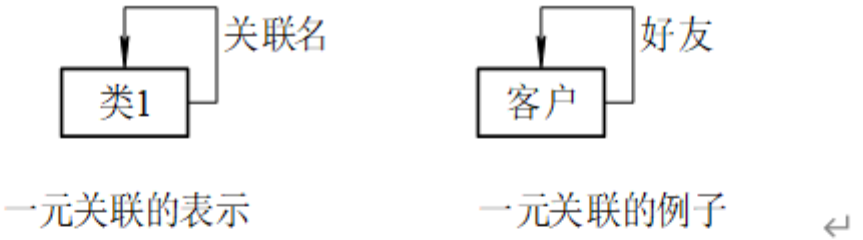


需求分析

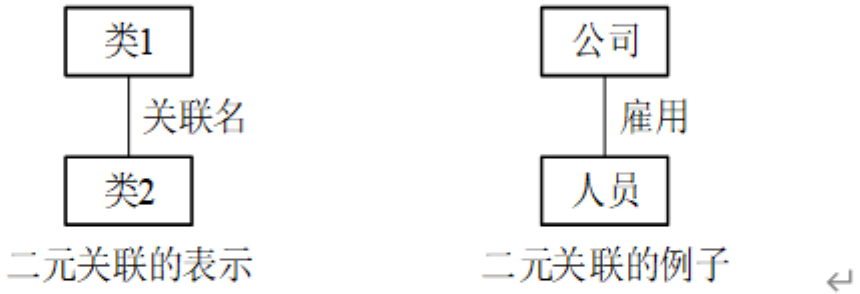
- 域模型类图
- 关联关系：
- 1）数量关系：
- （1）一元关系/回归关系：同一个类实例之间的关系，用于表达层级结构、递归关系，如员工上下级、客户及其关联好友
- （2）二元关系：两个不同类之间的关系，是最常见的关系类型，包括一对一、一对多、多对多三种基本形式，例如人员在公司工作
- （3）三元关系：三个不同类之间的关系，用于表达三因素共同决定的业务约束，如供应商-产品-客户定价

需求分析

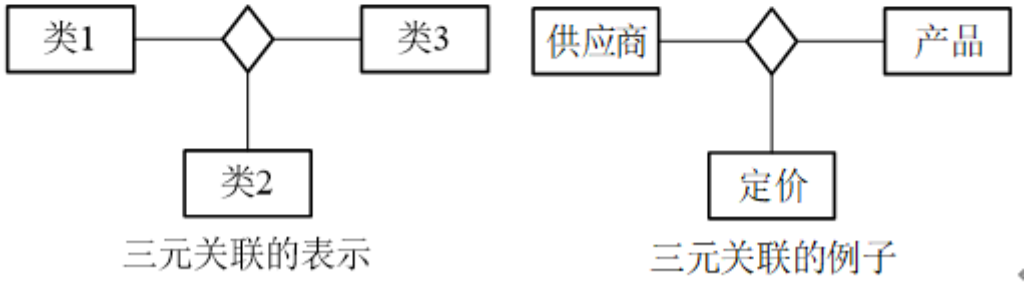
- 域模型类图
- 关联关系：
- 1）数量关系：



（a）一元关联



（b）二元关联

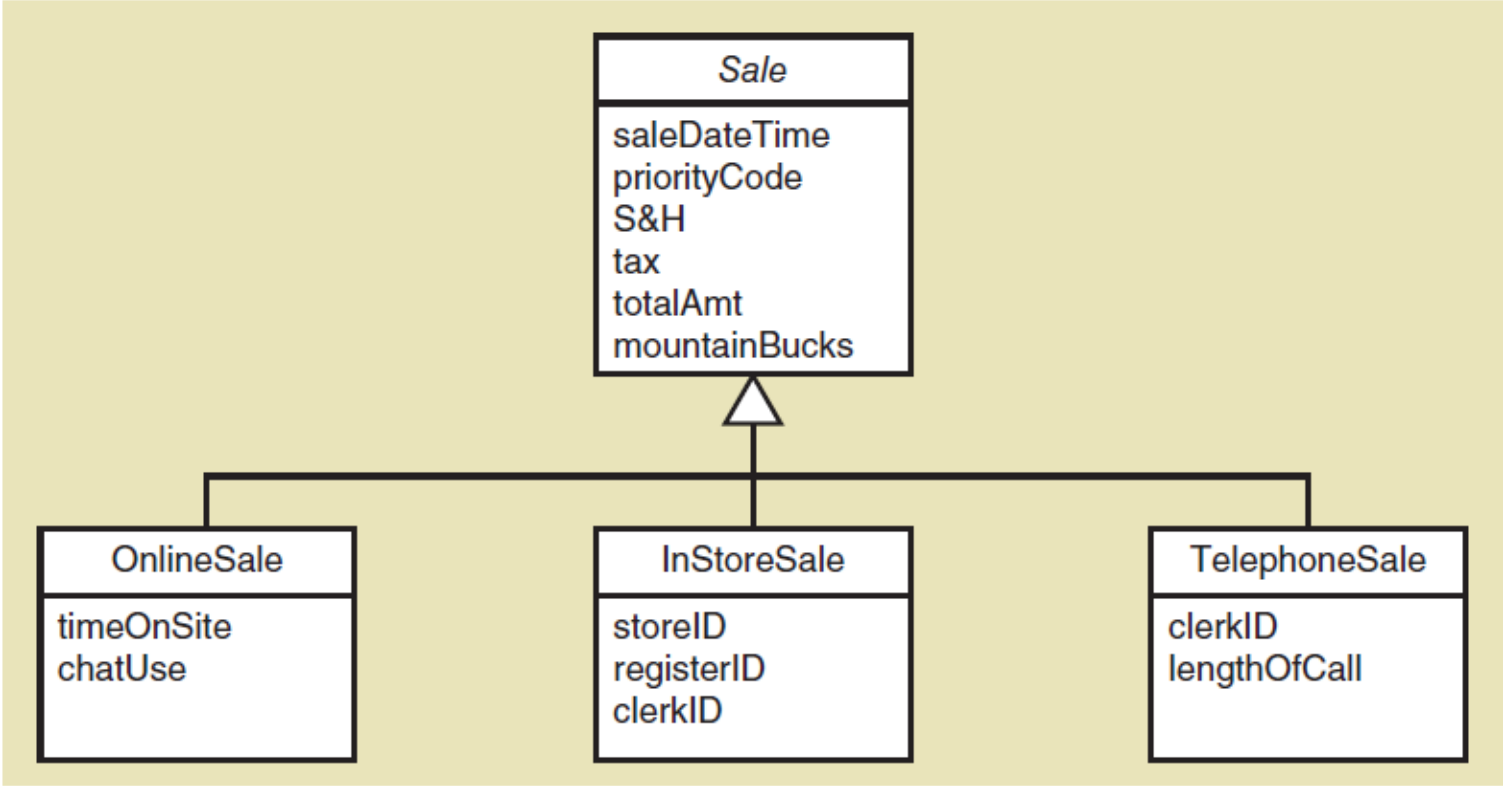


（c）三元关联

需求分析

- 域模型类图
- 关联关系：
- 2）泛化-特化关系：
- 父类：分为抽象类和具体类，如果是抽象类，应该用斜体写类名，例如*Sale*
- 子类
- 子类指向父类

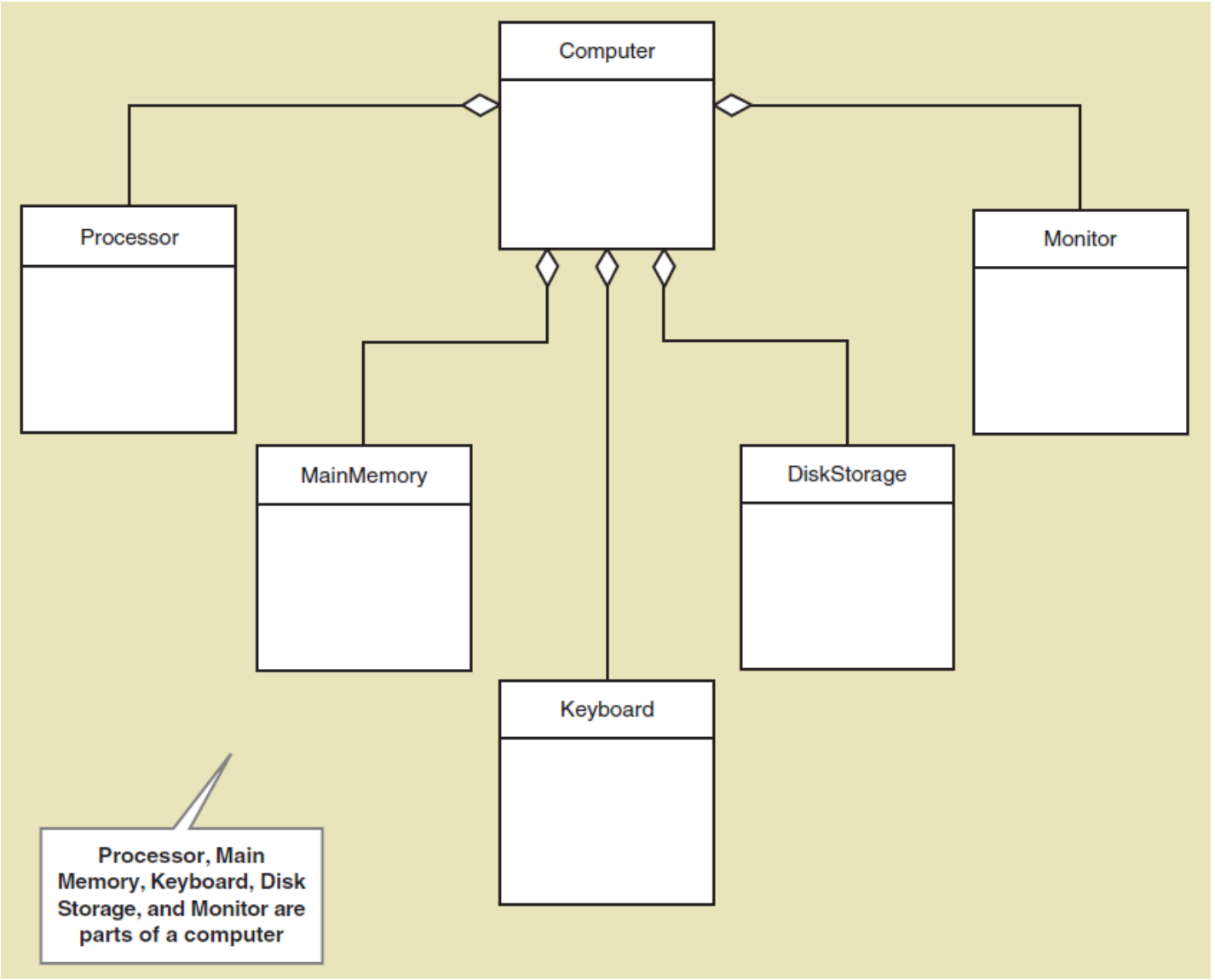
用三角形表示继承



需求分析

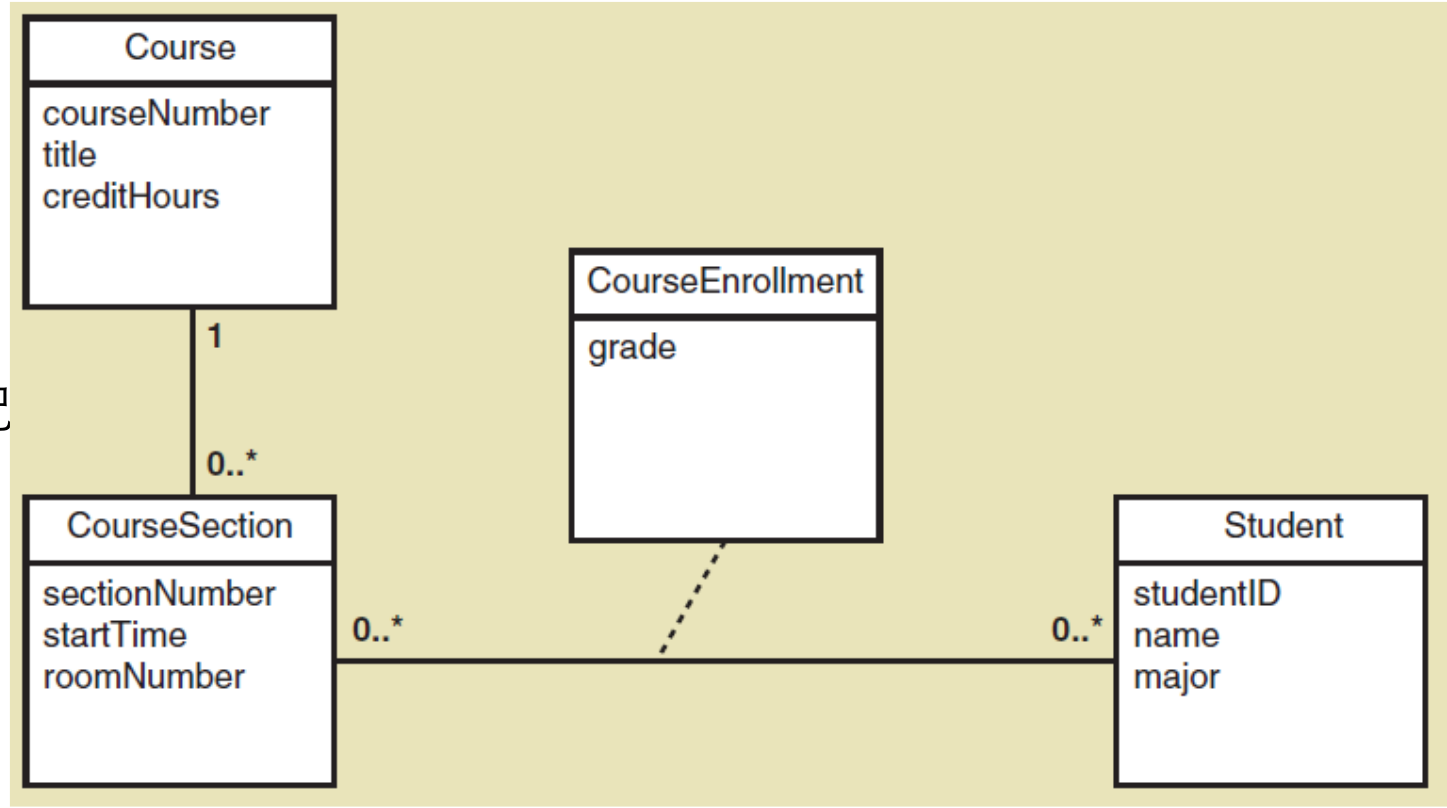
- 域模型类图
- 关联关系：
- 3）整体-部分关系：其中一个类是另一个类的一部分或组成部分

用菱形表示整体-部分关系

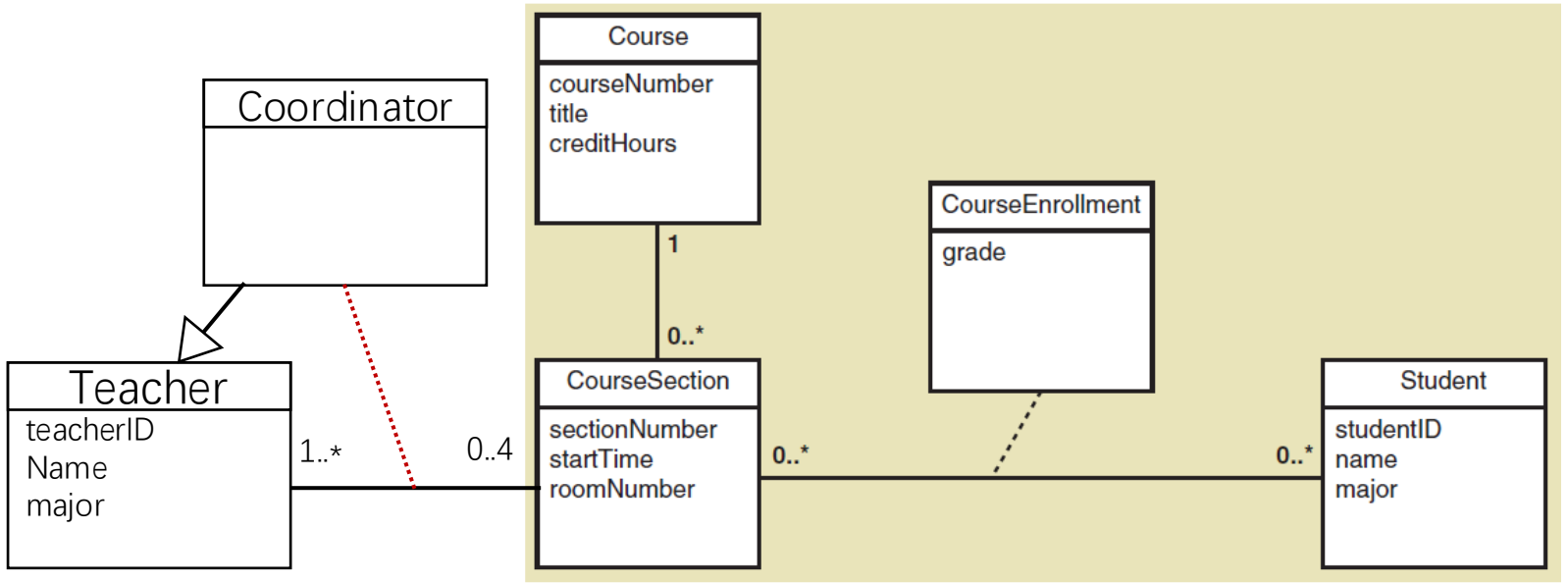


需求分析

- 域模型类图
- 关联关系：
- 4）关联类——在多对多关联中被视为类的关联，因为它具有需要记住的属性，比如成绩



例题

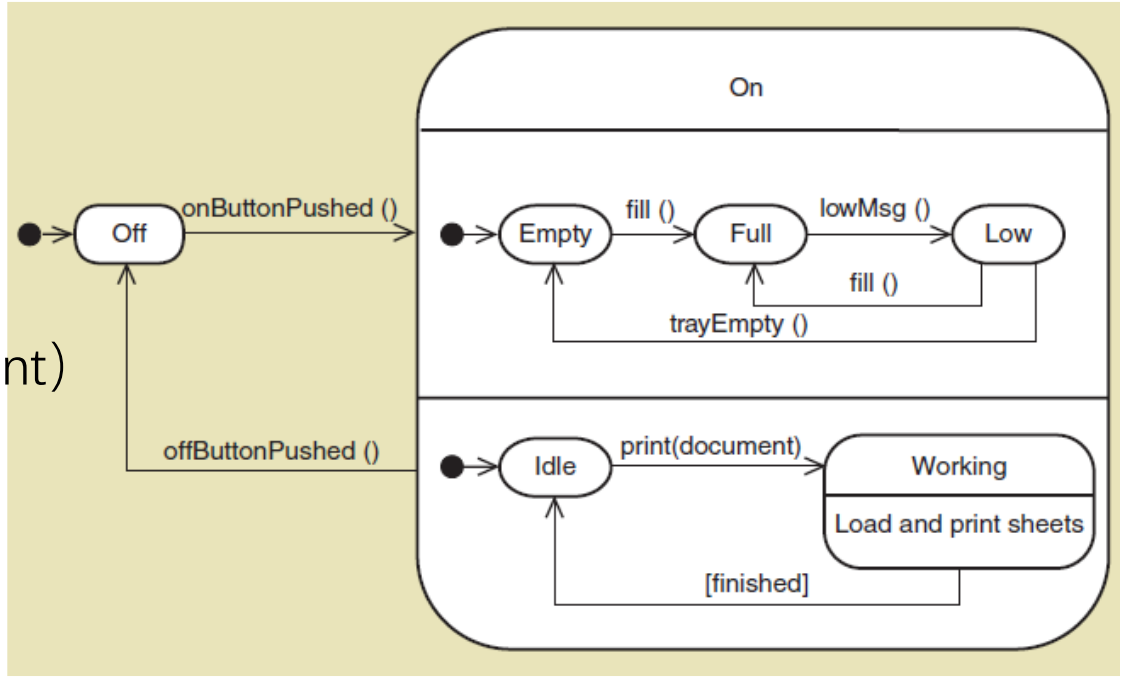


- 再次思考上图，添加以下内容：

一个教师经常教授很多课程部分，但为了保证授课质量，一个教师一学期最多教授4门课。每个课程部分必须至少有一个教师来教授。同时，每个课程需要安排一位教师作为课程协调者来监督这个课程。

需求分析

- 状态机图（State Machine Diagram）
  - 黑色原点：伪状态
  - 圆角矩形：状态
    - 多行：复合状态（打印机同时处于on和print）
  - 箭头：状态转移需要做的操作



【选择】软件开发过程中，抽取和整理用户需求并建立问题域精确模型的过程叫
A．生存期
B．面向对象设计
C．面向对象程序设计
D．面向对象分析

- 答案：D

【选择】以下哪项不是软件需求分析阶段的主要任务？
A. 识别利益相关者
B. 建立需求模型
C. 编写代码实现功能
D. 编写需求规格说明书

- 答案：C

【选择】以下哪种建模方法属于结构化分析方法的核心工具？
A. 用例图
B. 类图
C. 数据流图
D. 顺序图

- 答案：C

【选择】在面向对象需求分析中，用于描述系统功能边界的图是？

A. 类图
B. 用例图
C. 状态图
D. 部署图

- 答案：B

【选择】以下关于用例描述的说法，正确的是？

A. 用例描述只需要写正常流程

B. 用例描述不应包含异常情况

C. 用例描述应包含前置条件、基本流程和备选流程

D. 用例描述不需要后置条件

- 答案：C

【选择】数据字典的主要作用是?

A. 记录数据库表结构

B. 对数据流图中的数据元素进行详细定义

C. 存储用户密码

D. 记录系统日志

- 答案：B

【选择】判定表适合描述什么类型的业务规则？
A. 简单的顺序执行逻辑
B. 多条件组合的复杂逻辑
C. 单循环逻辑
D. 纯算术运算

- 答案：B

【选择】在实体-关系图中，连接线表示的是？
A. 实体
B. 属性
C. 关系
D. 主键

- 答案：C

【选择】以下哪项属于非功能性需求？
A. 用户可以登录系统
B. 系统响应时间不超过3秒
C. 用户可以查询订单
D. 管理员可以添加商品

- 答案：B

【选择】下面具有继承关系的一组是（ ）。
A．铅笔-钢笔　　B．冰箱-空调　C．汽车-客车　D．教师-学生

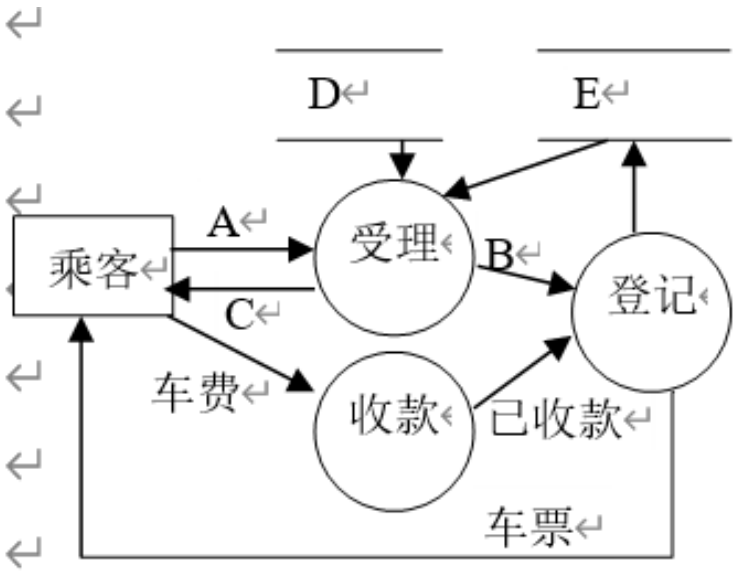
- 答案：C

【选择】汽车有一个发动机。汽车和发动机之间的关系是_____关系。
A．一般与具体
B．整体与部分
C．分类关系
D．主从关系

- 答案：B

【填空】将下列各事物之间的关系（继承、聚集或一般关联）写在括弧
(1) 小汽车——富康牌小汽车。 (继承)
(2) 人员——雇员。 (继承)
(3) 图书馆——期刊阅览室。 (聚集)
(4) 书——图书馆馆员。 (一般关联)
(5) 小汽车——司机。 (一般关联)
(6) 读者——借出的书。 (一般关联)
(7) 班级——学生。 (聚集)
(8) 教师——教授。 (继承)
(9) 丈夫——妻子。 (一般关联)
(10) 列车——餐车。 (聚集)

【简答】开发一个铁路售票系统，请完成图1所示的数据流图，从供选择的答案中选出A、B、C、D、E的内容。供选择的答案：车次表、接受的请求、售票记录、购票请求、拒绝的请求



- 答案：A购票请求、B接受、C拒绝的请求、D车次表、E 售票记录。

【简答】请用CRUD方法给出购物网站中用户类（Customer）常见的四个用例

- 答案:
- 添加用户，删除用户，更改用户信息，查询用户信息
- 考点：CRUD 增删改查

【应用】某图书馆计划开发一个图书管理系统，需求如下：
读者可以查询图书、借阅图书、归还图书、续借图书、预约图书
管理员可以管理读者信息（增删改查）、管理图书信息（增删改查）、处理逾期罚款
读者借书时，系统需验证读者资格（是否逾期、是否达上限）
读者借书时，系统需验证图书状态（是否在库）
教师借阅数量上限为10本，学生为5本
逾期罚款规则：每本每天0.2元，最高罚款50元

请回答：
（1）识别该系统的利益相关者，并分类说明（操作利益相关者/执行利益相关者、内部/外部）。
（2）绘制该系统的用例图（用文字描述即可）。
（3）为“借阅图书”用例编写详细的用例描述。
（4）用判定表描述读者借书资格的验证规则（考虑：是否逾期、是否达上限、读者类型）。

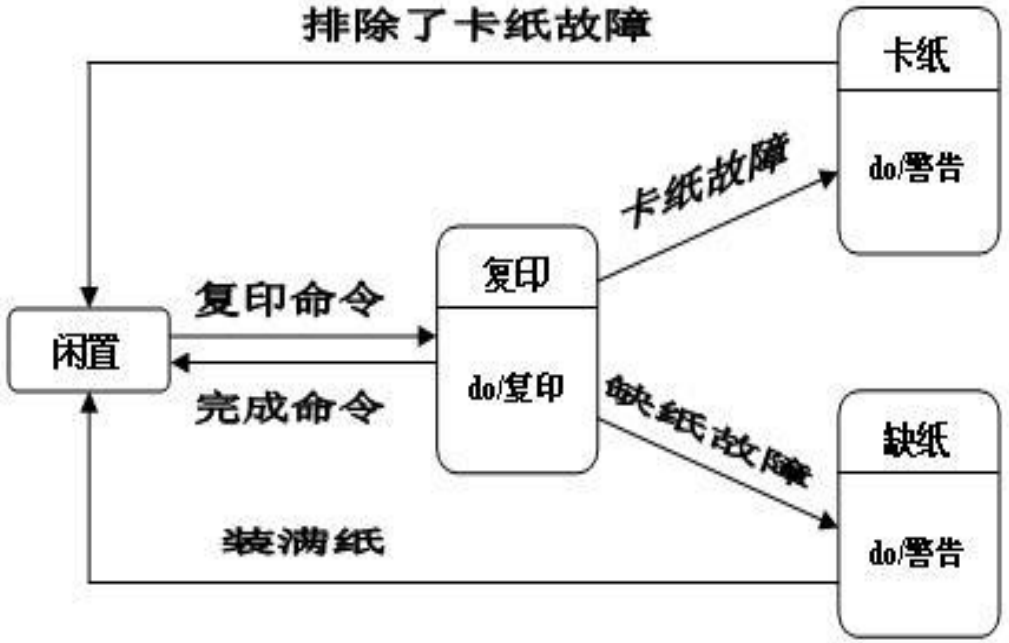
【应用】某图书馆计划开发一个图书管理系统，需求如下：
读者可以查询图书、借阅图书、归还图书、续借图书、预约图书
管理员可以管理读者信息（增删改查）、管理图书信息（增删改查）、处理逾期罚款
读者借书时，系统需验证读者资格（是否逾期、是否达上限）
读者借书时，系统需验证图书状态（是否在库）
教师借阅数量上限为10本，学生为5本
逾期罚款规则：每本每天0.2元，最高罚款50元

（1）利益相关者：读者（操作、外部）、管理员（操作、内部）、图书馆管理者（执行、内部）
（2）用例图包含：读者参与者，用例包括图书查询、借阅图书、归还图书、续借图书、预约图书；管理员参与者，用例包括读者管理、图书管理、罚款处理
（3）用例描述应包含：用例名称、参与者、前置条件、基本流程、备选流程、后置条件
（4）判定表：条件包括是否逾期、是否达上限、读者类型，动作包括允许借阅、拒绝借阅

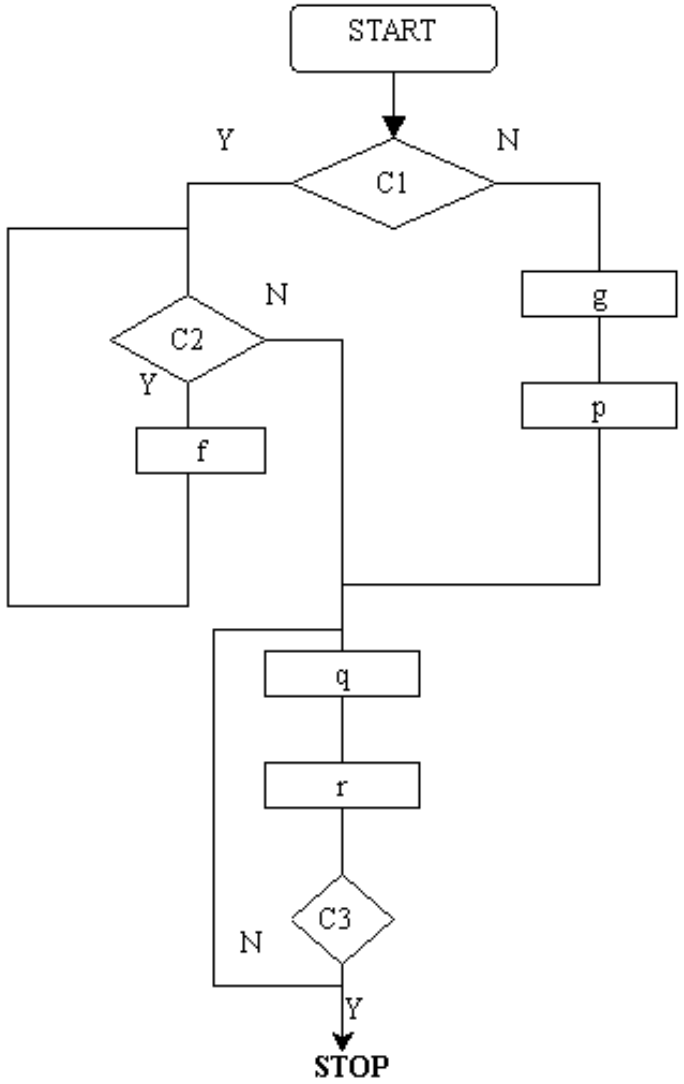
【应用】办公室复印机的工作过程大致如下：未接到复印命令时处于闲置状态，一旦接到复印命令则进入复印状态，完成一个复印命令规定的工作后又回到闲置状态，等待下一个复印命令；如果执行复印命令时发现缺纸，则进入缺纸状态，发出警告，等待装纸，装满纸后进入闲置状态，准备接受复印命令；如果复印时发生卡纸故障，则进入卡纸状态，发出警告等待维修人员来排队故障，故障排除后回到闲置状态。

请用状态转换图描绘复印机的行为：

- 答案：

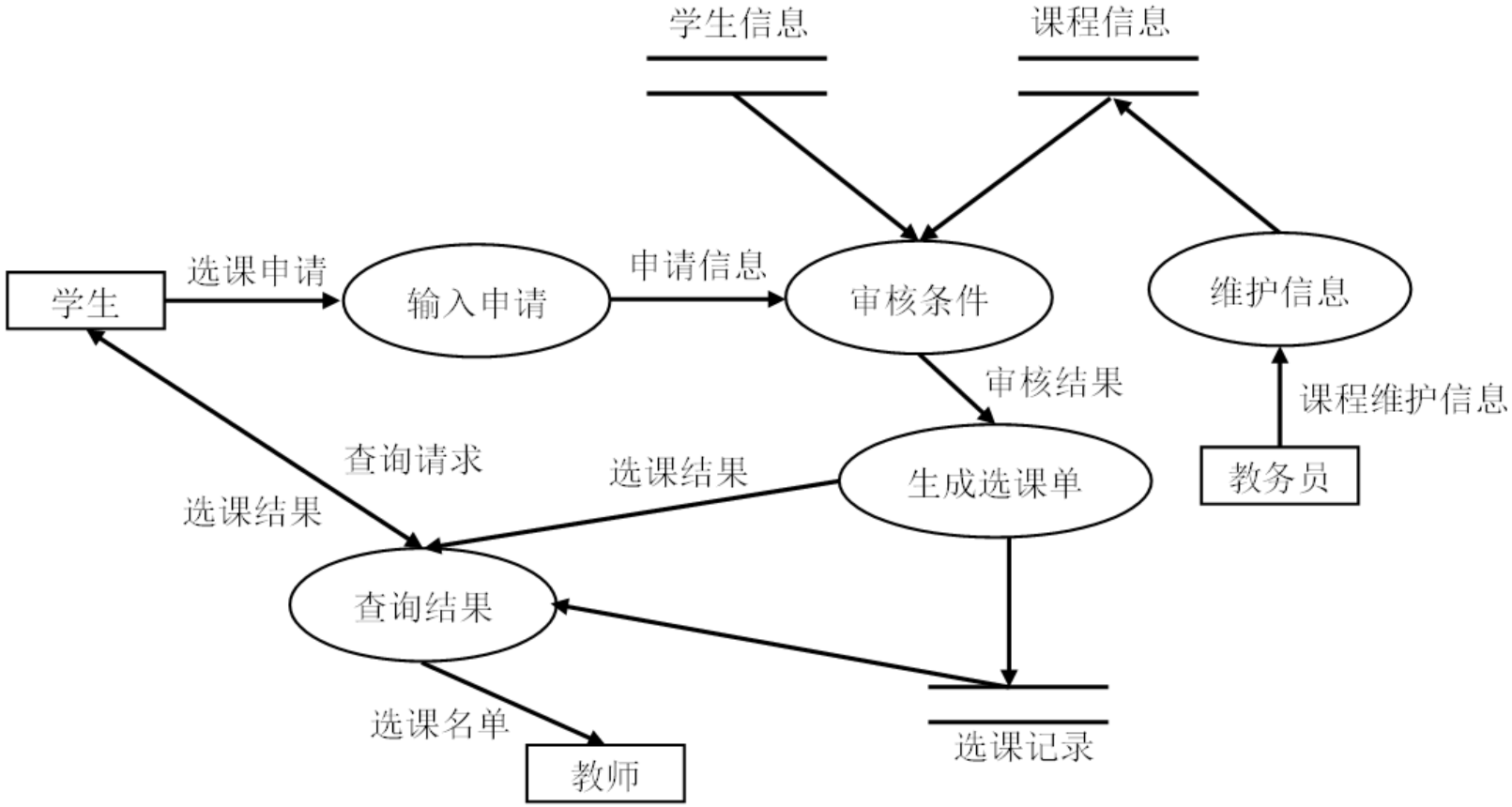


【应用】请根据下面的程序流程图，写出对应的结构化代码




```
START
  IF C1
     THEN
       WHILE C2 DO
          f
       ENDDO
   ELSE
       g
       p
   ENDIF
 REPEAT UNTIL C3
    q
    r
 ENDREP
 STOP
```

【应用】请根据下面的描述，画出学生选课管理系统的顶层数据流图：学生向系统提交选课申请，系统根据课程信息对选课申请进行处理，形成选课记录，并将选课结果返回给学生；教务员负责课程信息的维护。通过该图可以看出，数据流图能够较清楚地表示系统的数据来源、处理过程、存储位置和输出结果。



需求分析

- 2）事件分解技术

- 外部事件:外部事件发生在系统外部的事件，通常由外部代理或参与者发起

- 时间事件:时间事件由于到达某个时间点而发生的事件

- 状态事件:状态事件(内部事件):当系统内部发生某些事情时发生的事件（库存物品到达重新订购点）

大纲

- 软件工程概念
- 项目立项
- 需求分析
- 系统设计
- 系统实现
- 软件测试

系统设计

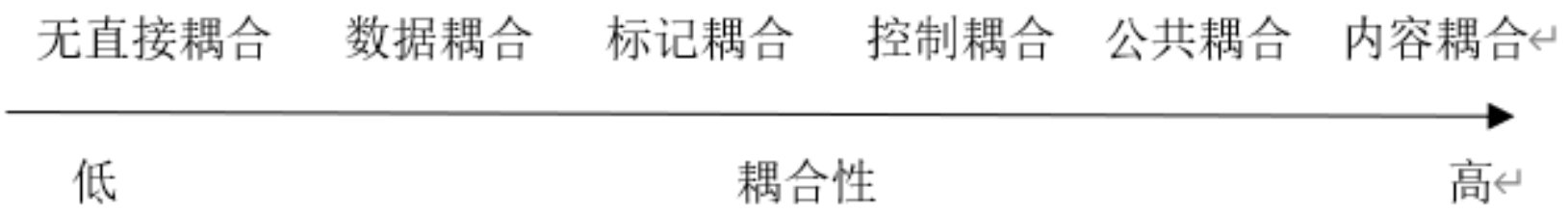
- **概念：**软件设计阶段的核心任务，是细化需求分析模型，最终形成设计模型和设计文档。
- **系统设计的两个阶段：**
- 概要设计：描述软件的总体体系结构，回答软件系统整体由哪些模块组成、模块间的关系如何、数据结构与数据库如何设计等宏观问题
- 详细设计：在确定总体体系结构后，对每个模块的具体执行过程、算法实现、数据定义等细节进行进一步细化设计，

系统设计

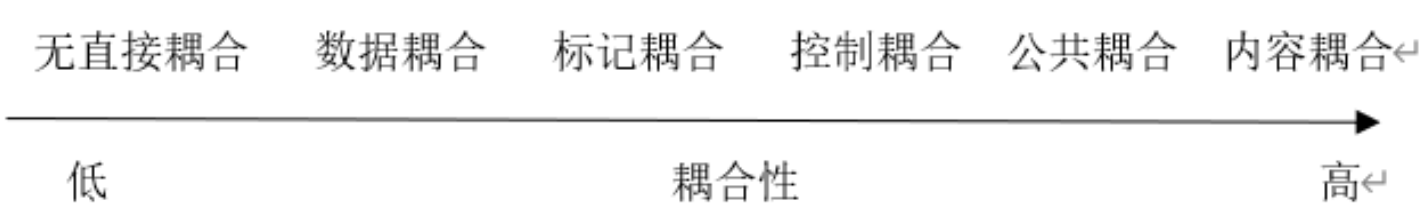
- **设计原理：**
- **1）模块化：**指将一个复杂的软件系统按照功能分解为若干相对独立、相互协作的模块，每个模块完成特定的子功能，通过模块之间的组合构成完整的系统。
- 2）**抽象**是认识复杂现象过程中使用的思维工具，即抽出事物本质的共同特性而暂不考虑它的细节，不考虑其他因素。
- 3）**信息隐蔽**(Information Hiding)是指将模块内部实现细节隐藏起来，仅通过公开的接口与外部进行交互。
- 4）**模块独立性**是指每个模块在功能上具有明确、单一的职责，并且模块之间的依赖关系尽可能弱。

系统设计

- **设计原理：**
- **模块独立性：**通常通过耦合性和内聚性两个维度来度量。
  - 耦合性(Coupling)衡量模块之间相互依赖的紧密程度。耦合性越低，模块之间的独立性越强，系统越易于维护和修改。模块之间联系越紧密，其耦合性就越强，模块的独立性则越差。



系统设计



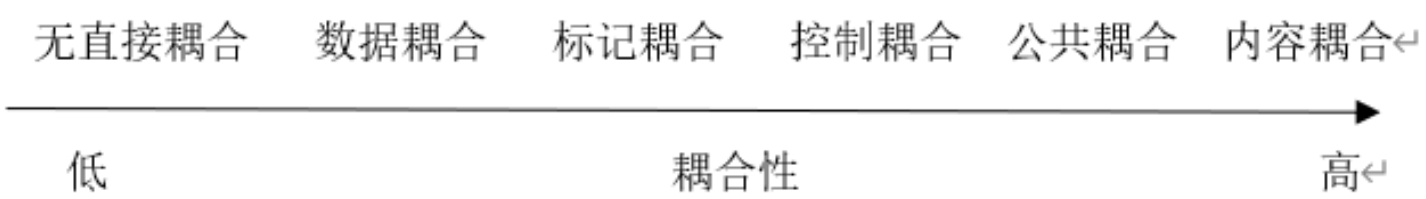
- **耦合**

  数据耦合：两个模块彼此间通过参数交换信息，而且交换的信息仅仅是数据，那么这种耦合称为数据耦合。数据耦合是低耦合。系统中至少必须存在这种耦合

  控制耦合：传递的信息中有控制信息(尽管有时这种控制信息以数据的形式出现)，则这种耦合称为控制耦合。控制耦合是中等程度的耦合。

  公共耦合：当两个或多个模块通过一个公共数据环境相互作用时，它们之间的耦合称为公共环境耦合。公共环境可以是变量、共享的通信区、内存的公共覆盖区、任何存储介质上的文件、物理设备等。

系统设计



- **耦合**

内容耦合：最高程度的耦合是内容耦合。如果出现下列情况之一，两个模块间就发生了内容耦合。

一个模块访问另一个模块的内部数据。

一个模块不通过正常入口而转到另一个模块的内部。

两个模块有一部分程序代码重叠(只可能出现在汇编程序中)。

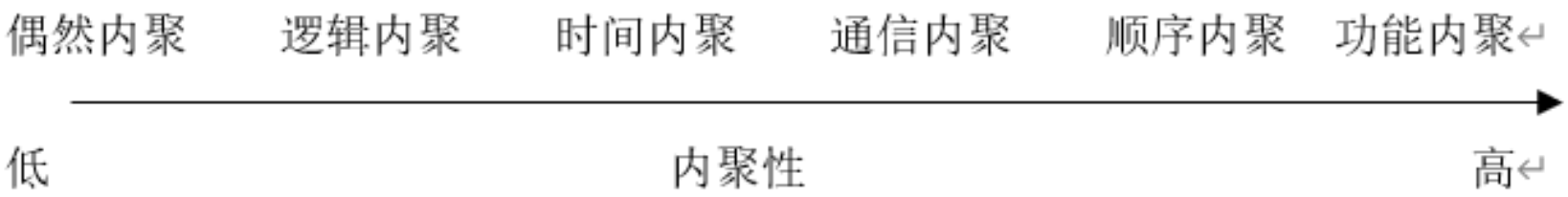
一个模块有多个入口(这意味着一个模块有几种功能)。

应该采取下述设计原则：

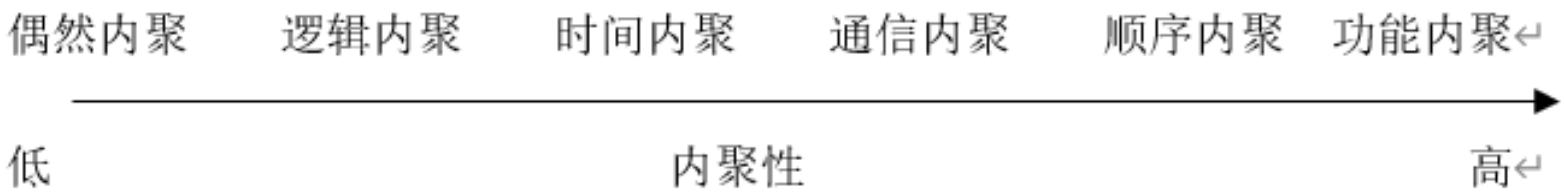
尽量使用数据耦合，少用控制耦合，限制公共环境耦合的范围，完全不用内容耦合。

系统设计

- **设计原理：**
- **模块独立性：**通常通过耦合性和内聚性两个维度来度量。
  - 内聚性(Cohesion)衡量模块内部各元素之间结合的紧密程度。内聚性越高，模块内部元素越相关，模块的职责越单一。



系统设计



- 内聚

巧合性内聚：当模块内各部之间没有联系，或者即使有联系，这种联系也很松散。

逻辑性内聚：这种模块是把几种功能组合在一起，每次调用时，则由传递给模块的判定参数来确定该模块应执行哪一种功能。

时间内聚：一个模块包含的任务必须在同一段时间内执行，就叫时间内聚。

过程内聚：一个模块内的处理元素是相关的，而且必须以特定次序执行，则称为过程内聚。

通信内聚：模块中所有元素都使用同一个输入数据和(或)产生同一个输出数据，则称为通信内聚。

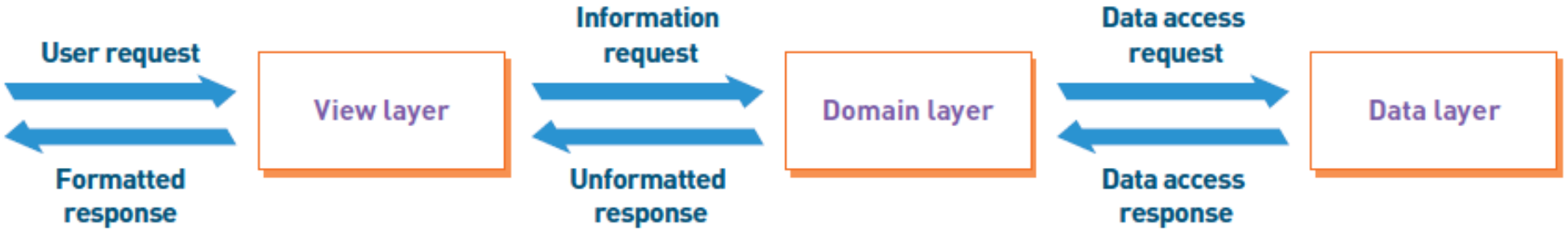
功能性内聚：模块内所有处理元素属于一个整体，完成一个单一的功能，则称为功能内聚。功能内聚是最高程度的内聚。

系统设计

- 概要设计:
- 系统架构设计是概要设计的核心活动，它定义了软件系统的整体结构和组织方式。
- 1）**集中式架构**是指所有计算任务和处理功能都集中在一个中心节点（如大型主机或服务器）上，客户端仅负责输入输出和结果显示。
- 2）**分布式架构**是指系统的不同组件分布在多个独立的计算机节点上，节点之间通过网络进行通信和协作，共同完成系统功能。
- 3）**分层架构**将系统按照功能层次组织为若干层，每一层提供特定的服务，并为上层提供接口。

系统设计

- 概要设计:
- 经典的**三层架构**包括
- **表示层**（负责用户界面和交互逻辑）
- **业务逻辑层**（负责业务规则和数据处理）
- **数据访问层**（负责数据库操作和数据存取）。



设计-其他知识点

- **有哪些Internet部署的托管替代方案？**

- **场地出租：**主机托管一种具有安全位置的托管服务，但其中的计算机通常由客户企业拥有
- **管理服务：**客户端拥有软件，但可能需要购买额外的服务，例如安装和管理操作系统、Internet服务器、数据库服务器和负载平衡软件
- **虚拟服务器：**客户公司租用一台虚拟服务器，该服务器配置为实服务器，具有一定的CPU容量、内部内存、硬盘内存和带宽，供Internet使用
- **云计算：**
  - 一种虚拟服务器的扩展，其中可用的资源包括计算、存储和Internet访问，并且它们似乎具有无限的可用性。
  - 客户机应该能够购买计算能力，就像购买水或电这样的公用事业一样。
  - 客户机不应该关心诸如如何或在何处提供计算能力等环境问题，就像个人不必担心如何产生电力一样

设计-其他知识点

- **3. 有哪些Internet部署的托管替代方案?**

场地和设施

客户拥有设备?

客户管理配置?

可扩展性

维护

备份与恢复

HOSTING OPTIONS				
Service options	**Colocation**	**Managed services**	**Virtual servers**	**Cloud computing**
Hosting service provides building and infrastructure	Yes	Yes	Yes	Yes
Client owns computer	Yes	Perhaps	No	No
Client manages computer configuration	Yes	No	Possible	No
Scalability	Client adds more computers	Client adds more computers	Client buys larger or more virtual servers	Client adds small increments of computing power
Maintenance	Client provides	Host provides	Host provides	Host provides
Backup and recovery	Client provides	Host provides	Available	Available

系统设计

- 概要设计:
- **设计原则**:
- 关注点分离原则：要求将系统划分为职责清晰、相互独立的部分
- 高内聚、低耦合原则：要求模块内部各元素之间紧密关联，模块之间松散依赖，降低修改一个模块对其他模块的影响。
- 可扩展性原则：要求架构应支持在不影响现有功能的前提下增加新功能，能够适应业务需求的演化。
- 可重用性原则要求识别系统中的通用功能，设计为可复用的组件，避免重复开发。

系统设计

- 概要设计:
- 架构设计原则:
- 最小知识原则要求每个模块应尽可能少地了解其他模块的内部细节，只通过公开接口进行交互。
- 渐进式设计原则要求架构设计应支持增量开发和迭代交付，能够分阶段实现系统功能。

系统设计

- 详细设计:
- 详细设计的主要内容包括模块内部逻辑设计、数据结构详细设计、算法设计、数据库详细设计、用户界面设计以及安全与性能设计等方面

系统设计

- 详细设计:
- 人机交互设计（Human-Computer Interaction Design，HCI Design）是详细设计阶段的重要组成部分，它定义了用户与软件系统之间的交互方式和界面表现形式。

系统设计

- 详细设计:
- 人机交互设计原则:
- 可用性，系统应易于学习、易于使用、易于记忆;
- 反馈原则：要求系统对用户的每个操作都给出明确的响应，让用户知道操作已被接收、系统正在处理以及处理结果。
- 容错性，系统应防止用户操作错误，或在错误发生时减少损失;
- 效率原则：要求系统应支持高效的操作方式，满足不同用户的需求。
- 一致性，相似的操作应有相似的交互方式，降低用户的学习成本。

设计

- **结构化方法：**
- 程序流程图
- PAD图
- NS图

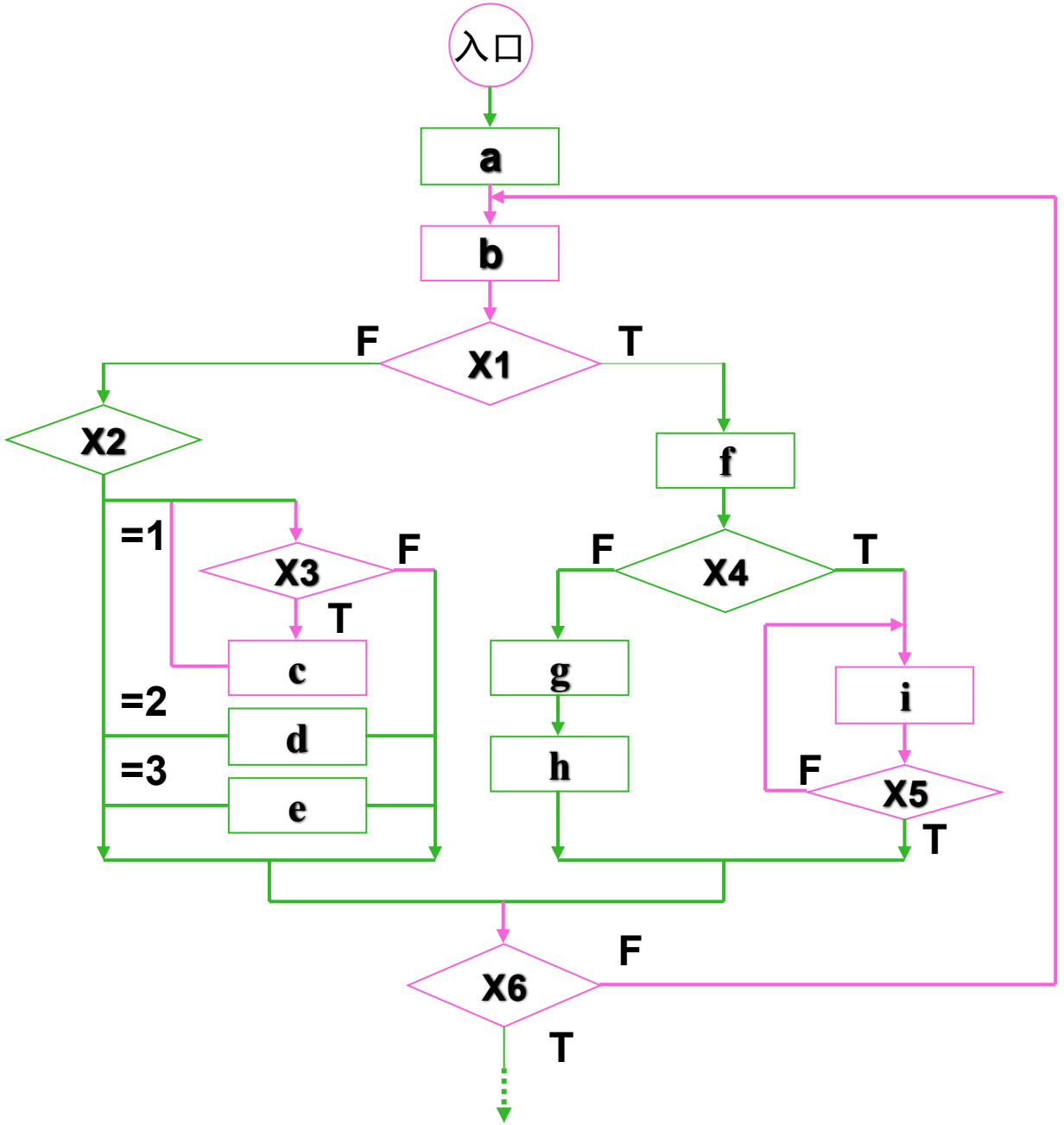
- **面向对象：**
- 设计类图（Design Class Diagram）
- 顺序图（Sequence Diagram）
- 协作图（Communication Diagram）
- 包图（Package Diagram）

设计

- **结构化方法：**
- 程序流程图
- PAD图
- NS图

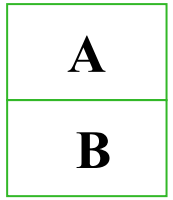
设计

- 结构化方法:
- 程序流程图

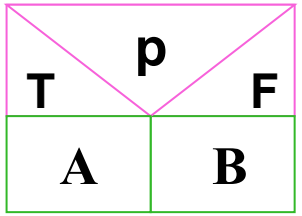


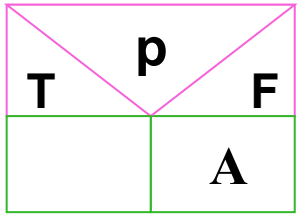
设计

- N-S图
- 也被称为盒图或NS图，是结构化编程中的一种可视化建模

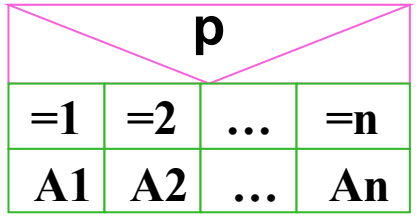


顺序型

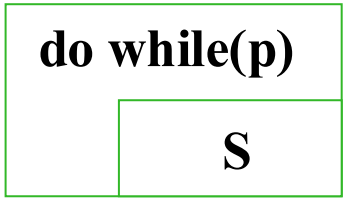




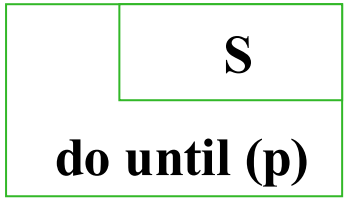
选择型



多分支选择型



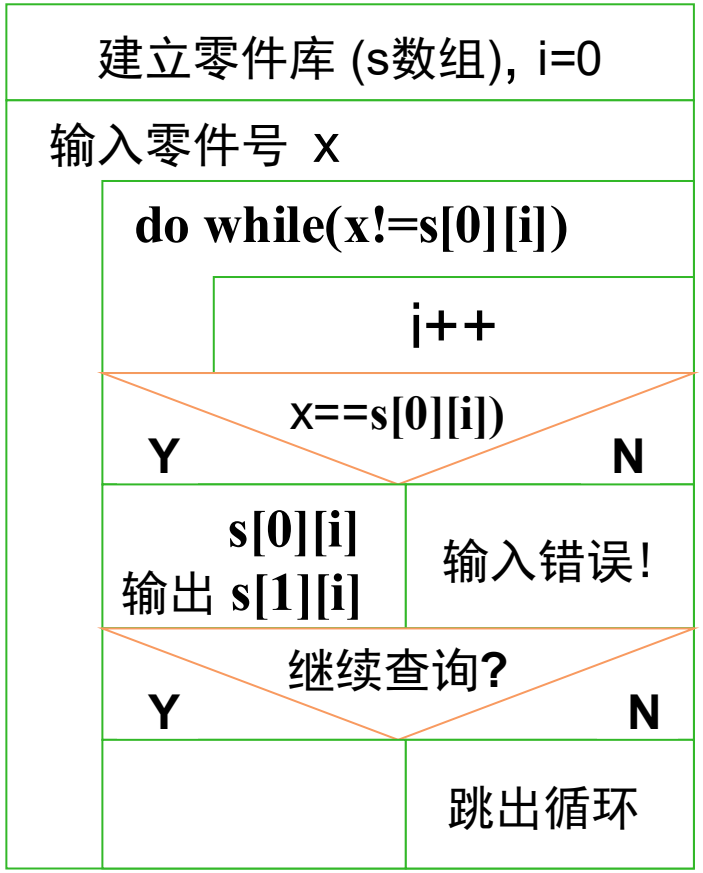
当型循环型



直到型循环型

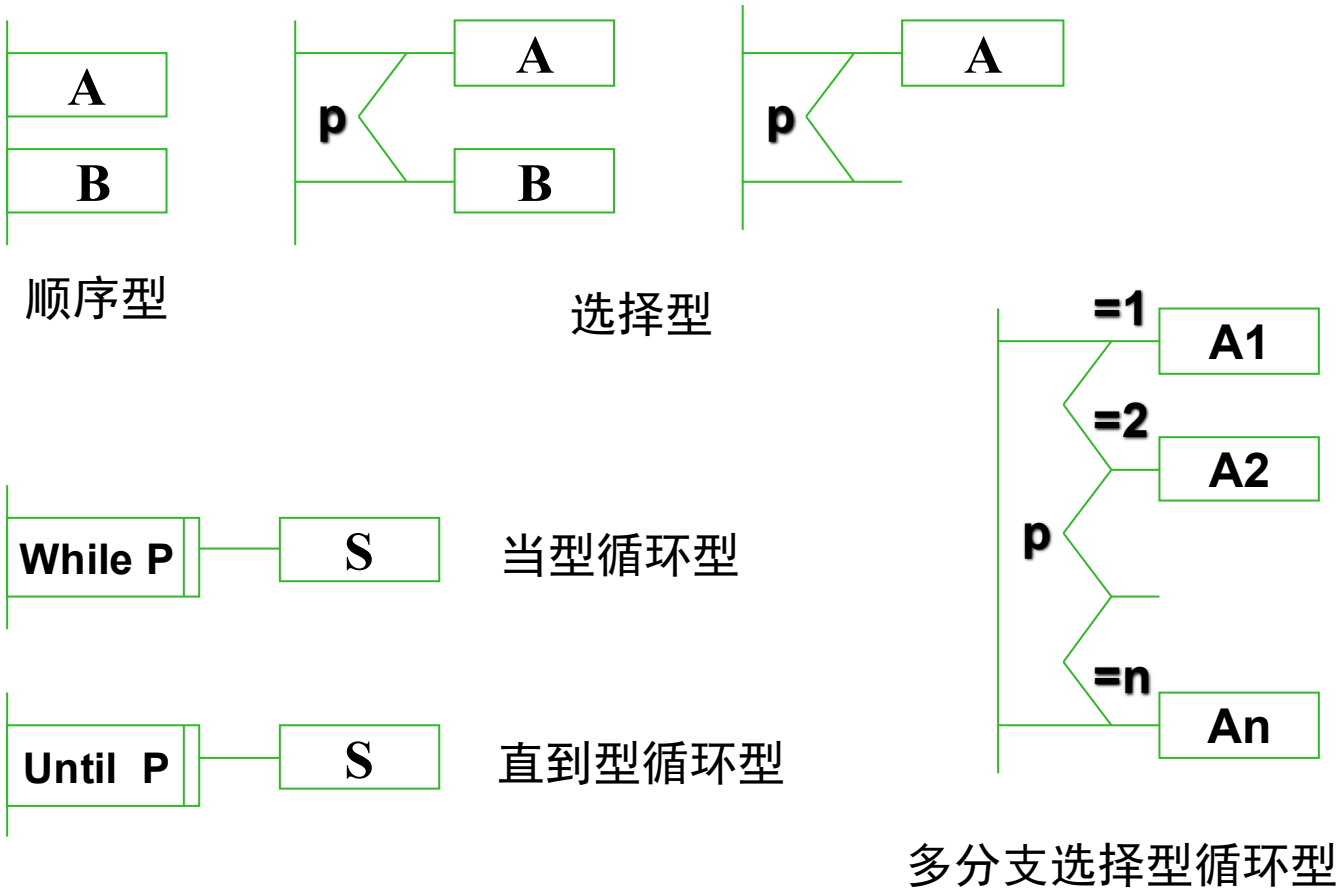
设计

- N-S图
- 也被称为盒图或NS图，是结构化编程中的一种可视化建模



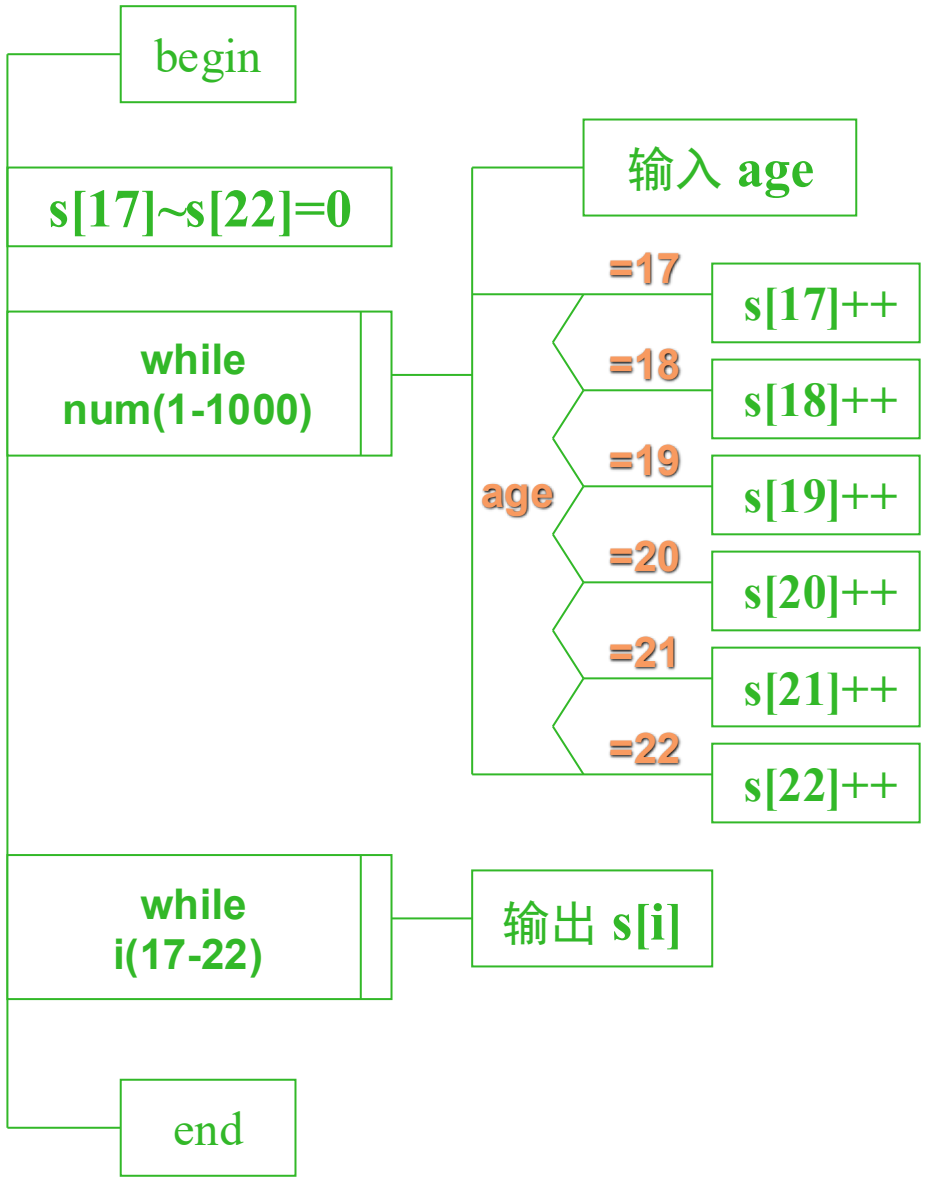
设计

- PAD图


A
B
顺序型
p
A
B
选择型
p
A
While P
S
当型循环型
Until P
S
直到型循环型
p
=1
A1
=2
A2
=n
An
多分支选择型循环型

设计

- PAD图

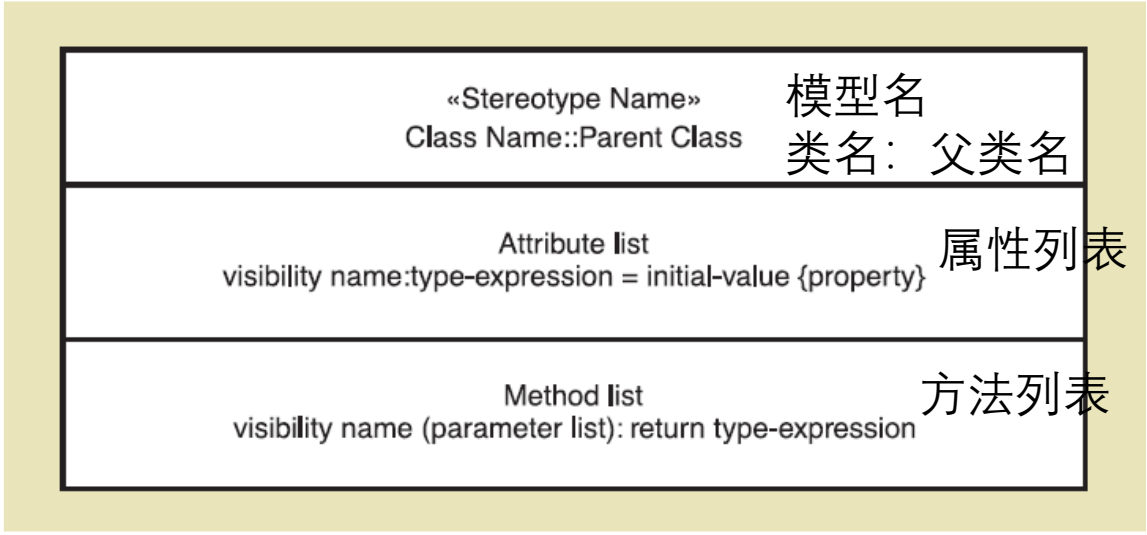

begin
s[17]~s[22]=0
while
num(1-1000)
输入 age
age
=17
s[17]++
=18
s[18]++
=19
s[19]++
=20
s[20]++
=21
s[21]++
=22
s[22]++
while
i(17-22)
输出 s[i]
end

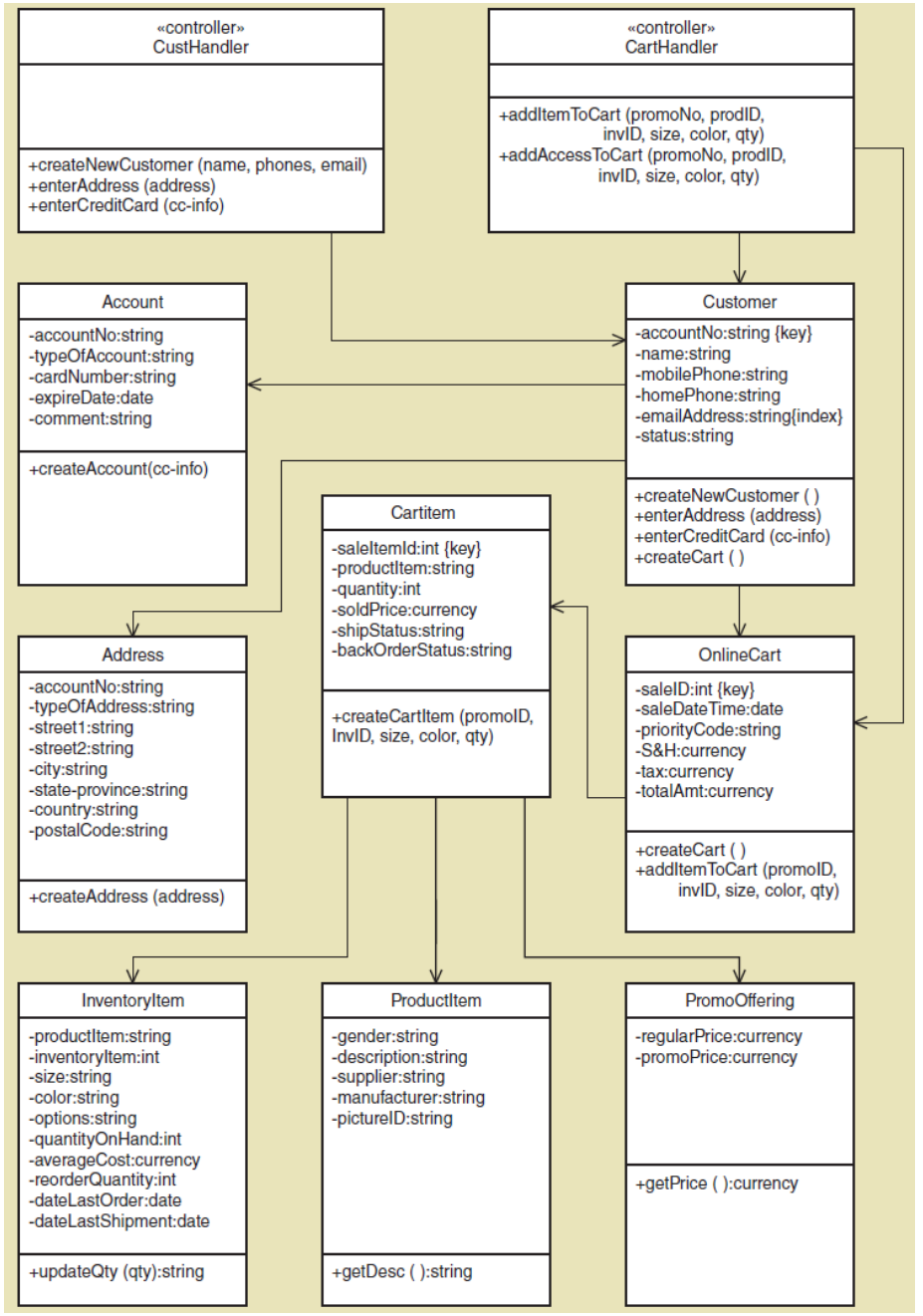
设计

- **面向对象：**
- 设计类图（Design Class Diagram）
- 顺序图（Sequence Diagram）
- 协作图（Communication Diagram）
- 包图（Package Diagram）

设计

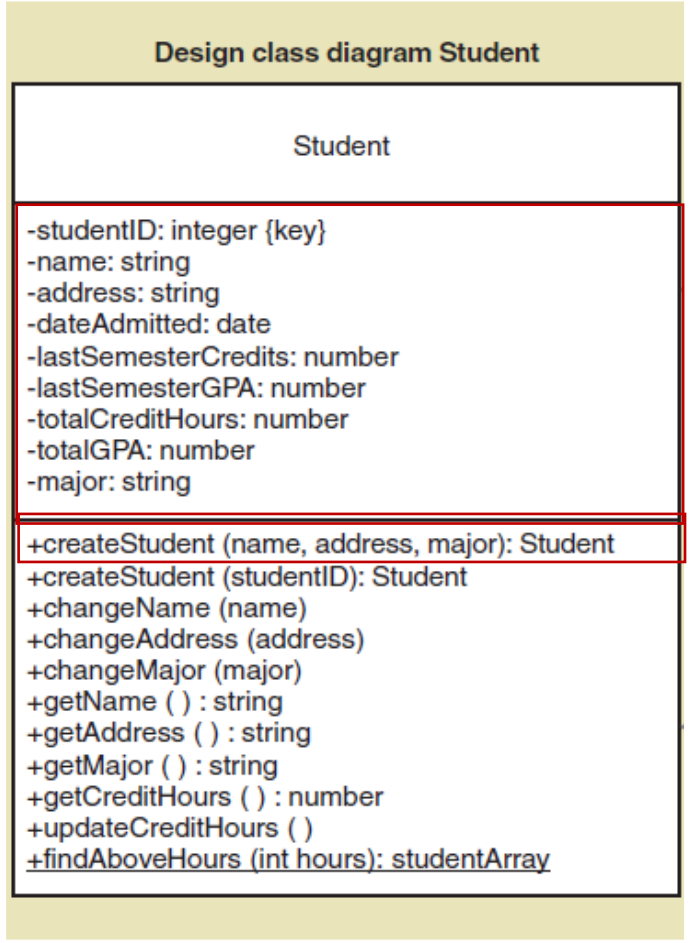
- 设计类图（Design Class Diagram）
  - 箭头：导航可见性
  - 属性 + 方法：
    - “+”：public
    - “-”：private
  - 类的类型：实体类、视图类、控制器类、数据访问类





设计-其他知识点

- 设计类图：**请根据图写出对应的类的代码**

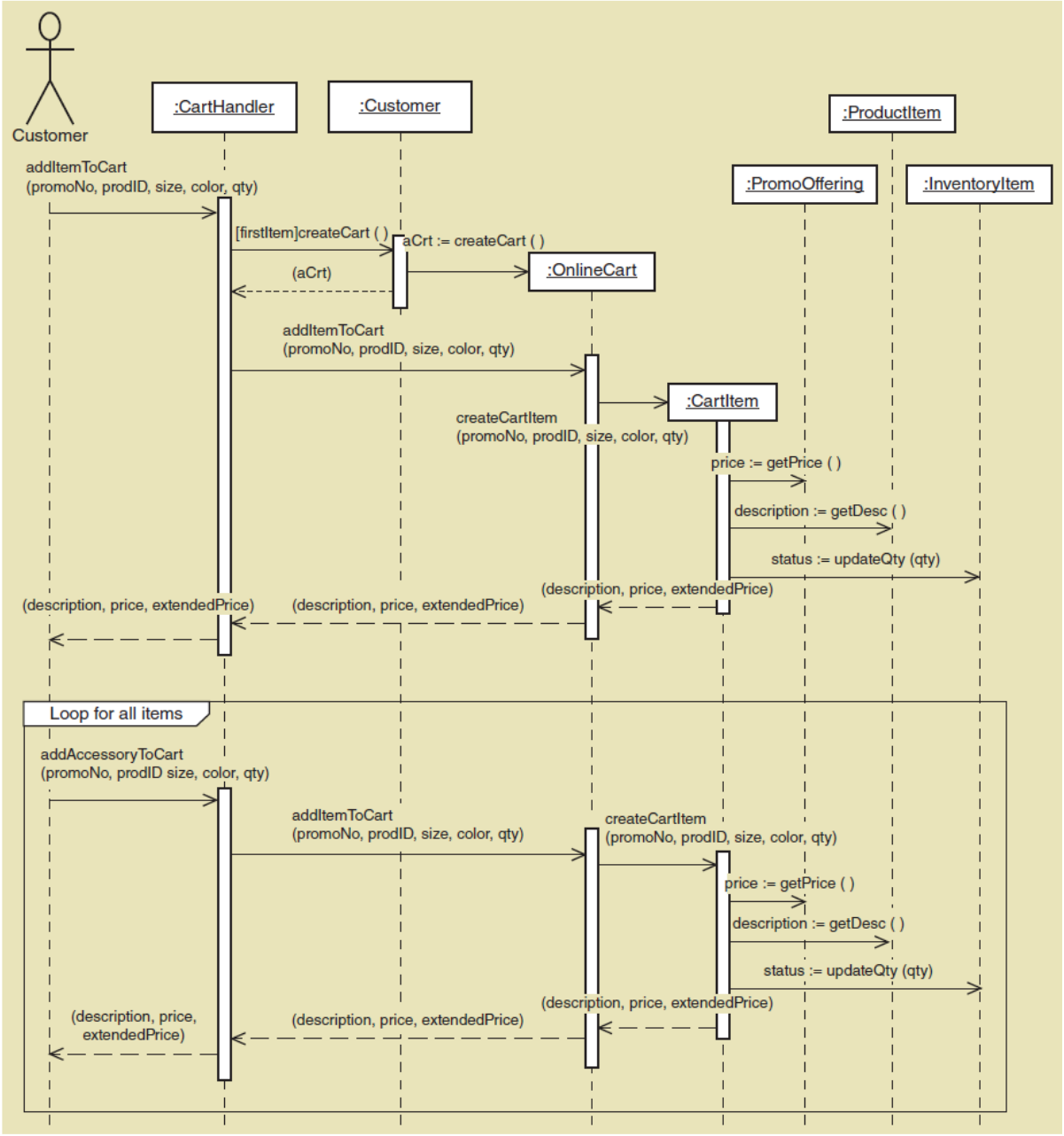


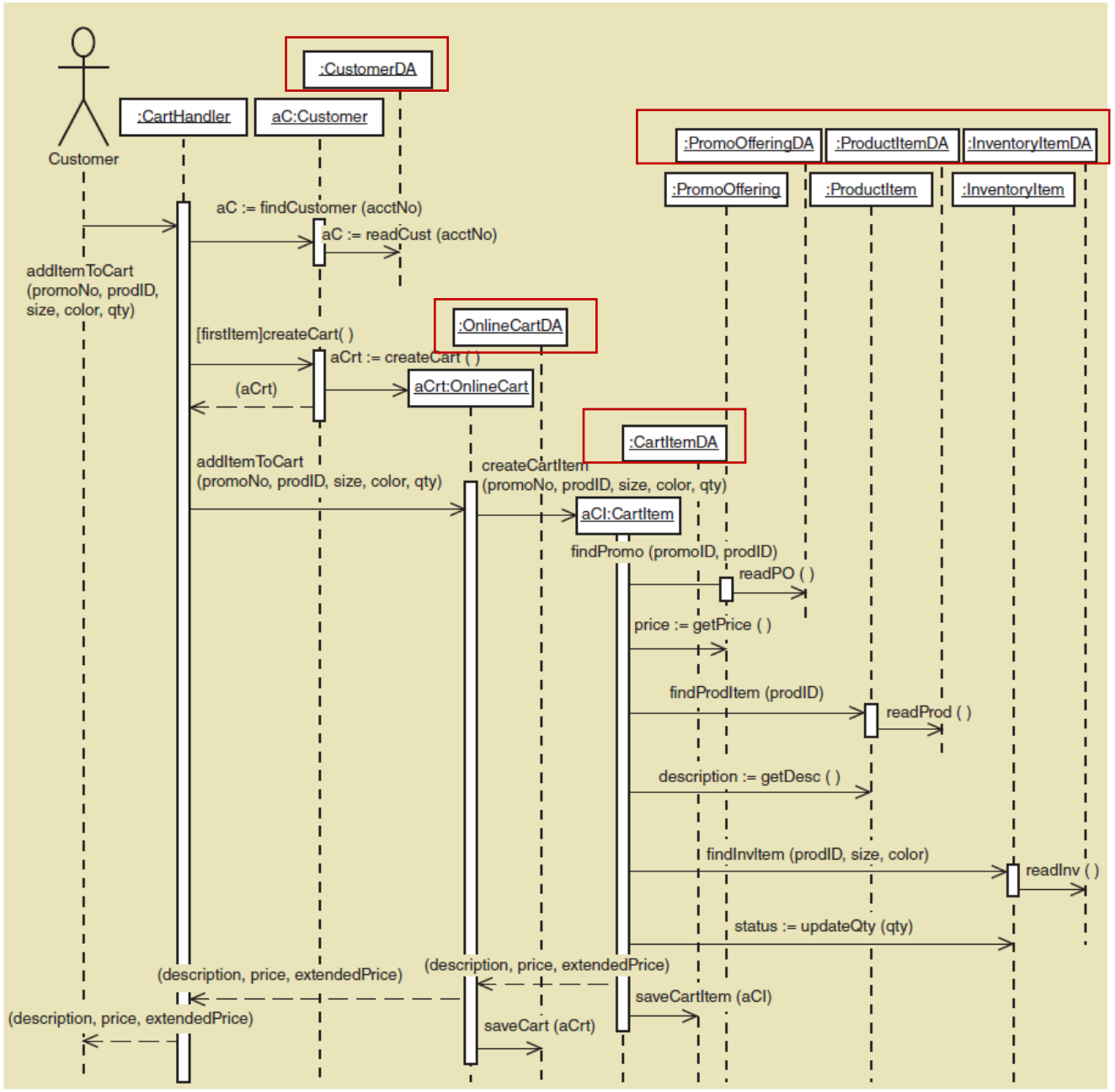

```
public class Student
{
	//attributes
	private int studentID;
	private String firstName;
	private String lastName;
	private String street;
	private String city;
	private String state;
	private String zipCode;
	private Date dateAdmitted;
	private float numberCredits;
	private String lastActiveSemester;
	private float lastActiveSemesterGPA;
	private float gradePointAverage;
	private String major;

	//constructors
	public Student (String inFirstName, String inLastName, String inStreet,
		String inCity, String inState, String inZip, Date inDate)
	{
		firstName = inFirstName;
		lastName = inLastName;
		...
	}
```

设计

- 顺序图（Sequence Diagram）
  - 将系统顺序图拆分成具体的类/对象
  - 命名：
    - ：类名
    - 对象名：类名
  - 箭头：
    - 一个方向为Method（Parameter），一个方向为返回值
  - 垂直虚线：生命线
  - 白色竖直方框：Use Case生命线



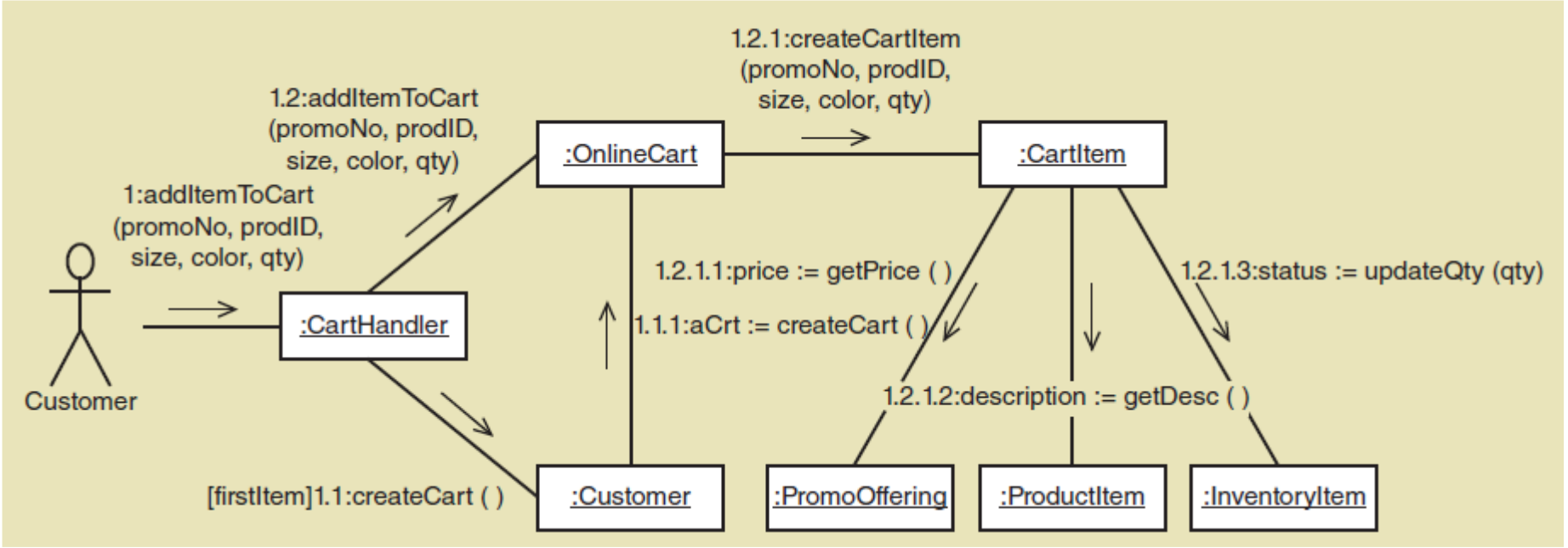


考虑三层架构的顺序图:

CartHandler findCustomer发消息给Customer类意味着Customer应该创建一个名为aC的新实例，向:CustomerDA发送消息，要求它从数据库中读取具有该账号的客户的信息，然后用数据库中的属性值填充新的客户实例。

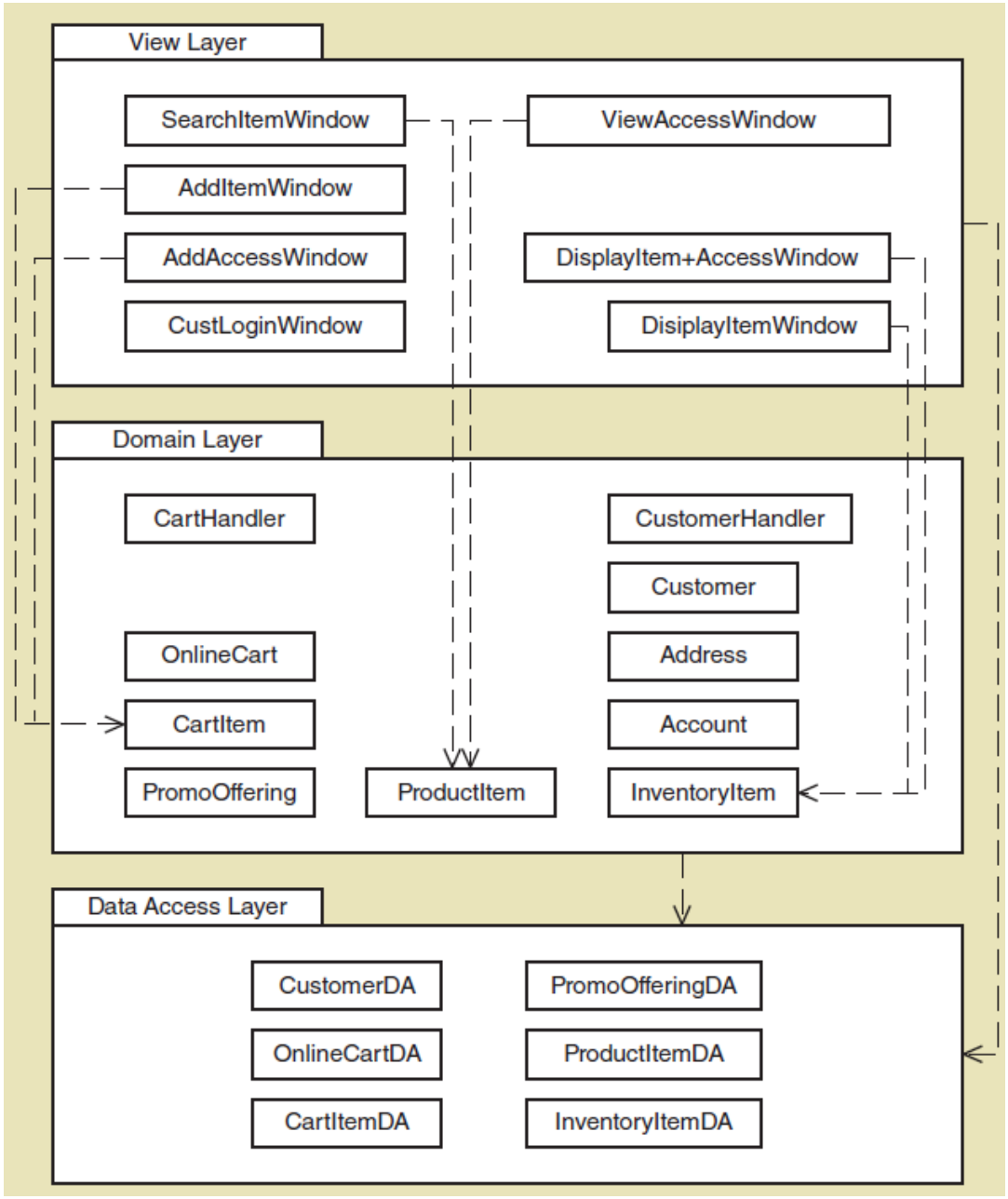
设计

- 协作图（Communication Diagram）
  - 与顺序图表示的内容是一致的
  - 用标号表示顺序
  - 语法：[true/false condition]sequence-number: return-value :=message-name(parameter-list)



设计

- 包图（Package Diagram）
  - 分为View Layer, Domain Layer和Data Access Layer
  - 箭头：依赖关系



【选择】1. 软件设计中划分模块的一个准则是（　　）。

A、　低内聚低耦合

B、　低内聚高耦合

C、　高内聚低耦合

D、　高内聚高耦合

- 答案：　C

【选择】以下哪个不是详细设计中常用的图形描述工具　　。

A、数据流图

B、程序流图

C、盒图（N-S图）

D、问题分析图（PAD）

- 答案： A

【选择】软件设计包括 （ ）两个阶段。

A、接口设计和结构设计　　B、数据设计和概要设计

C、结构设计和过程设计　　D、概要设计和详细设计

- 答案： D

【选择】在UML中，协作图反映收发消息的对象的结构组织，它与（）是同构的

A 用例图

B 顺序图

C 活动图

D 类图

- 答案： B

【选择】内聚是衡量模块内部聚合能力的量度。内聚越高，说明模块内各成分彼此联系的程度(　)
A．越松散
B．彼此相等
C．越紧密
D．没有联系

- 答案：C

【选择】模块本身的内聚是模块独立性的重要度量因素之一，在七类内聚中，具有最强内聚的一类是____。
A．顺序性内聚
B．过程性内聚
C．逻辑性内聚
D．功能性内聚

- 答案：D

【选择】如果模块中所有成分引用共同的数据，该模块内的联系类型是____。
A．顺序内聚
B．功能内聚
C．通信内聚
D．过程内聚

- 答案：C

【选择】某模块的功能是打印录取通知书或不录取通知书，调用模块可通过向此模块传递布尔标志信息决定本次调用哪种通知书的打印，这种模块之间的耦合称为____。
A．数据耦合
B．混合耦合
C．公共耦合
D．控制耦合

- 答案：D

【选择】要减少两个模块之间的耦合，则必须____。
A．两个模块间的调用次数要少
B．模块间传递的参数要少
C．模块间传递的参数要少且布传递开关型参数
D．模块间传递的参数要少且不传递开关型参数以及两模块不引用同样的全局变量

- 答案：D

【选择】内聚是衡量模块内部聚合能力的量度。内聚越高，说明模块内各成分彼此联系的程度(　)
A．越松散
B．彼此相等
C．越紧密
D．没有联系

- 答案：C

【选择】软件设计中划分模块的一个准则是（　　）。

A、　低内聚低耦合

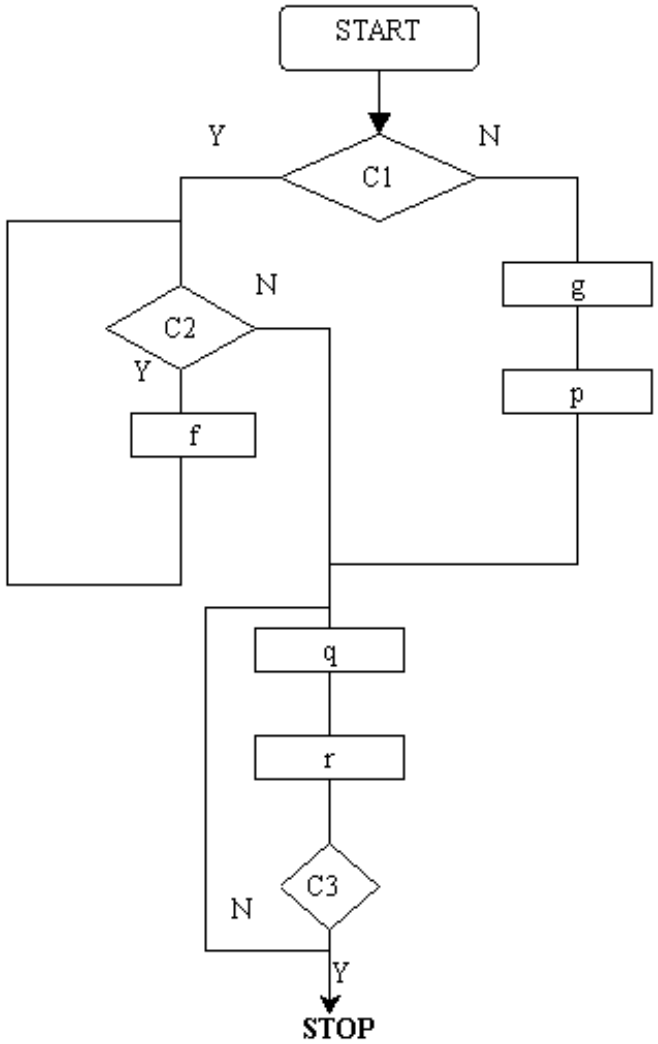
B、　低内聚高耦合

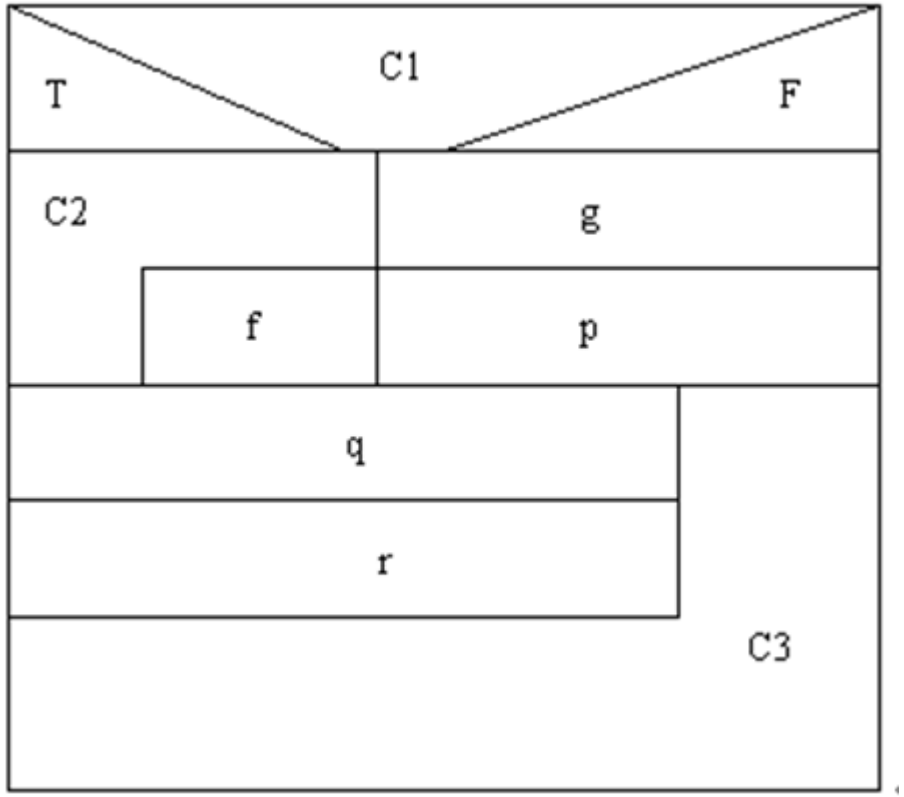
C、　高内聚低耦合

D、　高内聚高耦合

- 答案：C

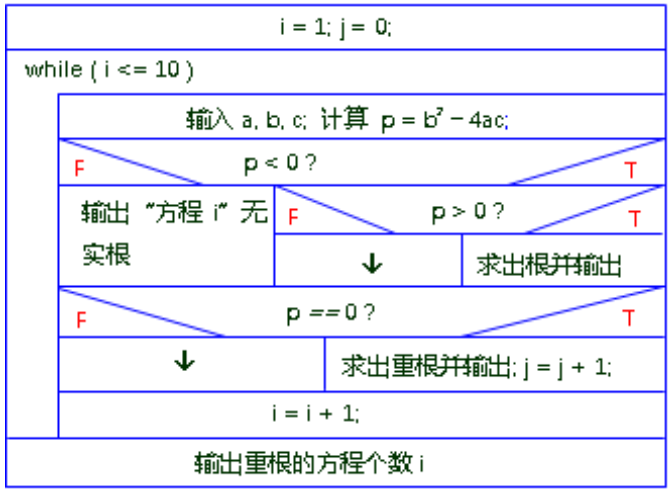
【应用】根据下面的程序流程图，画出NS图

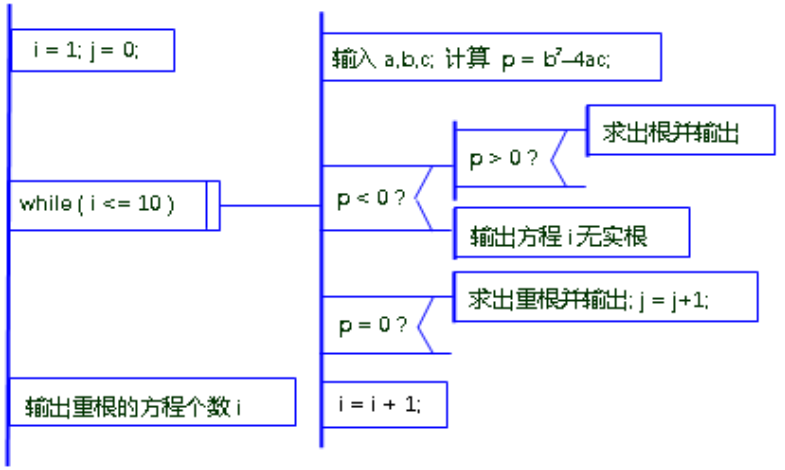




【应用】将下面给出的伪码转换为N-S图和PAD图。

```
void root ( float root1, float root2 ) {
   i = 1;  j = 0;
   while ( i <= 10 ) {
输入一元二次方程的系数a, b, c;
p = b*b – 4*a*c;
if ( p < 0 ) 输出“方程i无实数根”;
else if ( p > 0 ) 求出根并输出;
if ( p == 0 ) {
   求出重根并输出;
   j = j + 1;
}
i = i +1;
   }
   输出重根的方程的个数j;
}
```





大纲

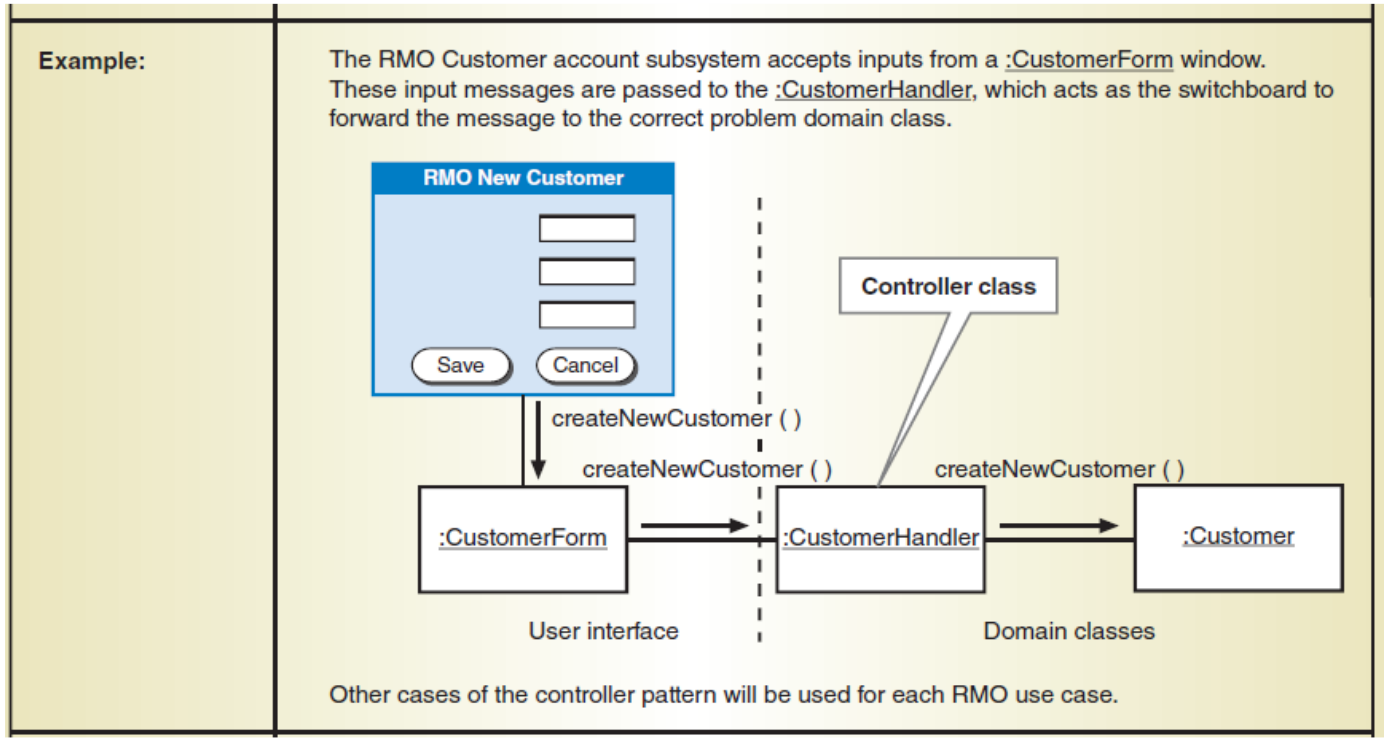
- 软件工程概念
- 项目立项
- 需求分析
- 系统设计
- 系统实现
- 软件测试

实现

- 设计模式
  - Controller（控制器）
  - Adapter（适配器）
  - Singleton（单例）
  - Factory（工厂）

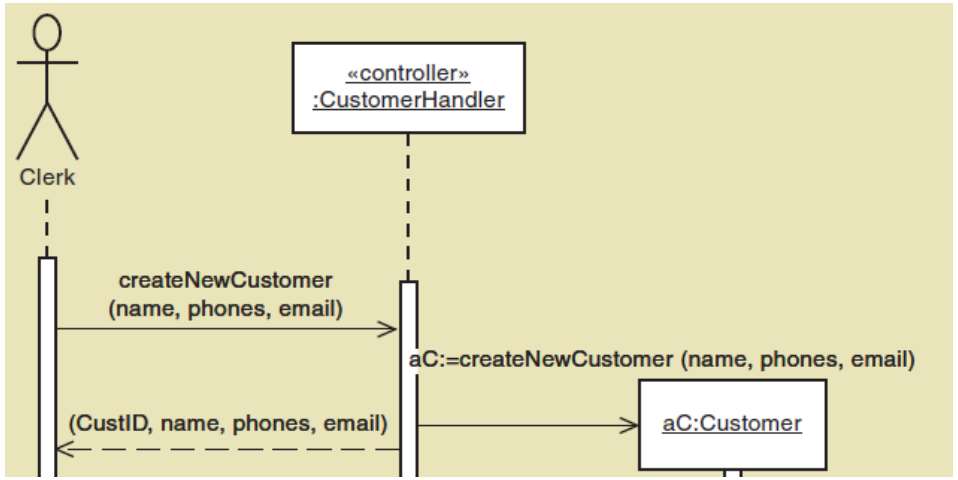
实现

- 设计模式：
  - Controller（控制器）： 为了使View Layer和Domain Layer的类更松耦合



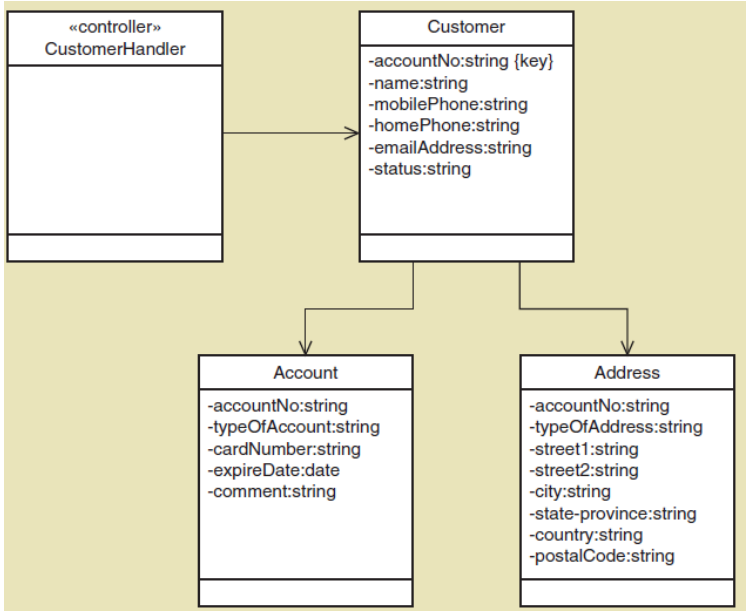
© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

实现控制器代码:




```
Public Class CustomerHandler {
  Private Custormer aC;
  Public cusInfo createNewCustomer(string name, string phone, string email) {
     aC = Customer.createNewCustomer(name,phone,email);
     return aC.getInfo();
     }
  Public String enterAddress(string address) {
  String s = aC.enterAddress(address);
  Return s;
  }
  …
}
```

实现对应类：



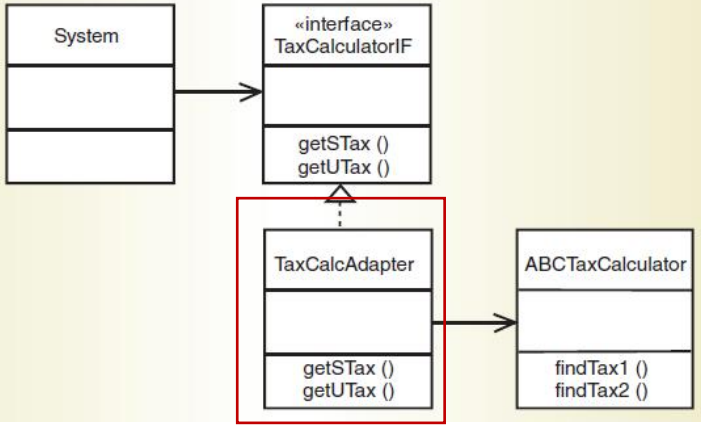

```
Public class Customer {
        Public class cusInfo{
  Public int CustID;
  Public string name;
  Public string phone;
  Public string email;
 }
 Private CustTable table = new CustTable();
 Private cusInfo = new cusInfo();
 Public static Customer createNewCustomer(string name, string phone,
string email) {
        Customer Obj=new Customer();
        Obj.setInfo(string name, string phone, string email);
        Return obj;
 }
 Public void setInfo(string name, string phone, string email) {
  cusInfo.name = name;
  cusInfo.phone = phone;
  cusInfo.email = email;
  cusInfo.CustID = Table.insert(this);
 }
 Public cusInfo getInfo(){
        Return cusInfo;
 }
 Public string enterAddress(string addr)
 {
        Address aAdd = Address.createAddress(addr);
        Return aAdd.insert(this);
 }
 …
}
```

实现

- 设计模式：
  - Adapter（适配器）： 为了应对类的修改

Example:

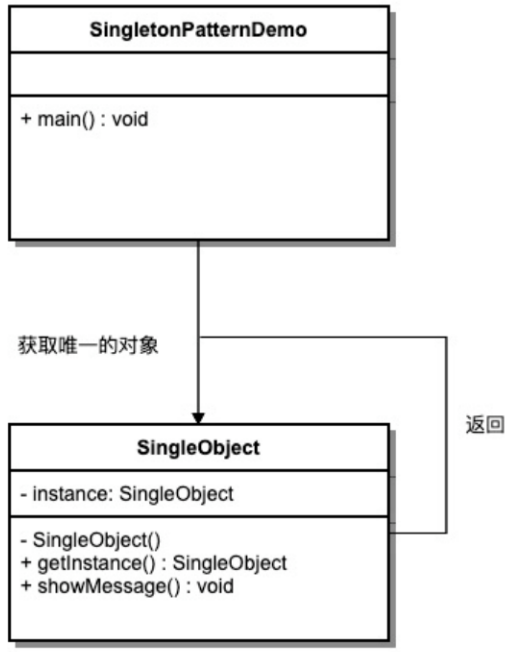
There are several places in the RMO system where class libraries were purchased to provide special processing. These purchased libraries provide specialized services such as tax calculations and shipping and postage rates. From time to time, these service libraries are updated with new versions. Sometimes a service library is even replaced with one from an entirely different vendor. The RMO systems staff applies protection from variations and indirection design principles by placing an adapter in front of each replaceable class.



© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

实现

- 设计模式
  - Singleton（单例）： 系统中有且只有一个单例对象



Example:

In RMO's system, the connection to the database is made through a class called Connection. However, for efficiency, we want each desktop system to open and connect to the database only once, and to do so as late as possible. Only one instance of Connection—that is, only one connection to the database—is desired. The Connection class is coded as a singleton. The following coding example is similar to C# and Java:

```
Class Connection
{
private static Connection conn = null;
public synchronized static getConnection ( )
      {
        if (conn == null) {
         conn = new Connection ( );}
        return conn;
      }
}
```

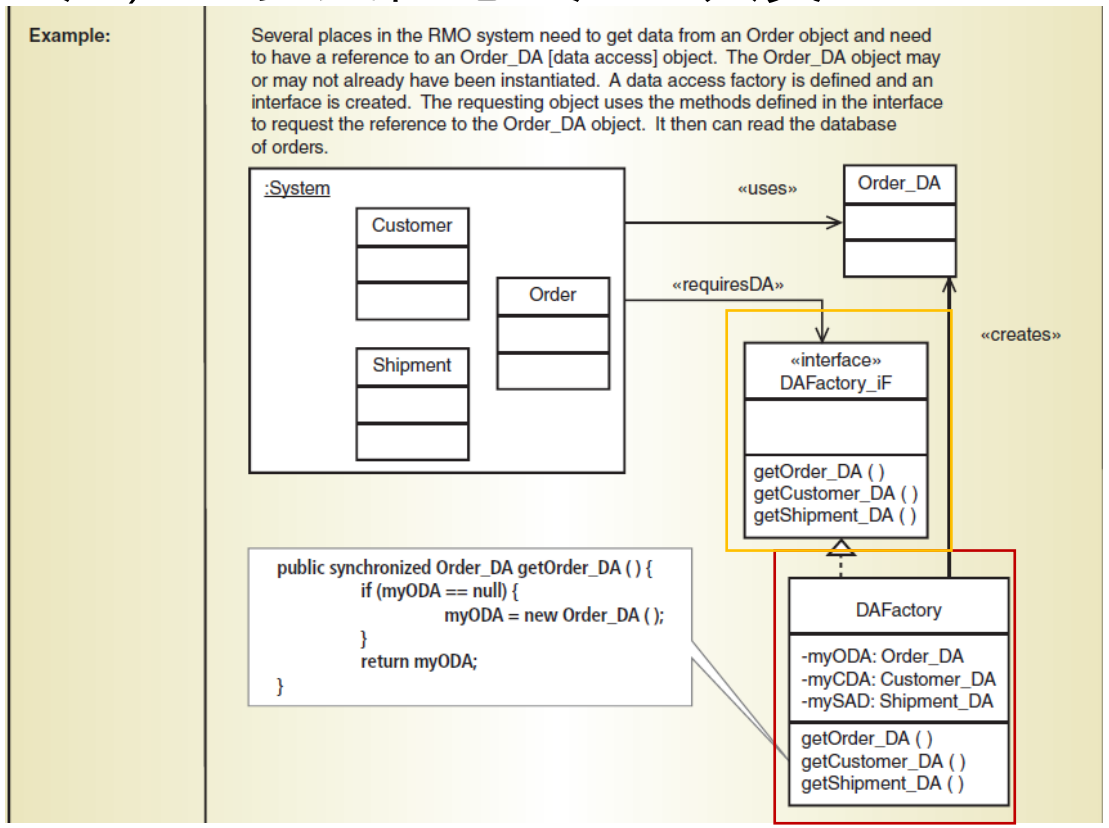
Another example of a singleton pattern is a utilities class that provides services for the system, such as a factory pattern. Because the services are for the entire system, it causes confusion if multiple classes provide the same services.

An additional example might be a class that plays audio clips. Since only one audio clip should be played at one time, the audio clip manager will control that. However, for this to work, there must be only one instance of the audio clip manager.

© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

实现

- 设计模式
  - Factory（工厂）：批量化地生产工具类



© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

- 【应用】现有支付宝支付类Alipay和微信支付类Wechatpay，请撰写一个适配器，让客户能够通过第三方接口Payment实现支付宝支付和微信支付。

- 答案：为了解决该问题，首先定义系统标准支付接口：

```
// 系统标准支付接口
interface Payment {
    void pay(double amount);
}
```

其次，定义第三方支付类：

```
// 支付宝支付类，方法名和参数与标准接口不同
class Alipay {
    public void doPay(String orderId, double money) {
        System.out.println("使用支付宝支付 " + money + " 元，订单号：" + orderId);
    }
}

// 微信支付类，方法名和参数与标准接口不同
class WechatPay {
    public void executePayment(double amount, String transactionId) {
        System.out.println("使用微信支付 " + amount + " 元，交易号：" + transactionId);
    }
}
```

- 第三，创建适配器类：

```
// 支付宝适配器
class AlipayAdapter implements Payment {
    private Alipay alipay;

    public AlipayAdapter(Alipay alipay) {
        this.alipay = alipay;
    }

    @Override
    public void pay(double amount) {
        // 生成订单号并调用支付宝的支付方法
        String orderId = System.currentTimeMillis() + "";
        alipay.doPay(orderId, amount);
    }
}
```

```
// 微信支付适配器
class WechatPayAdapter implements Payment {
    private WechatPay wechatPay;

    public WechatPayAdapter(WechatPay wechatPay) {
        this.wechatPay = wechatPay;
    }

    @Override
    public void pay(double amount) {
        // 生成交易号并调用微信的支付方法
        String transactionId = System.currentTimeMillis() + "";
        wechatPay.executePayment(amount, transactionId);
    }
}
```

- 客户端可以用下面的代码使用该适配器：

```
public class OrderService {
    public static void main(String[] args) {
        // 使用支付宝支付
        Alipay alipay = new Alipay();
        Payment alipayAdapter = new AlipayAdapter(alipay);
        alipayAdapter.pay(100.0);

        // 使用微信支付
        WechatPay wechatPay = new WechatPay();
        Payment wechatAdapter = new WechatPayAdapter(wechatPay);
        wechatAdapter.pay(200.0);

        // 系统可以统一调用标准支付接口，无需关心底层使用的是哪种支付方式
    }
}
```

输出结果为：

```
使用支付宝支付 100.0 元，订单号：17123456789O1
使用微信支付 200.0 元，交易号：1712345678902
```

大纲

- 设计模式
- 需求分析
- 设计
- 实现
- **测试**

测试

- 白盒测试
  - 基本路径覆盖
- 黑盒测试
  - 设计测试用例

© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

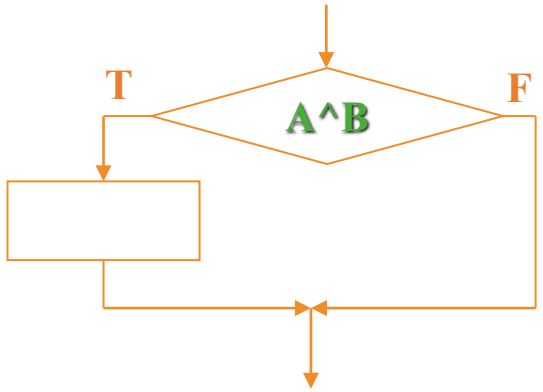
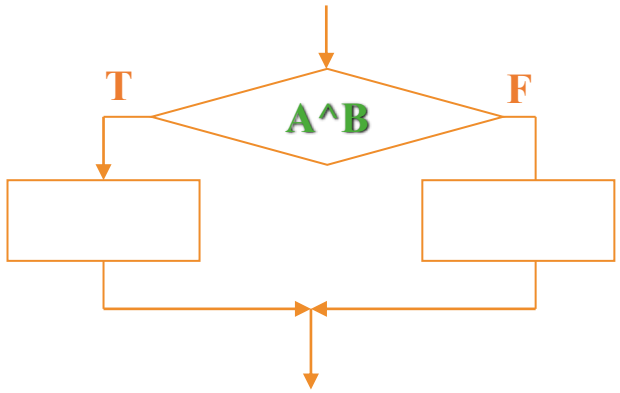
白盒测试用例的设计

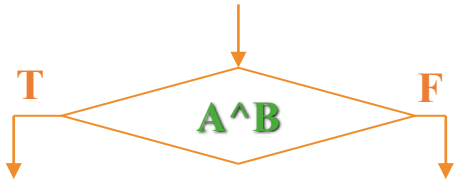
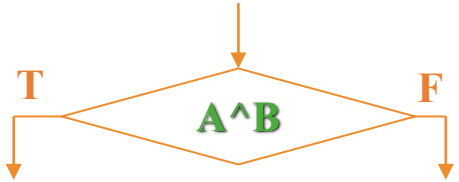
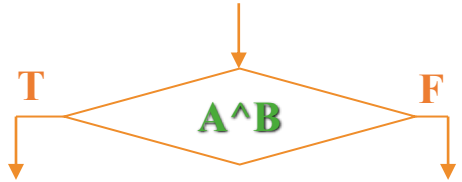
一、逻辑覆盖法

逻辑覆盖测试方法通常采用流程图来设计测试用例，它考察的重点是图中的判定框，因为这些判定通常是与选择结构有关或是与循环结构有关，是决定程序结构的关键成分。

逻辑覆盖测试的5种标准

发现错误的能力	标　准	含　　义
1(弱)	语句覆盖	每条语句至少执行一次
2	判定覆盖	每一判定的每个分支至少执行一次
3	条件覆盖	每一判定中的每个条件，分别按“真”、“假”至少各执行一次
4	判定/条件覆盖	同时满足判定覆盖和条件覆盖的要求
5 (强)	条件组合覆盖	求出判定中所有条件的各种可能组合值，每一可能的条件组合至少执行一次

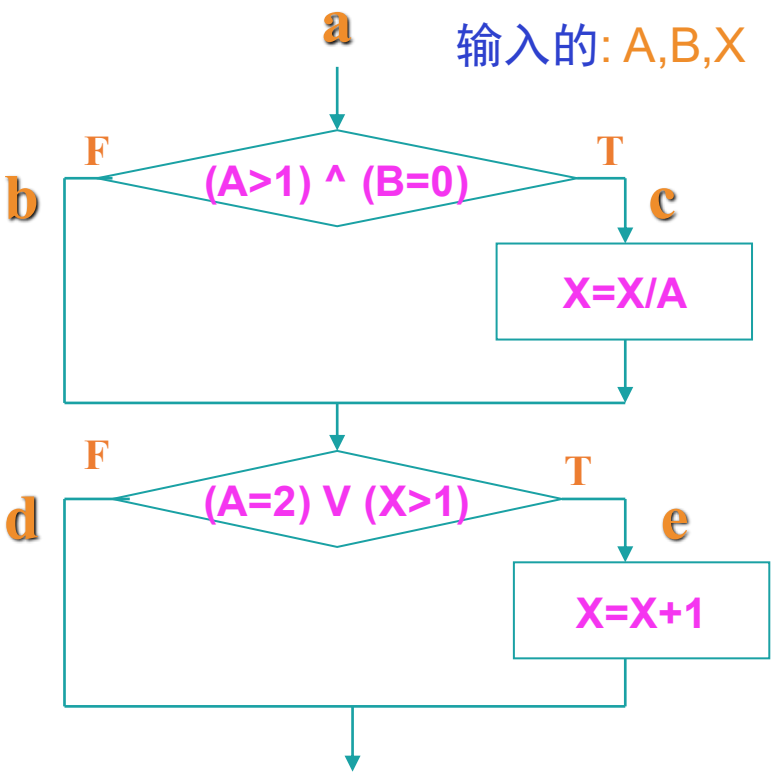
覆盖标准	程序结构举例	测试用例 应满足的条件
语句覆盖		A^B=.T.
判定覆盖		A^B=.T. A^B=.F.

覆盖标准	程序结构举例	测试用例应 满足的条件
条件覆盖		A=.T.　A=.F. B=.T.　B=.F.
判定/条件 覆盖		A^B=.T. ，A^B=.F. A=.T.　A=.F.　B=.T.　B=.F.
条件组合 覆盖		A=.T.　^　B=.T. A=.T.　^　B=.F. A=.F.　^　B=.T A=.F.　^　B=.F.

语句覆盖

ace (L1)	abd (L2)	abe (L3)	acd (L4)

“∧”是且的意思,“∨”是或的意思



满足语句覆盖的测试用例如下

2，0，4

2，0，3

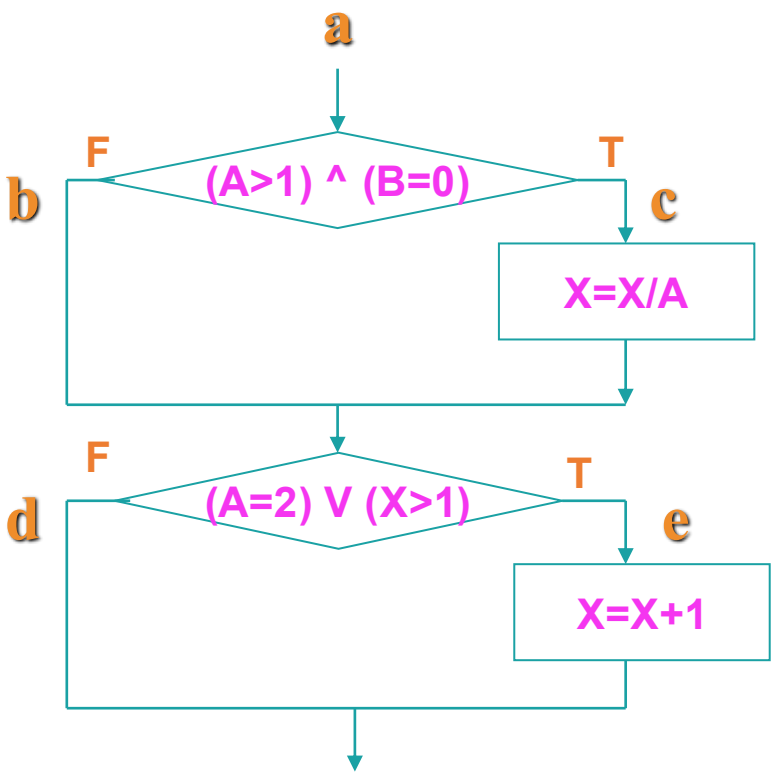
a -- c -- e

L1

问题：上下的“∧”“∨”调换顺序无法检测出来

判定覆盖(分支覆盖)

所谓的判定覆盖就是设计若干个测试用例，运行被测程序，使得程序中每个判断的取**真**分支和取**假**分支至少经历一次。



取“真”分支
测试用例如下

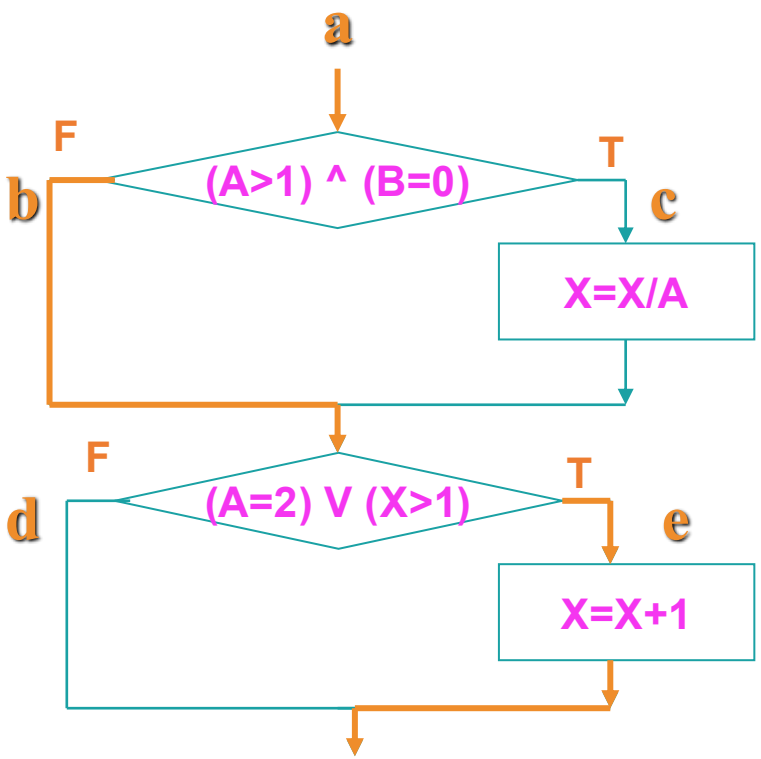
2，0，4
2，0，3

1，1，1
1，1，1

取“假”分支
测试用例如下

a -- c -- e L1

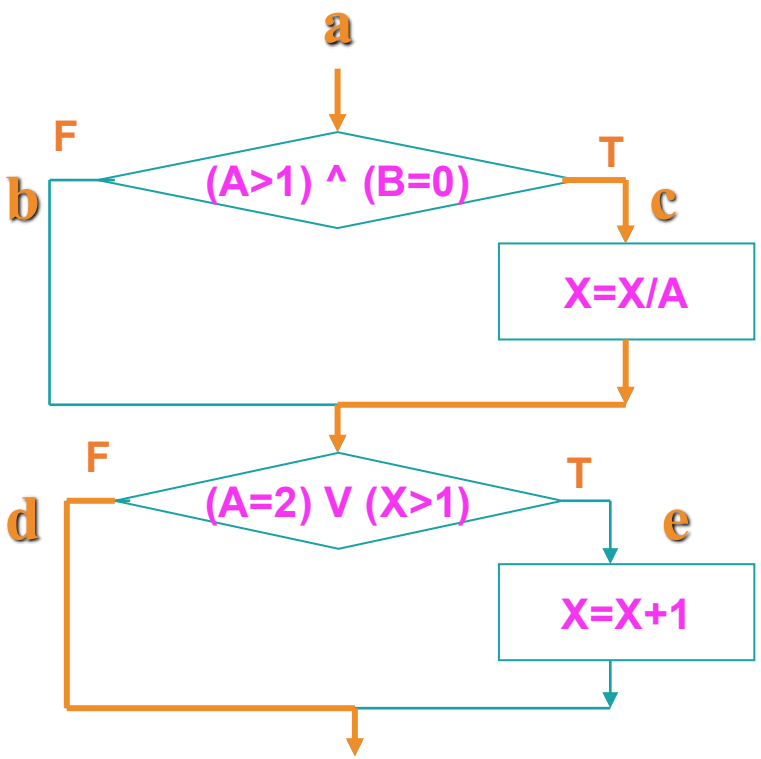
a -- b -- d L2



a -- b -- e L3

取“真假”分支测试用例如下

2，1，1
2，1，2



取“真假”分支
测试用例如下

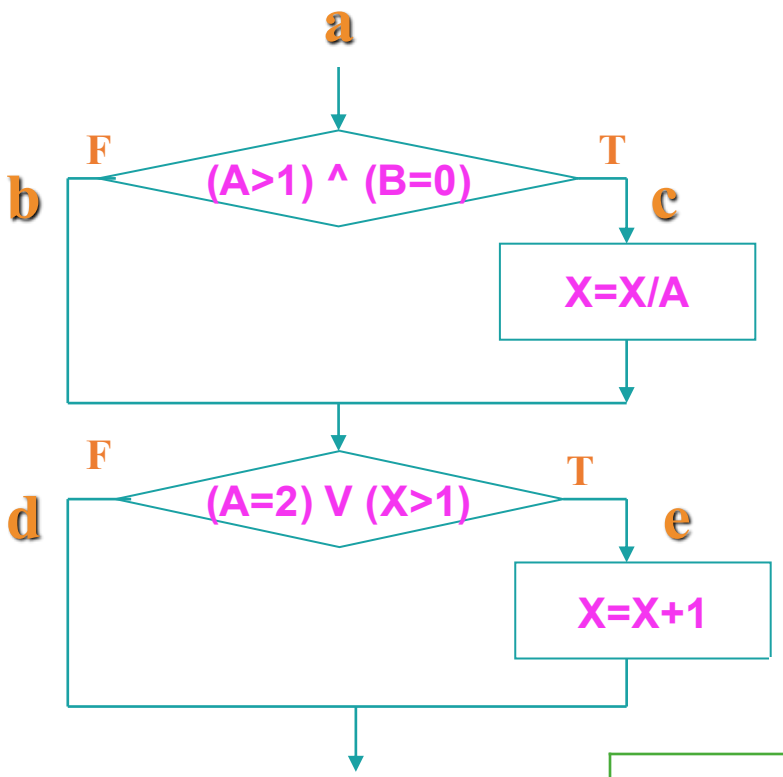
3，0，3
3，0，1

a -- c -- d L4

问题：若把图中第二个判断中的条件 **X>1 错写成 X<1** 那么再利用上面的两组测试用例，仍能得到同样的结果

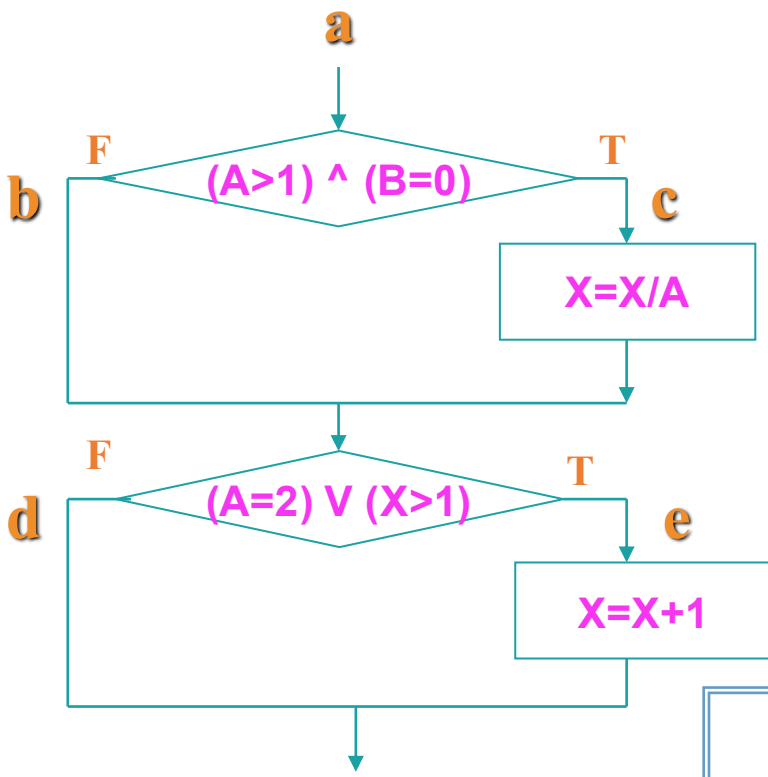
条件覆盖

所谓的条件覆盖就是设计若干个测试用例，运行被测程序，使得程序中每个判断的每个条件的可能取值至少执行一次。



设条件的取值标记

判断	条件	取真值	取假值
判断（一）	A>1	T1	$\overline{T1}$
	B=0	T2	$\overline{T2}$
判断（二）	A=2	T3	$\overline{T3}$
	X>1	T4	$\overline{T4}$

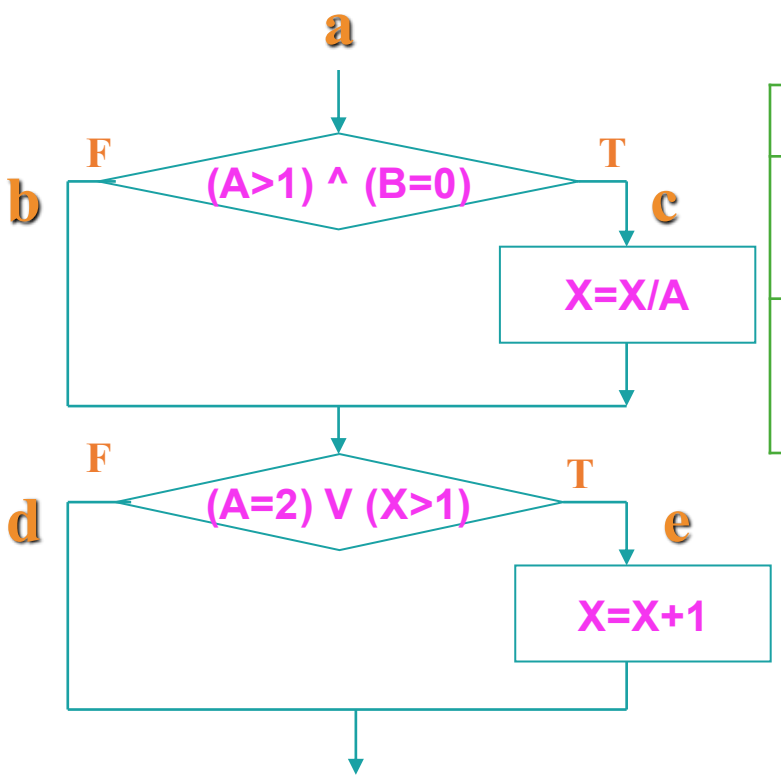


设条件的取值标记

判断	条件	取真值	取假值
判断（一）	A>1	T1	$\overline{T1}$
	B=0	T2	$\overline{T2}$
判断（二）	A=2	T3	$\overline{T3}$
	X>1	T4	$\overline{T4}$

条件覆盖可选取的
（第一组测试用例）如下表

测试用例	通过路径	条件取值	覆盖分支
(2,0,4),(2,0,3)	ace(L1)	T1 T2 T3 T4	c,e
(1,0,1),(1,0,1)	abd(L2)	$\overline{T1}$ T2 $\overline{T3}$ $\overline{T4}$	b,d



设条件的取值标记

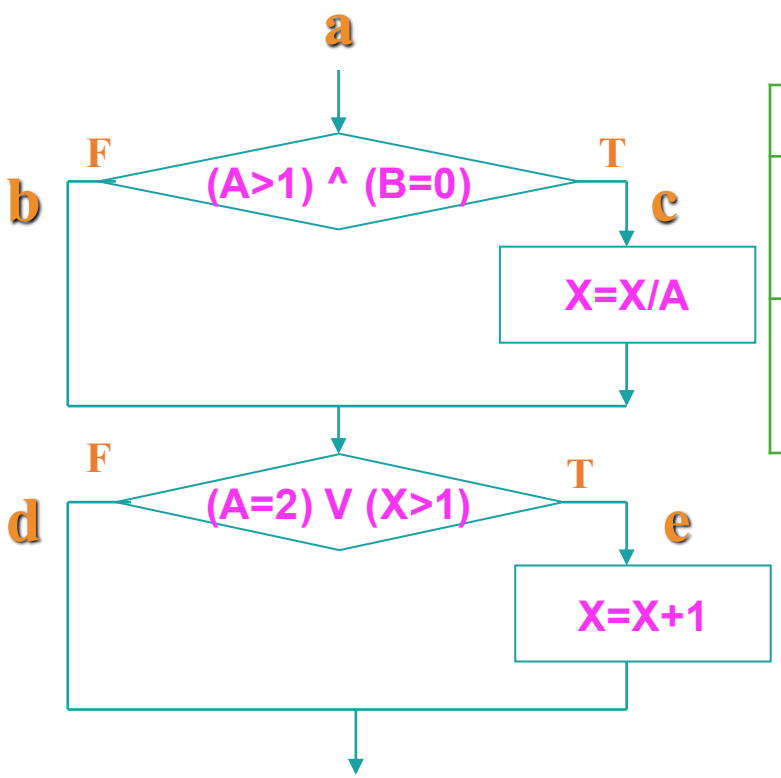
判断	条件	取真值	取假值
判断（一）	A>1	T1	$\overline{T1}$
	B=0	T2	$\overline{T2}$
判断（二）	A=2	T3	$\overline{T3}$
	X>1	T4	$\overline{T4}$

条件覆盖可选取的（第二组测试用例）如下表

测试用例	通过路径	条件取值	覆盖分支
(1,0,3),(1,0,4)	abe(L3)	$\overline{T1}$ T2 $\overline{T3}$ T4	b,e
(2,1,1),(2,1,2)	abe(L3)	T1 $\overline{T2}$ T3 $\overline{T4}$	b,e

判定/条件覆盖

所谓的判定/条件覆盖就是设计足够的测试用例，使得 判断中每个条件的所有可能取值至少执行一次， 同时每个判断本身的所有可能判断结果至少执行一次。



设条件的取值标记

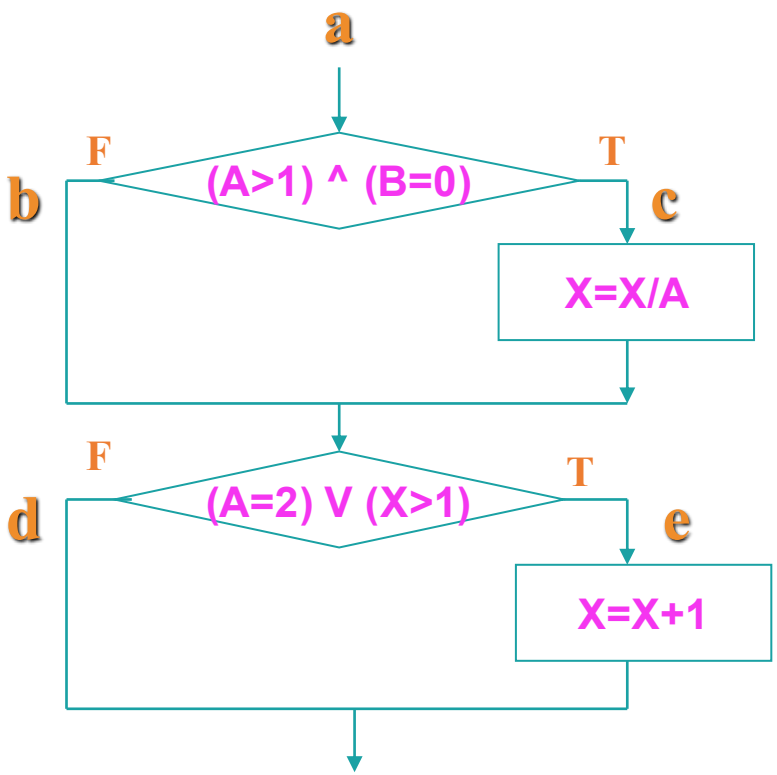
判断	条件	取真值	取假值
判断（一）	A>1	T1	$\overline{T1}$
	B=0	T2	$\overline{T2}$
判断（二）	A=2	T3	$\overline{T3}$
	X>1	T4	$\overline{T4}$

判定/条件覆盖可选取的 测试用例 如下表

测试用例	通过路径	条件取值	覆盖分支
(2,0,4),(2,0,3)	ace(L1)	T1 T2 T3 T4	c,e
(1,1,1),(1,1,1)	abd(L2)	$\overline{T1}$ $\overline{T2}$ $\overline{T3}$ $\overline{T4}$	b,d

条件组合覆盖

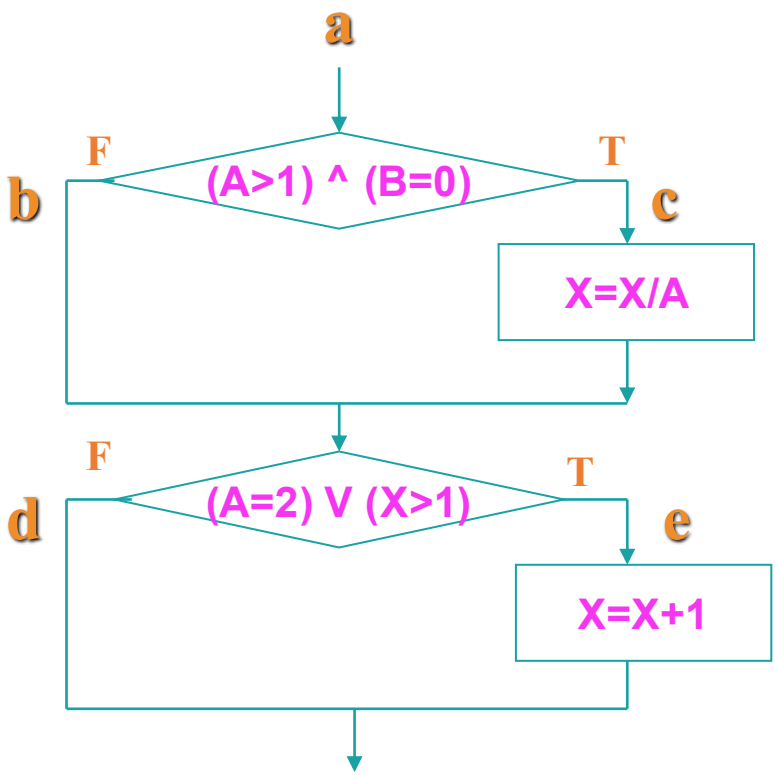
所谓的 条件组合覆盖就是设计足够的测试用例，运行被测程序，使得 每个判断的所有可能的条件取值组合至少执行一次。



设条件的取值标记

判断	条件	取真值	取假值
判断（一）	A>1	T1	$\overline{T1}$
	B=0	T2	$\overline{T2}$
判断（二）	A=2	T3	$\overline{T3}$
	X>1	T4	$\overline{T4}$

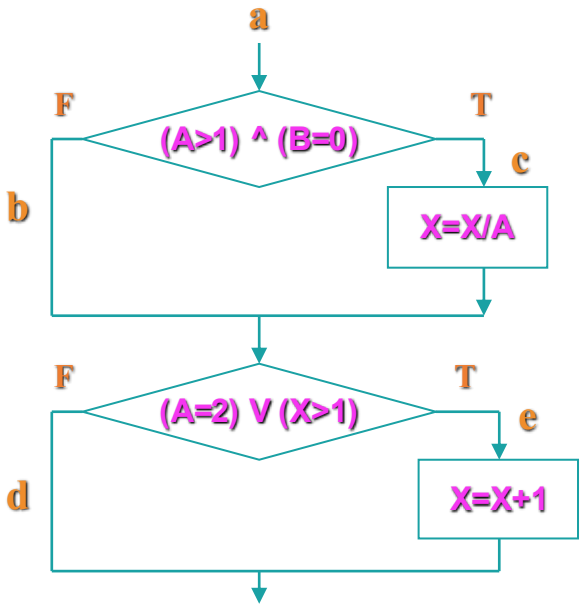
条件	标记	第一个判断取真假分支
A>1 B=0	T1 T2	① 取真分支
A>1 B ≠ 0	T1 $\overline{T2}$	② 取假分支
A ≯ 1 B = 0	$\overline{T1}$ T2	③ 取假分支
A ≯ 1 B ≠ 0	$\overline{T1}$ $\overline{T2}$	④ 取假分支



设条件的取值标记

判断	条件	取真值	取假值
判断（一）	A>1	T1	$\overline{T1}$
	B=0	T2	$\overline{T2}$
判断（二）	A=2	T3	$\overline{T3}$
	X>1	T4	$\overline{T4}$

条件	标记	第二个判断取真假分支
A=2 X>1	T3 T4	⑤ 取真分支
A=2 X≯1	T3 $\overline{T4}$	⑥ 取真分支
A≠2 X>1	$\overline{T3}$ T4	⑦ 取真分支
A≠2 X≯1	$\overline{T3}$ $\overline{T4}$	⑧ 取假分支



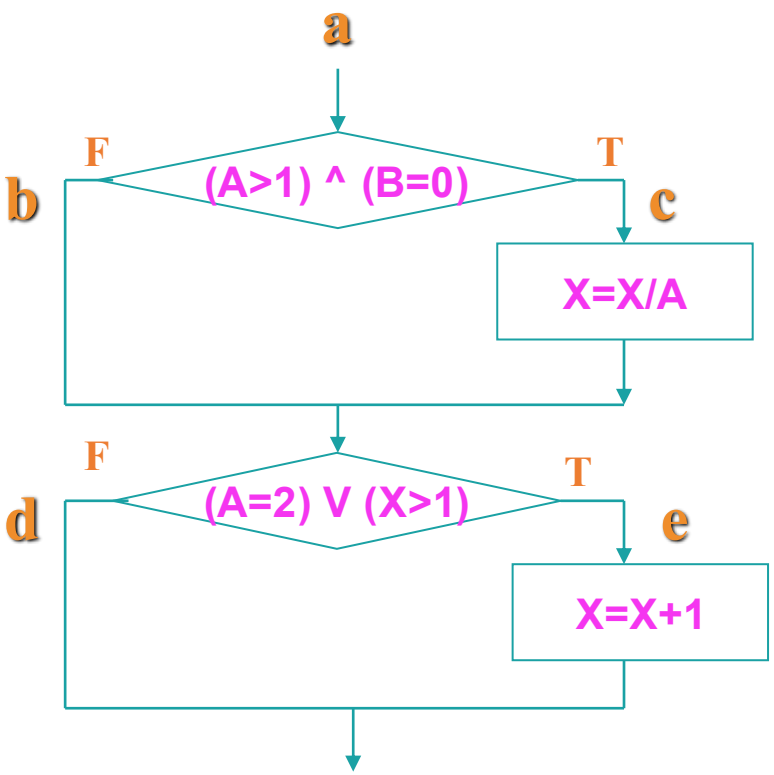
条件	标记	第一个判断取真假分支
A>1 B=0	T1 T2	① 取真分支
A>1 B≠0	$T1\ \overline{T2}$	② 取假分支
A≯1 B=0	$\overline{T1}\ T2$	③ 取假分支
A≯1 B≠0	$\overline{T1}\ \overline{T2}$	④ 取假分支

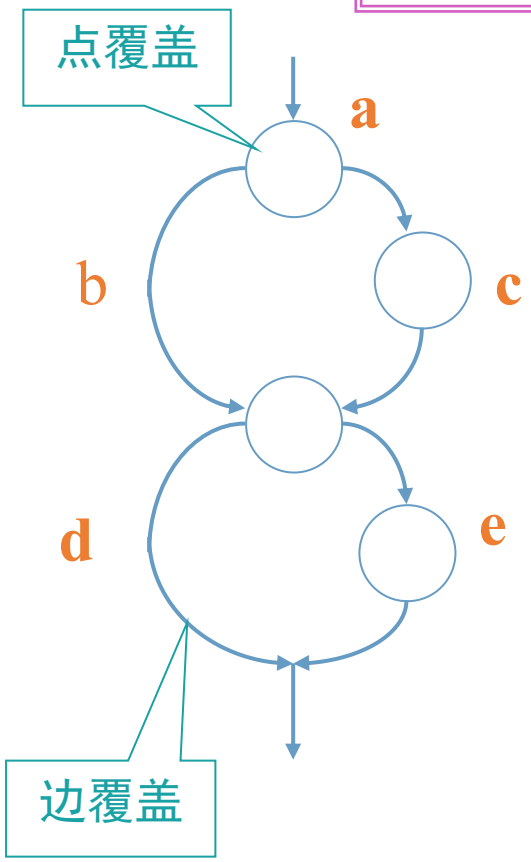
测试用例	通过路径	条件取值	覆盖组合号
(2,0,4),(2,0,3)	ace L1	T1 T2 T3 T4	①, ⑤
(2,1,1),(2,1,2)	abe L3	$T1\ \overline{T2}\ T3\ \overline{T4}$	②, ⑥
(1,0,3),(1,0,4)	abe L3	$\overline{T1}\ T2\ \overline{T3}\ T4$	③, ⑦
(1,1,1),(1,1,1)	abd L2	$\overline{T1}\ \overline{T2}\ \overline{T3}\ \overline{T4}$	④, ⑧

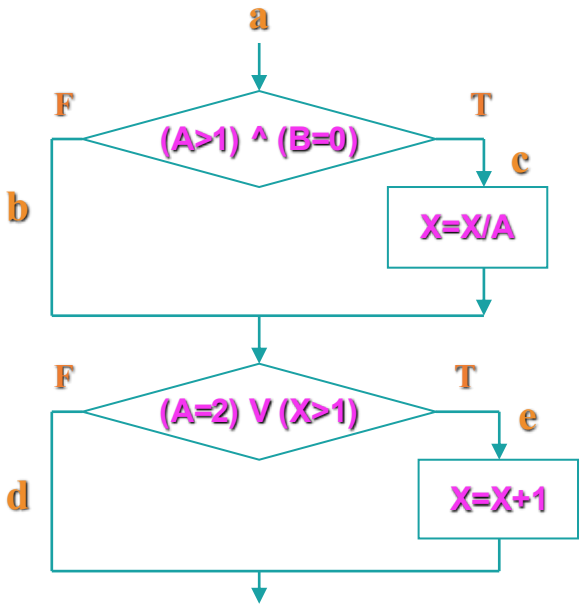
二、路径测试法

路径测试就是设计足够的测试用例，覆盖程序中每一条可能的程序执行路径至少测试一次，如果程序中含有循环(在程序图中表现为环)则每个循环至少执行一次。

a-c-e L1
a-b-d L2
a-b-e L3
a-c-d L4







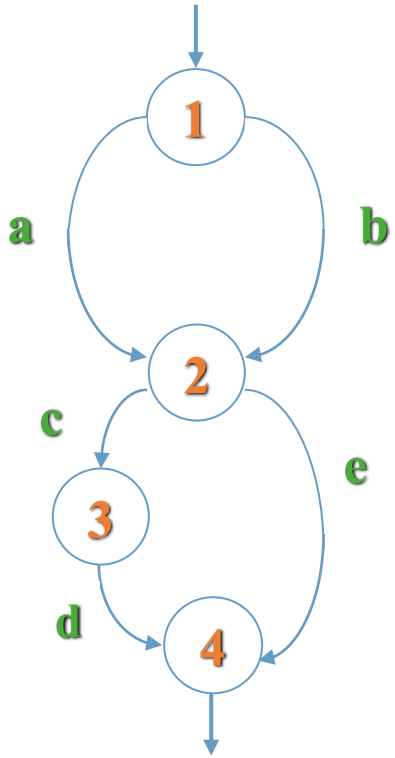
判断	条件	取真值	取假值
判断（一）	A>1	T1	$\overline{T1}$
	B=0	T2	$\overline{T2}$
判断（二）	A=2	T3	$\overline{T3}$
	X>1	T4	$\overline{T4}$

路径测试可选取的 测试用例 如下表

测试用例	通过路径	条件取值
(2,0,4),(2,0,3)	ace L1	T1 T2 T3 T4
(1,1,1),(1,1,1)	abd L2	$\overline{T1}$ $\overline{T2}$ $\overline{T3}$ $\overline{T4}$
(1,1,2),(1,1,3)	abe L3	$\overline{T1}$ $\overline{T2}$ $\overline{T3}$ T4
(3,0,3),(3,0,1)	acd L4	T1 T2 $\overline{T3}$ $\overline{T4}$

练习

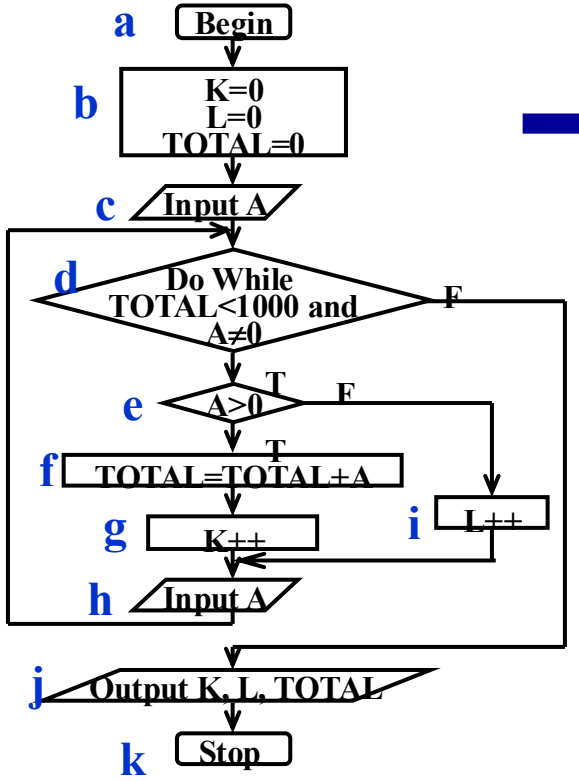
根据左侧给出的程序图，请填写下列表



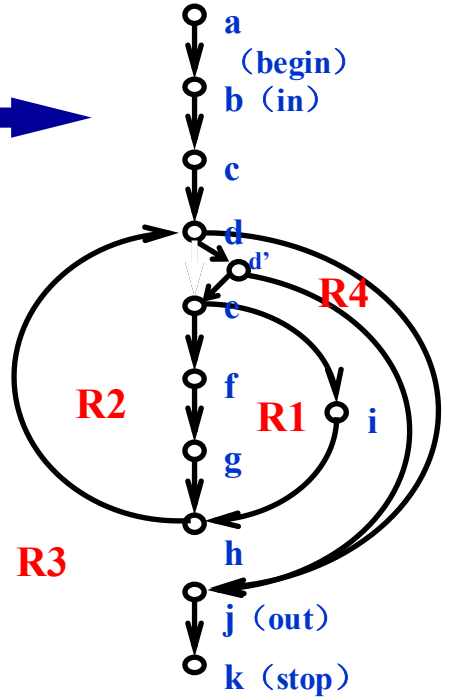
测试路径	覆盖结点/边	覆盖标准
a-c-d	①②③④	点覆盖
a-c-d, b-e	a,b,c,d,e	边覆盖
a-c-d,b-e a-e,b-c-d	①②③④ a,b,c,d,e	路径覆盖

测试

- 白盒测试
  - 画出有向图，计算测试用例上限（环形复杂度）
    - 注意: 判断中的or和and都要拆开，如左图的d拆为d和d'



有向图



环形复杂度=

- # of regions （连通域个数）
- *E*（边）– *N*（点）+ 2
- *P* + 1（判断节点个数+1）

测试

- 黑盒测试方法
  - 等价类划分
  - 边界值分析

© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

测试

- 黑盒测试方法
  - **等价类划分**

    等价类表示输入条件的一组有效或无效状态，因此没有特别的理由选择一个元素而不是另一个元素作为类代表。

    - 等价类划分的Guidelines
    - 如果输入条件指定了一个范围，则定义一个有效的和两个无效的等价类。
    - 如果输入条件需要特定的值，则定义一个有效和两个无效的等价类。
    - 如果输入条件指定了集合的成员，则定义一个有效等价类和一个无效等价类。
    - 如果输入条件是布尔值，则定义一个有效等价类和一个无效等价类。

© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

练习

请利用等价分类法为以下提供的内容设计测试用例

某工厂公开招工，规定报名者年龄应在16～35周岁之间（到1995年6月30日为止），即出生年月不早于1960年7月，不晚于1979年6月。 报名程序具有自动检验输入数据的功能。如出生年月不在上述范围内， 将拒绝接受，并显示“年龄不合格”等出错信息。

请试用等价分类法， 设计出生年月的等价分类表 。

1、划分出生年月等价分类表

假定已知出生年月是由 6 位数字字符表示，前4 位代表年，后2 位代表月，则可以划分为 3 个有效等价类和 7 个无效等价类。

输入数据	有效等价类	无效等价类
出生年月	① 6位有效数字字符	② 有非数字字符 ③ 少于6个数字字符 ④ 多于6个数字字符
对应数值	⑤ 196007-197906	⑥ < 196007 ⑦ > 197906
月份对应数值	⑧ 在1-12之间	⑨ 等于 “0” ⑩ >12

2、设计有效等价类需要的测试用例

输入数据	有效等价类	无效等价类
出生年月	① 6位有效数字字符	② 有非数字字符 ③ 少于6个数字字符 ④ 多于6个数字字符
对应数值	⑤ **196007-197906**	⑥ **< 196007** ⑦ **> 197906**
月份对应数值	⑧ 在1-12之间	⑨ 等于 “0” ⑩ >12

测试数据	期望结果	测试范围
197011	输入有效	①、⑤、⑧

输入数据	有效等价类	无效等价类
出生年月	① 6位有效数字字符	② 有非数字字符 ③ 少于6个数字字符 ④多于6个数字字符
对应数值	⑤ 196007-197906	⑥ < 196007 ⑦ > 197906
月份对应数值	⑧ 在1-12之间	⑨ 等于 “0” ⑩ >12

3、为每一个无效等价类至少设计一个测试用例

测试数据	期望结果	测试范围
MAY,70	输入无效	② 有非数字字符
19705	输入无效	③ 少于6个数字字符
1968011	输入无效	④ 多于6个数字字符
196008	年龄不合格	⑥ <196007
195512	年龄不合格	⑦ >197906
196200	输入无效	⑨ 等于 “0”
197222	输入无效	⑩ >12

测试

- 黑盒测试方法
  - **边界值分析**
    - 如果输入条件指定了一个由值a和b限定的范围，那么测试用例应该包括a和b，刚好高于和低于a和b的值。
    - 如果输入条件指定了值的数量，那么测试用例应该执行最小值和最大值，以及刚好高于和低于最小值和最大值的值。
    - 将指导方针1和2应用于输出条件，测试用例应该被设计成产生最小和最大的输出报告。
    - 如果内部程序数据结构有边界(例如，大小限制)，一定要测试边界。

© 2012 Cengage Learning. All Rights Reserved. This edition is intended for use outside of the U.S. only, with content that may be different from the U.S. Edition.
May not be scanned, copied, duplicated, or posted to a publicly accessible website, in whole or in part.

二、边界值分析法

采用边界值分析法来选择测试用例，可使得被测程序能在边界值及其附近运行，从而更有效地暴露程序中潜藏的错误。

为了接受年龄合格的报名者则程序中可能设有语句为：

```
If (196007 < value(birthdate) <= 197906)
   Then read(birthday)
Else write “invalid age！”
```

输入 等价类	测试用例说明	测试数据	期望结果
出 生 年 月	1 个数字字符 5 个数字字符 7 个数字字符 有1个非数字字符 全是非数字字符	**5** **19705** **1968011** **19705** **AUGUST**	输入无效
对应数值	**35 周岁** **16 周岁**	**196007** **197906**	合格年龄
	>35 周岁 **<16周岁**	**196006** **197907**	不合格年龄
月份 对应数值	月份值为 1 月 月份值为 12 月	**196701** **197412**	输入有效
	月份值 < 1 月份值 >12	**196700** **197413**	输入无效

【选择】内聚是衡量模块内部聚合能力的量度。内聚越高，说明模块内各成分彼此联系的程度(　)
A．越松散
B．彼此相等
C．越紧密
D．没有联系

- 答案：C

【选择】为了提高测试的效率，应该（ ）
A．随机地选取测试数据
B．取一切可能的输入数据作为测试数据
C．在完成编码以后制定软件的测试计划
D．选择发现错误可能性大的数据作为测试数据

- 答案：D

【选择】单元测试的测试对象是（）。

A. 系统

B. 程序模块。

C. 模块接口

D. 系统功能

- 答案： B

【选择】软件生命周期中所花费用最多的阶段是（ ）
A．详细设计　B．软件编码　C．软件测试　　D．软件维护

- 答案：　D

【选择】使用白盒测试方法时，确定测试数据应根据_____和指定的覆盖标准。
A．程序的内部逻辑
B．程序的复杂结构
C．使用说明书
D．程序的功能

- 答案：A

【选择】通过对软件的测试，可以证明的是（）。
A．软件的正确性
B．错误不存在
C．错误存在
D．软件不含有隐患

- 答案：C

【选择】软件测试方法中，黑盒、白盒测试法是常用的方法，其中白盒测试主要用于测试（　　）

A、结构合理性

B、软件外部功能

C、程序正确性

D、程序内部逻辑

- 答案：　D

【选择】以下不属于白盒测试技术的是（ ）
A. 逻辑覆盖
B. 基本路径测试
C. 循环覆盖测试
D. 等价类划分

- 答案： D

【选择】为了提高测试的效率，应该____。
A．随机地选取测试数据
B．取一切可能的输入数据作为测试数据库
C．在完成编码后制定软件的测试计划
D．选择发现错误可能性大的数据作为测试数据

- 答案： D

【选择】软件测试中设计测试实例（test case）主要由输入数据和____两部分组成。

A．测试规则　　B．测试计划

C．预期输出结果　　D．以往测试记录分析

- 答案：　C

【选择】成功的测试是指_____。

A．运行测试实例后未发现错误项　B．发现程序的错误

C．证明程序正确　　D．改正程序的错误

- 答案：　B

【简答】请阐述单元测试、集成测试和系统测试常使用的测试类型(黑盒/白盒)

- 答案:
- 单元测试：白盒
- 集成测试：白盒或黑盒
- 系统测试：黑盒

【简答】什么是存根模块？什么是驱动模块？

- 答案：
- 存根模块：用以模拟被测模块运行过程中所调用模块或是被测模块下层模块的替代程序
- 驱动模块：用以模拟被测模块的上级调用模埠，或向被测模块输以数据输出一些必要信息，调用被测模块

【选择】月收入<=800元者免税，现用输入数800元和801元测试程序，则采用的是____方法。
A．边界值分析
B．条件覆盖
C．错误推测
D．等价类

- 答案： A

【选择】采用白盒法测试模块（程序），应根据____。
A．内部逻辑结构
B．算法复杂度
C．外部功能特性
D．支撑环境

- 答案： A

【选择】采用黑盒法测试程序是根据____。
A．程序的逻辑
B．程序的功能说明
C．程序中的语句
D．程序中的数据

- 答案： B

【选择】在软件测试中，确认（验收）测试主要用于发现____阶段的错误。
A．软件计划
B．需求分析
C．软件设计
D．编码

- 答案： B

【选择】用黑盒法设计测试用例时采用的方法包括_____。
A、判定覆盖法
B、条件覆盖法
C 、等价类划分法
D、路径分析法

- 答案： C

【应用】某“调整工资”处理模块接受一个“职称”的变量，根据职称的不同（助教，讲师，副教授，教授）作不同的处理，其中若是助教还必须输入工龄，只有工龄超过两年才能调整工资。请用等价类划分法设计测试用例。

输入条件	合理等价类	不合理等价类
职称	①教授 ②副教授 ③讲师	⑤四种职称之外任意一种
职称兼工龄	④助教兼工龄大于2年	⑥助教兼工龄等于两年 ⑦助教兼工龄小于两年

【应用】考虑以下程序设计：输入a，b两个数，a的取值范围在（-100,0），b的取值范围在（0，100）之间，输出的结果应为a/b，请用等价类划分和边界值分析两类方法设计测试用例

答案：

1）根据等价类划分：

根据a的范围，可以划分成三个等价类：一个有效等价类（例如-50），两个无效等价类（50或者-150）

根据b的范围，可以划分成三个等价类：一个有效等价类（例如50），两个无效等价类（-50或者150）

因此可以设计下面几个测试用例

测试用例	预期结果	说明
a=-50,b=50	有效输入	a和b均为有效等价类
a=50，b=50	报错	a为无效，b为有效
a=-150,b=50	报错	a为无效，b为有效
a=-50，b=-50	报错	a为有效，b为无效
a=-50，b=150	报错	a为有效，b为无效
a=50，b=150	报错	a，b均为无效

【应用】考虑以下程序设计：输入a，b两个数，a的取值范围在（-100,0），b的取值范围在（0，100）之间，输出的结果应为a/b，请用等价类划分和边界值分析两类方法设计测试用例

答案：

2）根据边界值分析：

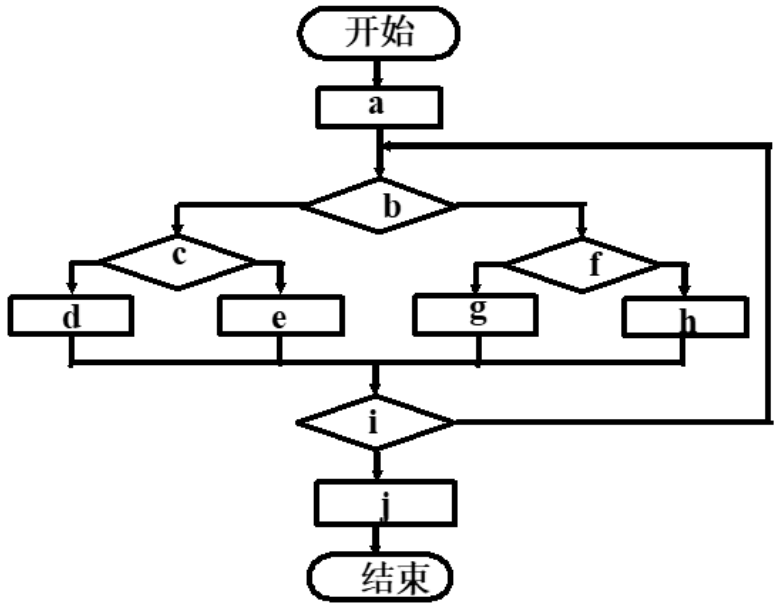
a的边界值为-101，-100，-99，-1,0,1

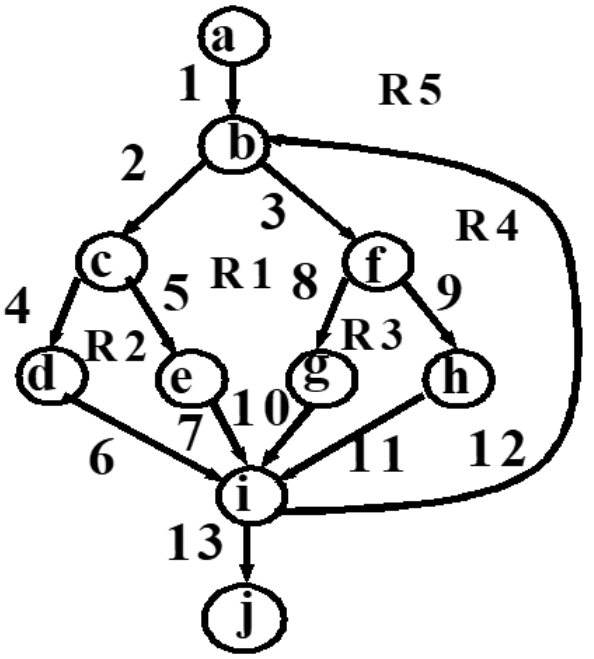
b的边界值为-1,0,1,99,100，101

因此可以设计下面几个测试用例

测试用例	预期结果	说明
a=-99,b=1	有效输入	a，b均不越界
a=-1，b=1	有效输入	a，b均不越界
a=-101,b=1	报错	a越界
a=-100,b=1	报错	a越界
a=0,b=1	报错	a越界
a=1,b=1	报错	a越界
a=-99,b=-1	报错	b越界
a=-99,b=0	报错	b越界
a=-99,b=99	报错	b越界
a=-99,b=100	报错	b越界
a=-99,b=101	报错	b越界
a=-101,b=101	有效输入	a，b均越界

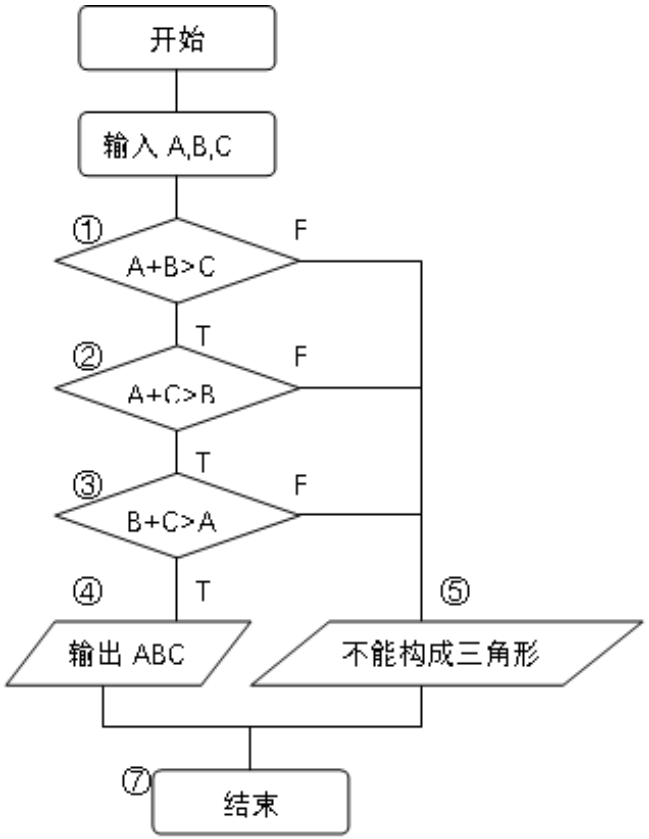
【应用】画出与图1所示的程序流程图对应的程序图，并试计算其循环复杂度（ ）

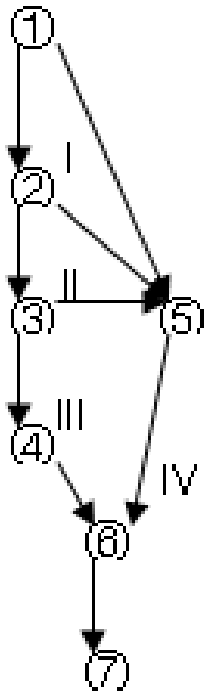




- 答案：循环复杂度：V（G）=E –N +2=13 –10 +2=5

【应用】输入三整数,判断是否构成三角形,如构成三角形,则输出三条边的值,否则输出”不能构成三角形”. 要求:1.用程序流程图表示该问题的算法; 2.计算程序复杂度; 3.设计路径覆盖的测试用例。





程序复杂度为4

4条独立路径为:

路径:

1. ①—②—③—④—⑥—⑦
2. ①—②—⑤—⑥—⑦
3. ①—⑤—⑥—⑦
4. ①—②—③—⑤—⑥—⑦

测试用例

路径	测试用例（A，B，C）	结果
1	A=3, B=4, C=5	A=3, B=4, C=5
2	A=5, B=8, C=2	不能构成三角形
3	A=2, B=2, C=5	不能构成三角形
4	A=5, B=2, C=2	不能构成三角形

【应用】某学籍管理系统需要输入学生代码。合法的代码符合以下3条规则:

（1）由6位数字构成;

（2）前四位是入学年份(2010—2014);

（3）后2位是专业类别(01-12)。

采用等价类划分法进行测试，请列出测试等价类表、设计测试用例。

“学生代码”输入条件的等价类表

输入条件	有效等价类	无效等价类
学生代码的类型及长度	6 位数字字符（1）	有非数字字符 (4) 少于 6 个数字字符 (5) 多于 6 个数字字符 (6)
入学年份的范围	在 2010-2014 之间（2）	小于 2010（7） 大于 2014（8）
专业类别的范围	在 01-12 之间（3）	小于 01（9） 大于 12（10）